



UNIVERSITÀ DEGLI STUDI DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Appunti di Algoritmi e Strutture dati

Lezioni, e Simulazioni d'esame

A cura di:

Mirco Negri

*Appunti redatti per gli studenti del DISI.
Eventuali errori possono essere segnalati su [GitHub](#).*

ottobre 2026

Indice

1	Introduzione	4
1.1	Problema Computazionale e Algoritmo	4
1.2	Cenni Storici	4
1.2.1	Le Origini	4
1.2.2	Origine del Nome	4
1.3	Problemi e Algoritmi: Esempi Introduttivi	4
1.3.1	Il Problema del Minimo	4
1.3.2	Il Problema della Ricerca	5
1.4	Lo Pseudo-codice	5
1.5	Valutazione degli Algoritmi	7
1.5.1	Efficienza	7
1.5.2	Correttezza	9
1.6	Considerazioni conclusive	10
2	Analisi di Algoritmi	10
2.1	Modelli di Calcolo	10
2.1.1	Obiettivi e Motivazioni	10
2.1.2	La Nozione di Complessità	10
2.1.3	Dimensione dell'Input	11
2.1.4	Definizione di Tempo (a Livello di Istruzioni Elementari)	11
2.1.5	Il Concetto di Modello di Calcolo	12
2.1.6	La Macchina di Turing	12
2.1.7	La Random Access Machine (RAM)	12
2.1.8	Esempio: Tempo di Calcolo di <code>min()</code>	13
2.1.9	Esempio: Tempo di Calcolo di <code>binarySearch()</code>	13
2.1.10	Ordini di Complessità	14
2.2	Notazione Asintotica	15
2.2.1	Le Notazioni O , Ω , Θ	15
2.2.2	Esempi: Vero o Falso?	16
2.3	Proprietà della Notazione Asintotica	18
2.3.1	Proprietà delle Notazioni	18
2.3.2	Proprietà delle Funzioni più Comuni	19
2.3.3	Le Notazioni o , ω	22
2.4	Analisi Ammortizzata	22
2.4.1	Introduzione	22
2.4.2	Contatore Binario	23
2.4.3	Vettori Dinamici	26
2.4.4	Conclusioni	31
2.5	Complessità degli Algoritmi vs Complessità dei Problemi	32
2.5.1	Introduzione	32
2.5.2	Moltiplicare Numeri Complessi	33
2.5.3	Sommare Numeri Binari	33
2.5.4	Moltiplicare Numeri Binari	34
2.5.5	Algoritmi vs Problemi	38
2.6	Algoritmi di Ordinamento	38
2.6.1	Tipologia dell'Input e dell'Analisi	38
2.6.2	Il Problema dell'Ordinamento	39

2.6.3	Selection Sort	39
2.6.4	Insertion Sort	40
2.6.5	Merge Sort	41
2.6.6	Limite Inferiore per l'Ordinamento	44
2.6.7	Counting Sort	45
2.6.8	Ordine Asintotico e Costanti	46
2.7	Ricorrenze e Analisi per Livelli	48
2.7.1	Introduzione	48
2.7.2	Analisi per Livelli	48
2.8	Ricorrenze: Metodo di Sostituzione	51
2.8.1	Introduzione	51
2.8.2	Primo Esempio: $T(n) = T(\lfloor n/2 \rfloor) + n$	51
2.8.3	Un Tentativo Sbagliato: $T(n) = T(n-1) + n$	52
2.8.4	Difficoltà Matematica: un Termine di Ordine Inferiore	53
2.8.5	Problemi con i Casi Base	54
2.8.6	Together Now: le Due Difficoltà Insieme	55
2.8.7	Riassumendo	56
2.9	Ricorrenze: Teoremi degli Esperti (Master Theorem)	57
2.9.1	Introduzione	57
2.9.2	Master Theorem, Versione Ridotta	57
2.9.3	Master Theorem, Versione Estesa	59
2.9.4	Master Theorem con Fattori Logaritmici	60
2.9.5	Oltre il Master Theorem	60
2.10	Ricorrenze: Altre Ricorrenze Comuni	60
2.11	Back to Algorithms!: la Massima Sottosequenza Contigua	61
2.12	Strutture Dati Astratte	65
2.12.1	Definizioni	65
2.12.2	Sequenza	66
2.12.3	Insiemi	67
2.12.4	Dizionari	68
2.12.5	Alberi e Grafi	69
2.12.6	Pila	71
2.12.7	Coda	73
2.13	Alberi: Introduzione	76
2.13.1	Esempi	76
2.13.2	Definizioni	76
2.14	Alberi Binari	77
2.14.1	Introduzione	77
2.14.2	Implementazione	78
2.14.3	Visite	79
2.15	Alberi Generici	80
2.15.1	Visite	81
2.15.2	Implementazione	82
2.16	Insiemi e Dizionari: Riassunto Finale	84
2.16.1	Introduzione	84
2.16.2	Insiemi Realizzati con Vettori Booleani	84
2.16.3	BitSet	86
2.16.4	Insiemi Realizzati con Liste/Vettori Non Ordinati	86

2.16.5	Insiemi Realizzati con Liste/Vettori Ordinati	86
2.16.6	Insiemi – Strutture Dati Complesse	87
2.16.7	Insiemi – Riassunto	87
2.16.8	Complessità in Python	88
2.17	Alberi Binari di Ricerca	89
2.17.1	Introduzione	89
2.17.2	Ricerca	90
2.17.3	Minimo-Massimo	91
2.17.4	Successore-Predecessore	91
2.17.5	Inserimento	92
2.17.6	Cancellazione	93
2.17.7	Costo Computazionale	95
2.18	Alberi Binari di Ricerca Bilanciati	95
2.18.1	Altezza degli Alberi Binari di Ricerca	95
2.18.2	Alberi Binari di Ricerca Bilanciati	95
2.18.3	Definizioni	96
2.18.4	Esempi	96
2.18.5	Memorizzazione	97
2.18.6	Altezza Nera	97
2.18.7	Esempi: Colorazioni Diverse	97
2.18.8	Esempio: Questo Albero Può Essere un Albero Red-Black?	98
2.18.9	Rotazioni	100

1 Introduzione

1.1 Problema Computazionale e Algoritmo

Definizione: Problema Computazionale

Dati un dominio di input e un dominio di output, un **problema computazionale** è rappresentato dalla relazione matematica che associa ogni elemento del dominio di input ad uno o più elementi del dominio di output.

Definizione: Algoritmo

Dato un problema computazionale, un **algoritmo** è un procedimento effettivo, espresso tramite un insieme di passi elementari ben specificati in un sistema formale di calcolo, che risolve il problema in tempo finito.

1.2 Cenni Storici

1.2.1 Le Origini

- Il **Papiro di Rhind** (o di Ahmes, 1850 a.C.) contiene il cosiddetto “algoritmo del contadino” per la moltiplicazione.
- Algoritmi di tipo numerico furono studiati anche da matematici **babilonesi** e **indiani**.
- Gli algoritmi rimasti in uso fino a tempi recenti furono studiati dai matematici **greco** più di 2000 anni fa:
 - l'algoritmo di **Euclide** per il massimo comune divisore;
 - algoritmi geometrici (calcolo di tangenti, sezioni di angoli, ...).

1.2.2 Origine del Nome

- **Abu Abdullah Muhammad bin Musa al-Khwarizmi**: matematico, astronomo, astrologo e geografo, nato in Uzbekistan, attivo a Baghdad. Dal suo nome deriva la parola **algoritmo**.
- *Algoritmi de numero indorum*: traduzione latina di un testo arabo ormai perduto, che introdusse i numeri indiani (arabi) nel mondo occidentale. Dal numero arabo *sifr* = 0: *zephirum* → *zevero* → *zero*, ma anche *cifra*.
- *Al-Kitab al-muhtasar fi hisab al-gabr wa-l-muqabala* (820 d.C.), l'opera più famosa di al-Khwarizmi, tradotta in latino come *Liber algebrae et almucabala*: dal titolo deriva la parola **algebra**.

1.3 Problemi e Algoritmi: Esempi Introduttivi

1.3.1 Il Problema del Minimo

Il minimo di un insieme S è l'elemento di S che è minore o uguale ad ogni elemento di S :

$$\min(S) = a \iff \exists a \in S : \forall b \in S : a \leq b$$

Algoritmo (informale): per trovare il minimo di un insieme, si confronta ogni elemento con tutti gli altri; l'elemento che è minore di tutti gli altri è il minimo.

1.3.2 Il Problema della Ricerca

Sia $A = a_0, a_1, \dots, a_{n-1}$ una sequenza di dati ordinati e distinti ($a_0 < a_1 < \dots < a_{n-1}$). Ricercare la posizione di un dato v in A significa restituire un indice i tale che $0 \leq i \leq n-1$ se v è presente in posizione i , oppure -1 se v non è presente:

$$\text{lookup}(A, v) = \begin{cases} i & \text{se } \exists i \in \{0, \dots, n-1\} : a_i = v \\ -1 & \text{altrimenti} \end{cases}$$

Algoritmo (informale): per trovare un valore v nella sequenza A , si confronta v con tutti gli elementi di A in sequenza e si restituisce la posizione corrispondente; si restituisce -1 se nessun elemento corrisponde.

1.4 Lo Pseudo-codice

Le descrizioni informali appena viste pongono due problemi aperti:

- **Descrizione:** sono espresse in linguaggio naturale, quindi imprecise; serve un linguaggio più formale.
- **Valutazione:** non è chiaro se esistano algoritmi “migliori” di quelli proposti; va definito un concetto di “migliore”.

Per la descrizione si adotta uno **pseudo-codice**: una notazione il più possibile formale ma indipendente dal linguaggio di programmazione. Va posta attenzione anche al livello di dettaglio: descrivere un algoritmo con passaggi vaghi come “amalgamare il tutto” o “far riposare” (per restare in tema di ricette) è impreciso quanto scrivere semplicemente “prepara il piatto”.

Convenzioni sintattiche.

```

a = b           % assegnamento
a += b          % equivalente a: a = a + b
a -= b          % equivalente a: a = a - b
a, b = b, a     % scambio, equivalente a: tmp=a; a=b; b=tmp

T[] A = new T[0 ... n-1] = {0}           % array 1D inizializzato a 0
T[][] B = new T[0 ... n-1][0 ... m-1]    % array 2D

int, real, boolean, int    % tipi di base [sic - così nella slide originale]

and, or, not                % operatori logici
==, !=, <=, >=              % operatori di confronto
+, -, *, /, floor(x), ceil(x), log, x^2, ... % operatori aritmetici

iif(condizione, v1, v2)    % operatore ternario: v1 se vera, v2 se falsa

```

```

if condizione then istruzione
if condizione then istruzione1 else istruzione2

while condizione do istruzione

foreach elemento in insieme do istruzione

return
% commento

```

Il ciclo for è zucchero sintattico per un while equivalente:

```

for indice = estremoInf to estremoSup do istruzione
% e' equivalente a:
int indice = estremoInf
while indice <= estremoSup do
    istruzione
    indice = indice + 1

for indice = estremoSup downto estremoInf do istruzione
% e' equivalente a:
int indice = estremoSup
while indice >= estremoInf do
    istruzione
    indice = indice - 1

```

Per le strutture dati simili a oggetti:

```

struct RECTANGLE
    int lunghezza
    int altezza

RECTANGLE r = new RECTANGLE
r.altezza = 10
delete r
r = nil

```

Primi esempi in pseudo-codice.

```

int min(int[] S, int n)
    for i = 0 to n-1 do
        boolean isMin = true
        for j = 0 to n-1 do
            if i != j and S[j] < S[i] then
                isMin = false
        if isMin then
            return S[i]

```

Per ogni elemento $S[i]$, il ciclo interno verifica se esiste un altro elemento più piccolo; se non esiste, $S[i]$ è il minimo.

```

int lookup(int[] S, int n, int v)
    for i = 0 to n-1 do
        if S[i] == v then
            return i
    return -1

```

Scandisce linearmente il vettore, restituendo l'indice del primo elemento uguale a v , oppure -1 se non trovato.

Nota: a questo punto le slide riportano una vignetta (xkcd) sull'incoerenza nell'uso di indici a partire da 0 o da 1, con una battuta attribuita a Donald Knuth: la convenzione scelta va sempre dichiarata esplicitamente e mantenuta coerente all'interno dello stesso algoritmo.

1.5 Valutazione degli Algoritmi

Un algoritmo va valutato secondo due assi ortogonali: è **efficiente**? è **corretto**?

1.5.1 Efficienza

Già nel 1864, riflettendo sulla futura Analytical Engine, Charles Babbage scriveva:

“[...] By what course of calculation can these results be arrived at by the machine in the shortest time?”

— Charles Babbage, *Passages from the Life of a Philosopher*, 1864.

Definizione: Complessità di un Algoritmo

La **complessità** di un algoritmo è l'analisi delle risorse impiegate per risolvere un problema, in funzione della dimensione e dalla tipologia dell'input.

Nota: una definizione più formale, basata sulla notazione O , Ω , Θ , verrà introdotta nella prossima lezione (Analisi di Algoritmi).

Le risorse principali considerate sono:

- **Tempo:** tempo impiegato per completare l'algoritmo.
 - Misurato con il cronometro?
 - Misurato contando il numero di operazioni rilevanti?
 - Misurato contando il numero di operazioni elementari?
- **Spazio:** quantità di memoria utilizzata.
- **Banda:** quantità di bit spediti (negli algoritmi distribuiti).

Come misurare il tempo. Una prima idea è il **wall-clock time**, cioè il tempo effettivamente impiegato per eseguire l'algoritmo. Dipende però da troppi parametri estranei all'algoritmo: bravura del programmatore, linguaggio utilizzato, codice generato dal compilatore, processore e memoria (cache, primaria, secondaria), sistema operativo e processi concorrenti. Serve una misura più astratta: si conta il **numero di operazioni rilevanti**, cioè le operazioni che caratterizzano lo scopo dell'algoritmo (i confronti $<$ nel problema del minimo, i confronti $==$ nel problema della ricerca).

Il problema del minimo. L'algoritmo `min` visto sopra effettua $n^2 - n$ confronti (per ogni elemento scandisce l'intero vettore). Si può fare di meglio tenendo traccia di un solo candidato durante un'unica scansione:

```
int min(int[] A, int n)
    int minSoFar = A[0]
    for i = 1 to n-1 do
        if A[i] < minSoFar then
            minSoFar = A[i]
    return minSoFar
```

Questa versione effettua solo $n - 1$ confronti.

Il problema della ricerca: la ricerca binaria. L'algoritmo `lookup` visto sopra effettua, nel caso peggiore, n confronti. Sfruttando il fatto che il vettore è **ordinato**, si può analizzare l'elemento centrale (indice m) del sottovettore considerato:

- se $A[m] = v$, il valore è stato trovato;
- se $v < A[m]$, si cerca nella metà di sinistra;
- se $A[m] < v$, si cerca nella metà di destra.

Esempio: ricerca del valore 50 nel vettore

1, 3, 7, 8, 12, 16, 21, 34, 37, 45, 50, 67, 68, 72, 88

- Passo 1: sottovettore $[0, 14]$, $m = 7 \Rightarrow A[7] = 34 < 50$: si cerca a destra;
- Passo 2: sottovettore $[8, 14]$, $m = 11 \Rightarrow A[11] = 67 > 50$: si cerca a sinistra;
- Passo 3: sottovettore $[8, 10]$, $m = 9 \Rightarrow A[9] = 45 < 50$: si cerca a destra;
- Passo 4: sottovettore $[10, 10]$, $m = 10 \Rightarrow A[10] = 50$: **trovato**.

Questa tecnica è la **ricerca binaria** (*binary search*):

```
int bsRec(int[] A, int v, int start, int end)
    if start > end then
        return -1                                % Vettore vuoto
    else
        int m = floor((start + end) / 2)
        if A[m] == v then
            return m                              % Trovato
        else if A[m] < v then
            return bsRec(A, v, m+1, end)          % Sottovettore di destra
        else
            return bsRec(A, v, start, m-1)        % Sottovettore di sinistra
```

Il numero di confronti passa da n (algoritmo naïf) a $2\lceil \log n \rceil$ (ricerca binaria).

Cenni storici: il metodo di bisezione per le radici di una funzione è di Bolzano (1817); la prima menzione della ricerca binaria si deve a John Mauchly, progettista di ENIAC (1946); la prima versione corretta su vettori di dimensione arbitraria è di Derrick Henry Lehmer (1960). Donald Knuth osserva:

“Although the basic idea of binary search is comparatively straightforward, the details can be surprisingly tricky.”

— Donald Knuth, *The Art of Computer Programming*.

Attenzione all’overflow: calcolare l’indice centrale come $(\text{start} + \text{end}) / 2$ può causare overflow se la somma supera il massimo intero rappresentabile. La versione sicura è:

```
int m = floor(start + (end - start) / 2)
```

1.5.2 Correttezza

Definizione: Invariante

Un **invariante** è una condizione sempre vera in un certo punto del programma.

- **Invariante di ciclo:** condizione sempre vera all’inizio dell’iterazione di un ciclo.
- **Invariante di classe:** condizione sempre vera al termine dell’esecuzione di un metodo della classe.

L’invariante di ciclo permette di dimostrare la correttezza di un algoritmo iterativo tramite tre passi, analoghi a una dimostrazione per induzione:

1. **Inizializzazione** (caso base): la condizione è vera alla prima iterazione del ciclo;
2. **Conservazione** (passo induttivo): se la condizione è vera prima di un’iterazione, rimane vera al suo termine (quindi anche all’inizio dell’iterazione successiva);
3. **Conclusione:** quando il ciclo termina, l’invariante deve implicare la correttezza dell’algoritmo.

Esempio: nell’algoritmo `min` efficiente, l’invariante è:

“All’inizio di ogni iterazione del ciclo `for`, la variabile `minSoFar` contiene il minimo parziale degli elementi $A[0, \dots, i - 1]$.”

Nota: le slide enunciano l’invariante e la struttura della dimostrazione (Inizializzazione / Conservazione / Conclusione) ma non ne riportano lo svolgimento esplicito, presumibilmente discusso a lezione.

La stessa tecnica, adattata a un’induzione sulla dimensione n dell’input, dimostra la correttezza degli algoritmi **ricorsivi** come `bsRec`:

1. **Caso base:** $n = 0$ (cioè `start > end`);
2. **Ipotesi induttiva:** la proprietà è vera per ogni $n' < n$;
3. **Passo induttivo:** si dimostra che la proprietà vale per n , assumendo l’ipotesi induttiva sulla chiamata ricorsiva.

Nota: anche qui le slide riportano solo lo schema della dimostrazione, senza svolgerla per esteso.

1.6 Considerazioni Conclusive

Oltre a efficienza e correttezza, un algoritmo (o la sua implementazione) può essere valutato secondo altre proprietà: semplicità, modularità, manutenibilità, espandibilità, robustezza, ...

- sono **secondarie** in un corso di algoritmi e strutture dati;
- sono **fondamentali** in un corso di ingegneria del software.

Osservazione

Alcune di queste proprietà hanno un costo aggiuntivo in termini di prestazioni: il codice modulare introduce un costo di gestione delle chiamate, l'uso di un bytecode (es. Java) un costo di interpretazione. **Progettare algoritmi efficienti è un prerequisito per poter pagare questo costo.**

2 Analisi di Algoritmi

2.1 Modelli di Calcolo

2.1.1 Obiettivi e Motivazioni

Obiettivo: stimare la complessità in tempo, tramite:

- modelli di calcolo;
- esempi di valutazioni;
- ordini di complessità.

Perché?

- per stimare il tempo impiegato per un dato input;
- per stimare il più grande input gestibile in tempi ragionevoli;
- per confrontare l'efficienza di algoritmi diversi;
- per ottimizzare le parti più importanti.

2.1.2 La Nozione di Complessità

Definizione: Complessità

La **complessità** di un algoritmo è una funzione matematica non negativa (**funzione di costo**) che descrive l'andamento del tempo di calcolo o dello spazio utilizzato in relazione alla dimensione dell'input:

$$T : \text{"Dimensione dell'input"} \rightarrow \text{"Tempo di calcolo"}$$

$$S : \text{"Dimensione dell'input"} \rightarrow \text{"Spazio occupato"}$$

Questa è la versione rigorosa della nozione di complessità già introdotta informalmente nella Definizione 1.5.1 (Sezione 1): non più genericamente “risorse impiegate”, ma una funzione matematica esplicita T o S .

Dalla definizione nascono due domande, lasciate aperte:

- Come definire la **dimensione dell’input**?
 - Come misuriamo la dimensione di un vettore?
 - Come misuriamo la dimensione di un intero?
- Come misurare il **tempo**?

2.1.3 Dimensione dell’Input

Sequenze di elementi.

- La dimensione naturale è il **numero di elementi** della sequenza.
- Esempio: ricerca del minimo in un vettore di n elementi.

Valori interi.

- La dimensione naturale è il **numero di bit** necessari per rappresentare l’intero.
- Esempio: moltiplicazione di numeri binari lunghi n bit.

In molti casi...

- possiamo assumere che gli “elementi” siano rappresentati da un numero costante di bit;
- le due misure coincidono a meno di una costante moltiplicativa.

2.1.4 Definizione di Tempo (a Livello di Istruzioni Elementari)

Definizione: Tempo

Tempo $\equiv$ numero di **istruzioni elementari**. Un’istruzione si considera elementare se può essere eseguita in tempo “costante” dal processore.

Domanda aperta (discussione in aula): quali delle seguenti sono operazioni elementari?

- `a *= 2`
- `Math.cos(d)`
- `min(A, n)`

Nota didattica: applicando la definizione appena data, `a *= 2` è chiaramente a costo costante; `min(A, n)` **non** è invece elementare, perché il suo costo dipende dalla dimensione n del vettore; il caso di `Math.cos(d)` è più sottile, poiché dipende dall’implementazione hardware/software della funzione coseno (in pratica la si assume spesso costante). La slide lascia il quesito aperto come spunto di discussione, senza fornire una risposta ufficiale.

2.1.5 Il Concetto di Modello di Calcolo

Definizione: Modello di Calcolo

Un modello di calcolo è una **rappresentazione astratta** di un calcolatore, che deve soddisfare tre requisiti:

- **Astrazione**: deve permettere di nascondere i dettagli;
- **Realismo**: deve riflettere la situazione reale;
- **Potenza matematica**: deve permettere di trarre conclusioni “formali” sul costo.

Secondo l'articolo di Wikipedia *Model of computation*, esistono:

- tre grandi famiglie di modelli: **sequenziali**, **funzionali** e **concorrenti**;
- venti modelli di calcolo distinti (con sottomodelli).

2.1.6 La Macchina di Turing

Una **macchina ideale** che manipola i dati contenuti su un nastro di lunghezza infinita, secondo un insieme prefissato di regole. Ad ogni passo, la Macchina di Turing:

- legge il simbolo sotto la testina;
- **modifica** il proprio stato interno;
- **scrive** un nuovo simbolo nella cella;
- **muove** la testina a destra o a sinistra.

Esempio: data una regola $\delta(q, b) = (q', a, Right)$ e una testina posizionata su una cella contenente b , la macchina scrive a al posto di b , passa allo stato q' e sposta la testina di una cella a destra.

È un modello importante nello studio della **calcolabilità**, ma di livello troppo basso per gli scopi di questo corso.

2.1.7 La Random Access Machine (RAM)

Memoria:

- parole di w bit, indirizzate direttamente;
- accesso in tempo costante, indipendente dall'indirizzo.

Processore (singolo):

- set di istruzioni elementari simile a quelle reali: somme, sottrazioni, moltiplicazioni, operazioni logiche, etc.;
- istruzioni di controllo (salti, salti condizionati).

Costo delle istruzioni elementari:

- costante per operazioni su parole di macchina;
- per interi più lunghi di w bit, dipende dal numero di bit.

2.1.8 Esempio: Tempo di Calcolo di `min()`

Assunzioni: $n \geq 1$; si considera il caso peggiore (vettore strettamente decrescente); ogni istruzione ha un costo costante, potenzialmente diverso; ogni istruzione viene eseguita un certo numero di volte, dipendente da n .

Istruzione	Costo	# Volte
<code>int min = A[0]</code>	c_1	1
<code>for i = 1 to n-1 do</code>	c_2	n
<code>if A[i] < min then</code>	c_3	$n - 1$
<code>min = A[i]</code>	c_4	$n - 1$
<code>return min</code>	c_5	1

$$\begin{aligned}
 T(n) &= c_1 + c_2n + c_3(n - 1) + c_4(n - 1) + c_5 \\
 &= (c_2 + c_3 + c_4)n + (c_1 + c_5 - c_3 - c_4) \\
 &= an + b
 \end{aligned}$$

2.1.9 Esempio: Tempo di Calcolo di `binarySearch()`

Domanda: come si valuta il tempo di calcolo di una funzione **ricorsiva**? Si riconsideri l'algoritmo di ricerca binaria già visto nella Sezione 1 (sottosezione *Efficienza*).

Definizione: Relazione di Ricorrenza

Quando un algoritmo richiama se stesso su un input più piccolo, il suo costo $T(n)$ dipende dal costo della chiamata ricorsiva. Schema tipico:

$$T(n) = \begin{cases} \text{costo del caso base} & n \leq n_0 \\ \text{costo delle chiamate ricorsive} + \text{lavoro locale} & n > n_0 \end{cases}$$

La ricerca binaria fornisce il primo esempio concreto; metodi sistematici per analizzare queste relazioni verranno introdotti più avanti nel corso.

Per l'analisi dettagliata, il vettore viene suddiviso in due parti (Parte SX: $\lfloor (n - 1)/2 \rfloor$; Parte DX: $\lfloor n/2 \rfloor$) e i parametri `start`, `end` vengono rinominati i , j :

Istruzione	Costo	$\#(i > j)$	$\#(i \leq j)$
<code>if i > j then</code>	c_1	1	1
<code>return -1</code>	c_2	1	0
<code>int m = floor((i+j)/2)</code>	c_3	0	1
<code>if A[m] == v then</code>	c_4	0	1
<code>return m</code>	c_5	0	0
<code>else if A[m] < v then</code>	c_6	0	1
<code>return bsRec(A,v,m+1,j)</code>	$c_7 + T(\lfloor n/2 \rfloor)$	0	0/1
<code>else return bsRec(A,v,i,m-1)</code>	$c_7 + T(\lfloor (n - 1)/2 \rfloor)$	0	1/0

Assunzioni (caso peggiore):

- per semplicità, si assume n potenza di 2: $n = 2^k$;

- l'elemento cercato non è presente;
- ad ogni passo, si sceglie sempre la parte DX di dimensione $n/2$.

Caso $i > j$ ($n = 0$) : $T(n) = c_1 + c_2 = c$

Caso $i \leq j$ ($n > 0$) : $T(n) = T(n/2) + c_1 + c_3 + c_4 + c_6 + c_7 = T(n/2) + d$

Da cui la relazione di ricorrenza:

$$T(n) = \begin{cases} c & n = 0 \\ T(n/2) + d & n \geq 1 \end{cases}$$

Risolvendola tramite espansione:

$$\begin{aligned} T(n) &= T(n/2) + d \\ &= T(n/4) + 2d \\ &= T(n/8) + 3d \\ &\vdots \\ &= T(1) + kd \quad (n = 2^k \Rightarrow k = \log n) \\ &= T(0) + (k + 1)d \\ &= kd + (c + d) \\ &= d \log n + e. \end{aligned}$$

2.1.10 Ordini di Complessità

Le due analisi precedenti hanno prodotto due funzioni di complessità:

- Ricerca: $T(n) = d \log n + e$ — **logaritmica**;
- Minimo (efficiente): $T(n) = an + b$ — **lineare**.

Una terza funzione deriva dall'algoritmo *naïf* per il minimo (visto nella Sezione 1):

- Minimo (naïf): $T(n) = fn^2 + gn + h$ — **quadratica**.

Classi di complessità (tempo stimato al crescere di n):

$f(n)$	$n = 10$	$n = 10^2$	$n = 10^3$	$n = 10^4$	Tipo
$\log_2 n$	3.3×10^0	6.6×10^0	1.0×10^1	1.3×10^1	logaritmica
n	1.0×10^1	1.0×10^2	1.0×10^3	1.0×10^4	lineare
$n \log_2 n$	3.3×10^1	6.6×10^2	1.0×10^4	1.3×10^5	loglineare
n^2	1.0×10^2	1.0×10^4	1.0×10^6	1.0×10^8	quadratica
n^3	1.0×10^3	1.0×10^6	1.0×10^9	1.0×10^{12}	cubica
2^n	1.0×10^3	1.3×10^{30}	1.1×10^{301}	2.0×10^{3010}	esponenziale
$n!$	3.6×10^6	9.3×10^{157}	4.0×10^{2567}	2.8×10^{35659}	fattoriale

Curiosità: un aneddoto reale racconta come un bug nell'algoritmo di controllo degli aggiornamenti di Windows XP lo abbia reso, di fatto, esponenziale¹.

¹<http://m.slashdot.org/story/195683>

2.2 Notazione Asintotica

2.2.1 Le Notazioni O , Ω , Θ

Definizione: Notazione O (O-grande)

Sia $g(n)$ una funzione di costo; indichiamo con $O(g(n))$ l'insieme delle funzioni $f(n)$ tali per cui:

$$\exists c > 0, \exists m \geq 0 : f(n) \leq c g(n), \forall n \geq m$$

- Come si legge: $f(n)$ è “O grande” (*big-O*) di $g(n)$.
- Scrittura formale: $f(n) \in O(g(n))$.
- Scrittura convenzionale: $f(n) = O(g(n))$.
- $g(n)$ è un **limite asintotico superiore** per $f(n)$.
- $f(n)$ **cresce al più** come $g(n)$.

Definizione: Notazione Ω (Omega-grande)

Sia $g(n)$ una funzione di costo; indichiamo con $\Omega(g(n))$ l'insieme delle funzioni $f(n)$ tali per cui:

$$\exists c > 0, \exists m \geq 0 : f(n) \geq c g(n), \forall n \geq m$$

- Come si legge: $f(n)$ è “Omega grande” di $g(n)$.
- Scrittura formale: $f(n) \in \Omega(g(n))$.
- Scrittura convenzionale: $f(n) = \Omega(g(n))$.
- $g(n)$ è un **limite asintotico inferiore** per $f(n)$.
- $f(n)$ **cresce almeno quanto** $g(n)$.

Definizione: Notazione Θ (Theta)

Sia $g(n)$ una funzione di costo; indichiamo con $\Theta(g(n))$ l'insieme delle funzioni $f(n)$ tali per cui:

$$\exists c_1 > 0, \exists c_2 > 0, \exists m \geq 0 : c_1 g(n) \leq f(n) \leq c_2 g(n), \forall n \geq m$$

- Come si legge: $f(n)$ è “Theta” di $g(n)$.
- Scrittura formale: $f(n) \in \Theta(g(n))$.
- Scrittura convenzionale: $f(n) = \Theta(g(n))$.
- $f(n)$ **cresce esattamente** come $g(n)$.
- **Proprietà fondamentale:** $f(n) = \Theta(g(n))$ se e solo se $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$.

Nota grafica: la slide corrispondente mostra graficamente le tre curve $c_2g(n)$, $f(n)$ e $c_1g(n)$: da un certo punto m in poi, $f(n)$ resta “intrappolata” tra la curva superiore $c_2g(n)$ e quella inferiore $c_1g(n)$ — è la rappresentazione visiva della Definizione di Θ appena data.

2.2.2 Esempi: Vero o Falso?

Esercizio 1

Verificare se $f(n) = 10n^3 + 2n^2 + 7 \stackrel{?}{=} O(n^3)$.

Soluzione (metodo 1): dobbiamo provare che $\exists c > 0, \exists m \geq 0 : f(n) \leq cn^3, \forall n \geq m$.

$$\begin{aligned} f(n) &= 10n^3 + 2n^2 + 7 \\ &\leq 10n^3 + 2n^3 + 7 && \forall n \geq 1 \\ &\leq 10n^3 + 2n^3 + 7n^3 && \forall n \geq 1 \\ &= 19n^3 \\ &\stackrel{?}{\leq} cn^3 \end{aligned}$$

che è vera per ogni $c \geq 19$ e per ogni $n \geq 1$, quindi $m = 1$.

Nota grafica: la slide mostra il confronto tra $10n^3 + 2n^3 + 7n^3$, $10n^3 + 2n^3 + 7$ e la $f(n) = 10n^3 + 2n^2 + 7$ originale, per evidenziare visivamente che la maggiorazione scelta domina $f(n)$ su tutto l'intervallo considerato.

Soluzione (metodo 2) — non è l'unico modo di procedere:

$$\begin{aligned} f(n) &= 10n^3 + 2n^2 + 7 \\ &\leq 10n^3 + 2n^3 + 7 && \forall n \geq 1 \\ &\leq 10n^3 + 2n^3 + n^3 && \forall n \geq \sqrt[3]{7} \\ &= 13n^3 \\ &\stackrel{?}{\leq} cn^3 \end{aligned}$$

che è vera per ogni $c \geq 13$ e per ogni $n \geq \sqrt[3]{7}$, quindi usiamo $m = 2$.

Osservazione

I due metodi producono coppie (c, m) diverse ($c = 19, m = 1$ contro $c = 13, m = 2$), entrambe valide: la coppia (c, m) che soddisfa la definizione di $O(\cdot)$ **non è unica**.

Esercizio 2

Verificare se $f(n) = 3n^2 + 7n \stackrel{?}{=} \Theta(n^2)$.

Soluzione. Occorre dimostrare separatamente il limite inferiore e quello superiore.

Limite inferiore: $\exists c_1 > 0, \exists m_1 \geq 0 : f(n) \geq c_1n^2, \forall n \geq m_1$.

$$\begin{aligned} f(n) &= 3n^2 + 7n \\ &\geq 3n^2 && \text{per } n \geq 0 \\ &\stackrel{?}{\geq} c_1n^2 \end{aligned}$$

vera per ogni $c_1 \leq 3$ e per ogni $n \geq 0$, quindi $m_1 = 0$.

Limite superiore: $\exists c_2 > 0, \exists m_2 \geq 0 : f(n) \leq c_2 n^2, \forall n \geq m_2$.

$$\begin{aligned} f(n) &= 3n^2 + 7n \\ &\leq 3n^2 + 7n^2 && \text{per } n \geq 1 \\ &= 10n^2 \\ &\stackrel{?}{\leq} c_2 n^2 \end{aligned}$$

vera per ogni $c_2 \geq 10$ e per ogni $n \geq 1$, quindi $m_2 = 1$.

Applicando la Definizione 2.2.1 con

$$\exists c_1 > 0, \exists c_2 > 0, \exists m \geq 0 : c_1 n^2 \leq f(n) \leq c_2 n^2, \forall n \geq m$$

si ottengono i parametri:

$$c_1 = 3, \quad c_2 = 10, \quad m = \max\{m_1, m_2\} = \max\{0, 1\} = 1$$

Nota grafica: la slide mostra le tre curve $10n^2$, $3n^2 + 7n$ e $3n^2$, con $f(n) = 3n^2 + 7n$ visibilmente compresa tra le altre due già a partire da $n = 1$.

Esercizio 3

Verificare se $n^2 \stackrel{?}{\in} O(n)$.

Soluzione. Dobbiamo dimostrare che $\exists c > 0, \exists m \geq 0 : n^2 \leq cn, \forall n \geq m$.
Dovrebbe esistere una costante $c > 0$ tale che

$$n^2 \leq cn \iff c \geq n.$$

Ma n cresce senza limite: **nessuna** costante c può soddisfare la disequazione per ogni $n \geq m$. L'affermazione è quindi **falsa**: $n^2 \notin O(n)$.

Nota grafica: la slide mostra n^2 insieme a tre rette $60n$, $40n$, $20n$, con i punti di incrocio rispettivamente a $n = 60$, $n = 40$, $n = 20$. Il grafico illustra bene il ragionamento: qualunque costante c si scelga, la parabola n^2 supera definitivamente la retta cn a partire da $n = c$ — non esiste alcuna retta che domini n^2 per *ogni* n sufficientemente grande.

Esercizio 4

Verificare se $n^2 \stackrel{?}{=} O(n^3)$.

Soluzione. Dobbiamo dimostrare che $\exists c > 0, \exists m \geq 0 : n^2 \leq cn^3, \forall n \geq m$.
Otteniamo:

$$n^2 \leq cn^3 \iff c \geq \frac{1}{n}$$

La funzione $1/n$ è monotona decrescente per $n > 0$. Possiamo scegliere $m = 1$ e $c = 1$: per ogni $n \geq 1$ vale infatti $1/n \leq 1$. L'affermazione è quindi **vera**: $n^2 = O(n^3)$.

2.3 Proprietà della Notazione Asintotica

2.3.1 Proprietà delle Notazioni

Proprietà: Dualità

$$f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$$

Dimostrazione:

$$\begin{aligned} f(n) = O(g(n)) &\iff f(n) \leq c g(n), \forall n \geq m \\ &\iff g(n) \geq \frac{1}{c} f(n), \forall n \geq m \\ &\iff g(n) \geq c' f(n), \forall n \geq m \quad \left(c' = \frac{1}{c}\right) \\ &\iff g(n) = \Omega(f(n)) \end{aligned}$$

Proprietà: Eliminazione delle Costanti

$$\begin{aligned} f(n) = O(g(n)) &\iff a f(n) = O(g(n)), \quad \forall a > 0 \\ f(n) = \Omega(g(n)) &\iff a f(n) = \Omega(g(n)), \quad \forall a > 0 \end{aligned}$$

Dimostrazione (lato O ; il lato Ω è analogo):

$$\begin{aligned} f(n) = O(g(n)) &\iff f(n) \leq c g(n), \forall n \geq m \\ &\iff a f(n) \leq ac g(n), \forall n \geq m, \quad \forall a > 0 \\ &\iff a f(n) \leq c' g(n), \forall n \geq m \quad (c' = ac > 0) \\ &\iff a f(n) = O(g(n)) \end{aligned}$$

Proprietà: Sommatoria (Sequenza di Algoritmi)

$$\begin{aligned} f_i(n) = O(g_i(n)), \quad i = 1, 2 &\implies f_1(n) + f_2(n) = O(\max\{g_1(n), g_2(n)\}) \\ f_i(n) = \Omega(g_i(n)), \quad i = 1, 2 &\implies f_1(n) + f_2(n) = \Omega(\max\{g_1(n), g_2(n)\}) \end{aligned}$$

Dimostrazione (lato O ; il lato Ω è analogo):

$$\begin{aligned} f_1(n) + f_2(n) &\leq c_1 g_1(n) + c_2 g_2(n) \\ &\leq (c_1 + c_2) \max\{g_1(n), g_2(n)\} \\ &\implies f_1(n) + f_2(n) = O(\max\{g_1(n), g_2(n)\}). \end{aligned}$$

Osservazione

Questa proprietà si applica tipicamente a una **sequenza di algoritmi** eseguiti uno dopo l'altro: il costo totale è dominato dal costo del passo asintoticamente più lento.

Proprietà: Prodotto (Cicli Annidati)

$$\begin{aligned} f_1(n) = O(g_1(n)), f_2(n) = O(g_2(n)) &\implies f_1(n) \cdot f_2(n) = O(g_1(n) \cdot g_2(n)) \\ f_1(n) = \Omega(g_1(n)), f_2(n) = \Omega(g_2(n)) &\implies f_1(n) \cdot f_2(n) = \Omega(g_1(n) \cdot g_2(n)) \end{aligned}$$

Dimostrazione (lato O ; il lato Ω è analogo):

$$\begin{aligned} f_1(n) = O(g_1(n)) \wedge f_2(n) = O(g_2(n)) &\implies \\ f_1(n) \leq c_1 g_1(n) \wedge f_2(n) \leq c_2 g_2(n) &\implies \\ f_1(n) \cdot f_2(n) \leq c_1 c_2 g_1(n) g_2(n) & \end{aligned}$$

Osservazione

Questa proprietà si applica tipicamente ai **cicli annidati**: il costo complessivo è il prodotto dei costi dei singoli cicli.

Proprietà: Simmetria

$$f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$$

Dimostrazione: grazie alla proprietà di dualità (Proprietà 2.3.1):

$$\begin{aligned} f(n) = \Theta(g(n)) &\implies f(n) = O(g(n)) \implies g(n) = \Omega(f(n)) \\ f(n) = \Theta(g(n)) &\implies f(n) = \Omega(g(n)) \implies g(n) = O(f(n)) \end{aligned}$$

L'implicazione inversa si ottiene scambiando f e g .

Proprietà: Transitività

$$f(n) = O(g(n)), g(n) = O(h(n)) \implies f(n) = O(h(n))$$

Dimostrazione:

$$\begin{aligned} f(n) = O(g(n)) \wedge g(n) = O(h(n)) &\implies \\ f(n) \leq c_1 g(n) \wedge g(n) \leq c_2 h(n) &\implies \\ f(n) \leq c_1 c_2 h(n) &\implies \\ f(n) = O(h(n)) & \end{aligned}$$

2.3.2 Proprietà delle Funzioni più Comuni

Proprietà: Espressioni Polinomiali

$$f(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0, \quad a_k > 0 \implies f(n) = \Theta(n^k)$$

Dimostrazione (limite superiore): $\exists c > 0, \exists m \geq 0 : f(n) \leq cn^k, \forall n \geq m.$

$$\begin{aligned}
 f(n) &= a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 \\
 &\leq a_k n^k + |a_{k-1}| n^{k-1} + \dots + |a_1| n + |a_0| \\
 &\leq a_k n^k + |a_{k-1}| n^k + \dots + |a_1| n^k + |a_0| n^k & \forall n \geq 1 \\
 &= (a_k + |a_{k-1}| + \dots + |a_1| + |a_0|) n^k \\
 &\stackrel{?}{\leq} c n^k
 \end{aligned}$$

che è vera per $c \geq (a_k + |a_{k-1}| + \dots + |a_1| + |a_0|) > 0$ e per $m = 1$.

Dimostrazione (limite inferiore): $\exists d > 0, \exists m \geq 0 : f(n) \geq dn^k, \forall n \geq m.$

$$\begin{aligned}
 f(n) &= a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 \\
 &\geq a_k n^k - |a_{k-1}| n^{k-1} - \dots - |a_1| n - |a_0| \\
 &\geq a_k n^k - |a_{k-1}| n^{k-1} - \dots - |a_1| n^{k-1} - |a_0| n^{k-1} & \forall n \geq 1 \\
 &\stackrel{?}{\geq} d n^k
 \end{aligned}$$

L'ultima disequazione è vera se:

$$d \leq a_k - \frac{|a_{k-1}|}{n} - \frac{|a_{k-2}|}{n} - \dots - \frac{|a_1|}{n} - \frac{|a_0|}{n} > 0 \iff n > \frac{|a_{k-1}| + \dots + |a_0|}{a_k}$$

Nota

La slide si ferma qui: mostra che, superata questa soglia su n , il membro destro diventa positivo (quindi **esiste** un m adeguato), ma non esplicita una coppia finale (d, m) come invece fa per il limite superiore. Concettualmente basta fissare m pari (o superiore) alla soglia indicata e scegliere poi $d > 0$ abbastanza piccolo da soddisfare la disuguaglianza per quell' m .

Costo costante. Qual è la complessità di $f(n) = 17$?

$$\begin{aligned}
 f(n) = 17 &\geq c_1 n^0 \implies c_1 \leq 17 \\
 f(n) = 17 &\leq c_2 n^0 \implies c_2 \geq 17
 \end{aligned}$$

quindi $f(n) = \Theta(n^0) = \Theta(1)$.

Regola pratica

Iterazioni limitate da un valore costante (non dipendente da n) hanno **costo costante**.

```

real distance3D(real[] p, real[] q)
real s = 0
for i = 0 to 2 do
    s = s + (p[i] - q[i])^2
return sqrt(s)

```

Il ciclo scandisce sempre e solo 3 coordinate, indipendentemente da eventuali altri parametri: il numero di iterazioni non dipende da n , quindi il costo è $\Theta(1)$.

Proprietà: Logaritmi vs Funzioni Polinomiali

Per ogni costante $k > 0$: $\log n = O(n^k)$.

Dimostrazione: per la concavità del logaritmo, $\ln x \leq x - 1$, $\forall x > 0$. Ponendo $x = n^k$, con $k > 0$:

$$\begin{aligned}\ln(n^k) &\leq n^k - 1 \\ k \ln n &\leq n^k - 1 \\ \ln n &\leq \frac{n^k - 1}{k} \leq \frac{1}{k} n^k \\ \log n &\leq \frac{1}{k \ln 2} n^k \quad (\text{poiché } \ln n = \ln 2 \cdot \log n)\end{aligned}$$

Quindi, scegliendo $c = \frac{1}{k \ln 2}$, si ha $\log n = O(n^k)$ per ogni $k > 0$.

Prendiamo Confidenza

Verifichiamo rapidamente alcune uguaglianze asintotiche notevoli.

- È vero che $\log_a n = \Theta(\log n)$?
Sì: $\log_a n = (\log_a 2) \cdot (\log_2 n) = \Theta(\log n)$.
- È vero che $\log n^a = \Theta(\log n)$, per $a > 0$?
Sì: $\log n^a = a \log n = \Theta(\log n)$.
- È vero che $2^{n+1} = \Theta(2^n)$?
Sì: $2^{n+1} = 2 \cdot 2^n = \Theta(2^n)$.
- È vero che $2^n = \Theta(3^n)$?
No. Ovviamente $2^n = O(3^n)$. Ma:

$$3^n = \left(\frac{3}{2} \cdot 2\right)^n = \left(\frac{3}{2}\right)^n \cdot 2^n$$

quindi non esiste $c > 0$ tale per cui $\left(\frac{3}{2}\right)^n \cdot 2^n \leq c \cdot 2^n$: dunque $2^n \neq \Omega(3^n)$, e di conseguenza $2^n \neq \Theta(3^n)$.

Classificazione delle Funzioni

Estendendo le relazioni dimostrate finora, è possibile ottenere un ordinamento delle principali espressioni di costo. Per ogni $0 < r < s$, $0 < h < k$, $1 < a < b$:

$$\begin{aligned}O(1) &\subset O(\log_r n) \subset O(\log_s n) \subset O(n^h) \subset O(n^h \log_r n) \subset \\ &O(n^h \log_s n) \subset O(n^k) \subset O(a^n) \subset O(b^n) \subset O(n!) \subset O(n^n)\end{aligned}$$

2.3.3 Le Notazioni o , ω

Definizione: Notazioni o , ω (o-piccolo, omega-piccolo)

Sia $g(n)$ una funzione di costo; indichiamo con $o(g(n))$ l'insieme delle funzioni $f(n)$ tali per cui:

$$\forall c > 0, \exists m \geq 0 : 0 \leq f(n) < c g(n), \forall n \geq m.$$

Sia $g(n)$ una funzione di costo; indichiamo con $\omega(g(n))$ l'insieme delle funzioni $f(n)$ tali per cui:

$$\forall c > 0, \exists m \geq 0 : f(n) > c g(n) \geq 0, \forall n \geq m.$$

- Come si leggono: $f(n)$ è “o piccolo”, “omega piccolo” di $g(n)$.
- Come si scrivono: $f(n) = o(g(n))$ oppure $f(n) = \omega(g(n))$.

Caratterizzazione tramite limite. Utilizzando il concetto di limite, date due funzioni $f(n)$ e $g(n)$, valgono le seguenti implicazioni:

$$\begin{aligned}\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 &\implies f(n) = o(g(n)) \\ \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c, \quad 0 < c < +\infty &\implies f(n) = \Theta(g(n)) \\ \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = +\infty &\implies f(n) = \omega(g(n))\end{aligned}$$

Si noti inoltre che:

$$\begin{aligned}f(n) = o(g(n)) &\implies f(n) = O(g(n)) \\ f(n) = \omega(g(n)) &\implies f(n) = \Omega(g(n))\end{aligned}$$

2.4 Analisi Ammortizzata

2.4.1 Introduzione

Definizione: Analisi Ammortizzata

L'**analisi ammortizzata** è una tecnica di analisi di complessità che valuta il tempo richiesto per eseguire, nel caso pessimo, una **sequenza di operazioni** su una struttura dati.

- Esistono operazioni più o meno costose.
- Se le operazioni più costose sono poco frequenti, allora il loro costo può essere **ammortizzato** dalle operazioni meno costose.

Importante Differenza

- **Analisi caso medio:** probabilistica, su singola operazione.
- **Analisi ammortizzata:** deterministica, su operazioni multiple, caso pessimo.

Tre metodi per l'analisi ammortizzata:

- **Metodo dell'aggregazione:** si calcola la complessità $T(n)$ per eseguire n operazioni in sequenza nel caso pessimo (tecnica derivata dalla *matematica*).
- **Metodo degli accantonamenti:** alle operazioni vengono assegnati costi ammortizzati che possono essere maggiori/minori del loro costo effettivo (tecnica derivata dall'*economia*).
- **Metodo del potenziale:** lo stato del sistema viene descritto con una funzione di potenziale (tecnica derivata dalla *fisica*).

2.4.2 Contatore Binario

Definizione: Contatore Binario

Contatore binario di k bit rappresentato con un vettore A di booleani 0/1: bit meno significativo in $A[0]$, bit più significativo in $A[k-1]$. Valore del contatore:

$$x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$$

L'operazione `increment()` incrementa il contatore di 1:

```
increment(int[] A, int k)
int i = 0
while i < k and A[i] == 1 do
    A[i] = 0
    i = i + 1
if i < k then
    A[i] = 1
```

Il ciclo azzerava tutti i bit a 1 consecutivi a partire da $A[0]$ (il “riporto”), poi accende il primo bit a 0 incontrato.

Esempio di evoluzione (colonne $A[7] \dots A[5]$ omesse: sempre 0 in questo intervallo):

x	$A[4]$	$A[3]$	$A[2]$	$A[1]$	$A[0]$
0	0	0	0	0	0
1	0	0	0	0	1
2	0	0	0	1	0
3	0	0	0	1	1
4	0	0	1	0	0
5	0	0	1	0	1
6	0	0	1	1	0
7	0	0	1	1	1
8	0	1	0	0	0

(Tabella abbreviata a $x = 0, \dots, 8$ per brevità; nelle slide originali prosegue identica fino a $x = 16$.)

Metodo dell'Aggregazione Domanda: qual è il costo $T(n)$ per eseguire n operazioni (caso pessimo)? Qual è il costo ammortizzato $T(n)/n$?

Analisi “grossolana”. Sono necessari $k = \lceil \log(n+1) \rceil$ bit per rappresentare n ; una chiamata a `increment()` richiede tempo $O(k)$ nel caso pessimo. Dunque:

$$T(n) = O(nk) \quad \implies \quad \frac{T(n)}{n} = O(k) = O(\log n)$$

Nota grafica: un benchmark sperimentale (Java, $n = 2^k$ con $k \in [20, 35]$) mostra il tempo medio per operazione $T(n)/n$ che resta sostanzialmente piatto (pochi nanosecondi) al crescere di n su più ordini di grandezza — un’evidenza empirica che l’analisi “grossolana” $O(\log n)$ sovrastima il vero costo ammortizzato, che l’analisi più fine sotto dimostra essere $O(1)$.

Analisi più stretta. Si osserva che il tempo per l’intera sequenza è proporzionale al numero di bit modificati. Dalla simulazione:

- $A[0]$ viene modificato ogni 1 incremento;
- $A[1]$ viene modificato ogni 2 incrementi;
- $A[2]$ viene modificato ogni 4 incrementi;
- in generale, $A[i]$ viene modificato ogni 2^i incrementi.

Risultato: Costo Ammortizzato del Contatore Binario

$$\begin{aligned} \text{Costo totale: } T(n) &= \sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor \leq n \sum_{i=0}^{k-1} \frac{1}{2^i} \leq n \sum_{i=0}^{+\infty} \left(\frac{1}{2}\right)^i = 2n \\ \text{Costo ammortizzato: } \frac{T(n)}{n} &\leq \frac{2n}{n} = 2 = O(1) \end{aligned}$$

Metodo degli Accantonamenti Obiettivi del metodo:

- dimostrare che la somma dei costi ammortizzati a_i è un limite superiore ai costi effettivi c_i :

$$\sum_{i=1}^n c_i \leq \sum_{i=1}^n a_i$$

- dimostrare che il valore così ottenuto è “poco costoso”.

Punti da ricordare:

- la dimostrazione deve valere per **tutte** le sequenze (caso pessimo);
- il credito dopo l’operazione t -esima deve essere sempre positivo:

$$\sum_{i=1}^t a_i - \sum_{i=1}^t c_i \geq 0$$

Applicazione al contatore binario:

- costo effettivo dell'operazione `increment()`: d , dove d è il numero di bit che cambiano valore;
- costo ammortizzato dell'operazione `increment()`: 2
 - 1 per il cambio di un bit da 0 a 1 (costo effettivo);
 - 1 per il futuro cambio dello stesso bit da 1 a 0 (accantonato come credito).
- ne consegue che, in ogni istante, il credito accumulato è pari al numero di bit a 1 presenti nel contatore;
- costo totale: $O(n)$; costo ammortizzato: $O(1)$.

Esempio (stessa evoluzione di sopra, con colonna Credito):

x	$A[4]$	$A[3]$	$A[2]$	$A[1]$	$A[0]$	Credito
0	0	0	0	0	0	0
1	0	0	0	0	1	1
2	0	0	0	1	0	1
3	0	0	0	1	1	2
4	0	0	1	0	0	1
5	0	0	1	0	1	2
6	0	0	1	1	0	2
7	0	0	1	1	1	3
8	0	1	0	0	0	1

Il credito in ogni riga coincide, come atteso, con il numero di bit a 1 nel contatore in quello stato.

Definizione: Funzione di Potenziale

Una **funzione di potenziale** Φ associa a uno stato S della struttura dati una "quantità di lavoro" $\Phi(S)$ che è stata contabilizzata nell'analisi ammortizzata. L'energia potenziale "immagazzinata" in quello stato è $\Phi(S)$.
Costo ammortizzato = costo effettivo + variazione di potenziale:

$$a_i = c_i + \Phi(S_i) - \Phi(S_{i-1})$$

dove S_i è lo stato associato alla i -esima operazione.

Metodo del Potenziale

Costo ammortizzato di una sequenza di n operazioni (somma telescopica):

$$\begin{aligned}
 A &= \sum_{i=1}^n a_i \\
 &= \sum_{i=1}^n (c_i + \Phi(S_i) - \Phi(S_{i-1})) \\
 &= \sum_{i=1}^n c_i + \sum_{i=1}^n (\Phi(S_i) - \Phi(S_{i-1})) \\
 &= C + \underbrace{\Phi(S_1) - \Phi(S_0) + \Phi(S_2) - \Phi(S_1) + \dots + \Phi(S_n) - \Phi(S_{n-1})}_{\text{telescopica}} \\
 &= C + \Phi(S_n) - \Phi(S_0)
 \end{aligned}$$

Osservazione Chiave

Se la variazione di potenziale $\Phi(S_n) - \Phi(S_0)$ è **non negativa**, il costo ammortizzato A è un limite superiore al costo reale C .

Applicazione al contatore binario. Si sceglie come funzione di potenziale Φ il numero di bit a 1 presenti nel contatore. Per l'operazione `increment()`, sia t il numero di bit a 1 incontrati a partire dal meno significativo, prima di incontrare uno 0:

- costo effettivo: $1 + t$;
- variazione di potenziale: $1 - t$;
- costo ammortizzato: $(1 + t) + (1 - t) = 2$.

Infine: all'inizio, zero bit accesi $\Rightarrow \Phi(S_0) = 0$; alla fine, $\Phi(S_n) \geq 0$ sempre $\Rightarrow$ la differenza di potenziale è non negativa, il che rende valida la maggiorazione.

Nota

I tre metodi applicati allo stesso problema (contatore binario) convergono tutti sullo stesso risultato: costo ammortizzato $O(1)$ per operazione, contro il costo $O(\log n)$ del caso pessimo di una singola operazione — è proprio questo scarto a rendere l'analisi ammortizzata interessante.

2.4.3 Vettori Dinamici

Inserimento Sequenze implementate tramite vettori dinamici:

- si alloca un vettore di una certa dimensione detta **capacità**;
- l'inserimento di un elemento “in mezzo” ha costo $O(n)$;
- inserire un elemento “in fondo” alla sequenza (**append**) ha costo $O(1)$.

Nota grafica: le slide illustrano la differenza con un vettore di 8 celle, di cui le prime 5 occupate (grigie) e le ultime 3 libere (bianche): un **Insert** in mezzo richiede lo spostamento di più elementi, mentre un **Append** inserisce il nuovo elemento (evidenziato in giallo) direttamente nella prima cella libera, senza spostare nulla.

Problema: non è noto a priori quanti elementi entreranno nella sequenza; la capacità selezionata può rivelarsi insufficiente.

Soluzione: si alloca un vettore di capacità maggiore, si ricopia il contenuto del vecchio vettore nel nuovo e si libera la memoria del vecchio vettore. Esempi: `java.util.Vector`, `java.util.ArrayList` (Java), `std::vector` (C++), `list` (Python).

```
private Object[] buffer = new Object[INITSIZE];

// Raddoppiamento -- utilizzato in ArrayList (1.2)
private void doubleStorage() {
    Object[] newb = new Object[2*buffer.length];
    System.arraycopy(buffer, 0, newb, 0, buffer.length);
    buffer = newb;
```

```

}

// Incremento fisso -- utilizzato in Vector (1.0)
private void incrementStorage() {
    Object[] newb = new Object[buffer.length + INCREMENT];
    System.arraycopy(buffer, 0, newb, 0, buffer.length);
    buffer = newb;
}

```

Domanda: qual è il migliore fra i due metodi di espansione (raddoppiamento o incremento costante)? Dalla documentazione Java, `ArrayList.add()`:

“As elements are added to an ArrayList, its capacity grows automatically. The details of the growth policy are not specified beyond the fact that adding an element has constant amortized time cost.”

(Nella slide originale la parola “grows” risulta tagliata dal bordo del box di testo; la citazione ufficiale nota della documentazione Java è riportata qui per completezza.)

Analisi Ammortizzata: Raddoppiamento del Vettore

Assunzioni: dimensione iniziale 1; costo di scrittura di un elemento 1.

Costo effettivo di un’operazione `add()`:

$$c_i = \begin{cases} 1 + 2^k & \text{se } \exists k \in \mathbb{Z}_0^+ : i = 2^k + 1 \\ 1 & \text{altrimenti} \end{cases}$$

n	costo
1	1
2	$1 + 2^0 = 2$
3	$1 + 2^1 = 3$
4	1
5	$1 + 2^2 = 5$
6–8	1
9	$1 + 2^3 = 9$
10–16	1
17	$1 + 2^4 = 17$

Costo effettivo di n operazioni `add()`:

$$\begin{aligned}
 T(n) &= \sum_{i=1}^n c_i = n + \sum_{j=0}^{\lfloor \log n \rfloor} 2^j \\
 &= n + 2^{\lfloor \log n \rfloor + 1} - 1 \\
 &\leq n + 2^{\log n + 1} - 1 \\
 &= n + 2n - 1 = O(n)
 \end{aligned}$$

Costo ammortizzato di un’operazione `add()`:

$$\frac{T(n)}{n} = \frac{O(n)}{n} = O(1)$$

Analisi Ammortizzata: Incremento Costante del Vettore

Assunzioni: incremento d ; dimensione iniziale d ; costo di scrittura di un elemento 1. (Nell'esempio, $d = 4$.)

Costo effettivo di un'operazione `add()`:

$$c_i = \begin{cases} i & \text{se } (i \bmod d) = 1 \\ 1 & \text{altrimenti} \end{cases}$$

n	costo ($d = 4$)
1-4	1
5	$1 + d = 5$
6-8	1
9	$1 + 2d = 9$
10-12	1
13	$1 + 3d = 13$
14-16	1
17	$1 + 4d = 17$

Costo effettivo di n operazioni `add()`:

$$\begin{aligned} T(n) &= \sum_{i=1}^n c_i = n + \sum_{j=1}^{\lfloor n/d \rfloor} d \cdot j \\ &= n + d \sum_{j=1}^{\lfloor n/d \rfloor} j \\ &= n + d \frac{(\lfloor n/d \rfloor + 1) \lfloor n/d \rfloor}{2} \\ &\leq n + \frac{(n/d + 1) n}{2} = O(n^2) \end{aligned}$$

Costo ammortizzato di un'operazione `add()`:

$$\frac{T(n)}{n} = \frac{O(n^2)}{n} = O(n)$$

Confronto

Il raddoppiamento garantisce costo ammortizzato $O(1)$ per inserimento; l'incremento costante degrada a $O(n)$. È per questo che tutte le implementazioni reali (Reality check, sotto) usano una crescita **moltiplicativa**, non additiva.

Reality check:

Linguaggio	Struttura dati	Fattore espansione
GNU C++	<code>std::vector</code>	2.0
Microsoft VC++ 2003	<code>vector</code>	1.5
C#	<code>List</code>	2.0
Python	<code>list</code>	1.125
Oracle Java	<code>ArrayList</code>	2.0
OpenSDK Java	<code>ArrayList</code>	1.5

Non Ancora Convinti? Allocazione Memoria

Domande: quanta memoria occupa un intero a 32 bit se memorizzato in un vettore dinamico (caso pessimo/ottimo)? E se memorizzato in un elemento di una lista?

- **Vettori:** caso ottimo (vettore pieno) $4n$ byte; caso pessimo (vettore appena raddoppiato) $2 \cdot 4n$ byte.
- **Liste:** alcuni dati su Java — un `Object` vuoto richiede 16 byte su architettura a 64 bit; l’allocazione di memoria avviene per multipli di 8 byte; un oggetto con un intero e due reference (lista bidirezionale) richiede 32–40 byte (con Java compressed references).

Non Ancora Convinti? String vs StringBuffer

```
public static void print(int dim)
{
    StringBuffer buffer = new StringBuffer();
    for (int i=0; i < dim; i++) {
        buffer.append('x');
    }

    String string = new String();
    for (int i=0; i < dim; i++) {
        string = string + 'x';
    }
}
```

Ogni concatenazione `string + 'x'` su una `String` (immutabile in Java) alloca un nuovo oggetto e ricopia l’intera stringa esistente: è l’analogo di un vettore che si “riespande” **ad ogni** inserimento, non solo occasionalmente — da cui il costo quadratico osservato sperimentalmente:

dim	StringBuffer (ms)	String (ms)
1 024	0	3
2 048	1	7
4 096	1	13
8 192	1	22
16 384	3	71
32 768	4	285
65 536	3	1 130
131 072	4	4 847
262 144	6	20 853
524 288	11	84 982
1 048 576	24	400 653

Non Ancora Soddisfatti? Somma di Stringhe in Python

```
s = ""
for i in range(10000):
    s = s + "1"
```

In **CPython**, questo codice viene ottimizzato utilizzando **realloc** invece di **malloc** e ha costo **lineare** – fino a un certo punto; oltre una certa dimensione, il costo torna a essere **quadratico**. Altre implementazioni di Python non seguono necessariamente questo approccio.

Fonte: <https://stackoverflow.com/questions/4435169/how-do-i-append-one-string-to-another-in-python> (Credits: Michele Baldo).

Cancellazione Domande: quanto costa togliere un elemento da un vettore? Quanto costa togliere un elemento da un vettore **non ordinato**?

Nota grafica: le slide mostrano un vettore con celle occupate (grigie) e libere (bianche); l'elemento da rimuovere (evidenziato in giallo) si trova in una posizione n arbitraria, non necessariamente l'ultima — coerente con la tecnica standard per vettori *non ordinati*: si copia l'ultimo elemento nella posizione da cancellare e si riduce la dimensione di 1, ottenendo un costo $O(1)$ invece di $O(n)$.

Contrazione. Per ridurre lo spreco di memoria, è opportuno contrarre il vettore quando il **fattore di carico** $\alpha = \text{dim}/\text{capacità}$ diventa troppo piccolo, dove dim è il numero di elementi attualmente presenti. Una contrazione richiede comunque allocazione, copia e deallocazione. (*La slide pone qui una domanda in aula tramite sondaggio live: quale soglia scegliere per il fattore di carico?*)

Strategia naïf: dimezzare la capacità quando α scende a $1/2$. Si dimostra che questa strategia può portare a un costo ammortizzato **lineare** (non costante!). Controesempio: sequenza di Inserimenti/Rimozioni su un vettore di capacità iniziale 8:

Ops	I	I	I	I	I	R	I	R	I	R	I	R	I
Dim	1	2	3	4	5	4	5	4	5	4	5	4	5
Cap	8	8	8	8	8	4	8	4	8	4	8	4	8

Qual è il problema?

Non abbiamo un numero di inserimenti/rimozioni sufficienti per ripagare le espansioni/contrazioni: la sequenza oscilla continuamente intorno alla soglia $\alpha = 1/2$, innescando un'espansione o una contrazione (entrambe $\Theta(\text{dim})$) quasi a ogni operazione.

Soluzione: dobbiamo lasciar decrescere il sistema fino a $\alpha = 1/4$ prima di contrarre. Dopo una contrazione, il fattore di carico torna a $1/2$ — esattamente come dopo un'espansione. Si usa allora una funzione di potenziale che:

- vale 0 all'inizio e subito dopo un'espansione o una contrazione;
- cresce fino a raggiungere il numero di elementi presenti quando α aumenta fino a 1 o diminuisce fino a $1/4$.

Funzione di Potenziale per la Contrazione

$$\Phi = \begin{cases} 2 \cdot \text{dim} - \text{capacità} & \alpha \geq \frac{1}{2} \\ \frac{\text{capacità}}{2} - \text{dim} & \alpha \leq \frac{1}{2} \end{cases}$$

Casi esplicativi:

- $\alpha = 1/2$ (dopo espansione/contrazione) $\Rightarrow \Phi = 0$;
- $\alpha = 1$ (prima di un'espansione) $\Rightarrow \text{dim} = \text{capacità} \Rightarrow \Phi = \text{dim}$;
- $\alpha = 1/4$ (prima di una contrazione) $\Rightarrow \text{capacità} = 4 \cdot \text{dim} \Rightarrow \Phi = \text{dim}$.

In altre parole: subito prima di espansioni e contrazioni, il potenziale accumulato è sufficiente per “pagare” il costo della copia dei dim valori.

Nota grafica: le slide mostrano un grafico con tre curve in funzione dell'indice di operazione i : **size** (nero, andamento a “dente di sega” che sale e scende con inserimenti/rimozioni), **capacity** (verde, andamento “a gradini” che raddoppia/dimezza solo alle soglie $\alpha = 1$ e $\alpha = 1/4$) e **Φ** (rosso, anch'esso a dente di sega, che si azzera esattamente a ogni gradino di **capacity**) — la rappresentazione visiva del perché Φ funziona da “ammortizzatore” tra le due curve.

Dimostrazione (costo ammortizzato di un inserimento):

Caso $\alpha \geq 1/2$, senza espansione:

$$\begin{aligned} a_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (2 \cdot \text{dim}_i - \text{capacità}_i) - (2 \cdot \text{dim}_{i-1} - \text{capacità}_{i-1}) \\ &= 1 + 2 \cdot (\text{dim}_{i-1} + 1) - \text{capacità}_{i-1} - 2 \cdot \text{dim}_{i-1} + \text{capacità}_{i-1} \\ &= 3 \end{aligned}$$

Caso $\alpha = 1$, con espansione:

$$\begin{aligned} a_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + \text{dim}_{i-1} + (2 \cdot \text{dim}_i - \text{capacità}_i) - (2 \cdot \text{dim}_{i-1} - \text{capacità}_{i-1}) \\ &= 1 + \text{dim}_{i-1} + 2 \cdot (\text{dim}_{i-1} + 1) - 2 \cdot \text{dim}_{i-1} - 2 \cdot \text{dim}_{i-1} + \text{dim}_{i-1} \\ &= 3 \end{aligned}$$

In entrambi i casi il costo ammortizzato è costante ($= 3$): la soglia $\alpha = 1/4$ per la contrazione garantisce quindi un costo ammortizzato $O(1)$ per operazione.

Esercizio (non risolto nelle slide originali)

Verificare gli altri casi per valori differenti di α e per l'operazione di contrazione.

2.4.4 Conclusioni

Altri esempi di applicazione dell'analisi ammortizzata:

- espansione/contrazione di tabelle hash;

- insiemi disgiunti, con euristica sul rango e compressione dei cammini;
- heap di Fibonacci;
- ...

Reality Check Ristrutturazione delle tabelle hash:

- non è conveniente che α cresca troppo — in particolare con la scansione interna, ma vale anche per le liste di trabocco;
- sopra una soglia t_α prefissata (tipicamente 0.5–0.75): si alloca una nuova tabella di dimensione $2m$ e si reinseriscono tutte le chiavi;
- **risultato**: fattore di carico dimezzato (tipicamente 0.25), nessun elemento *deleted*;
- **costi**: $O(m)$ per la ristrutturazione nel caso pessimo; costo ammortizzato **costante** (stessa dimostrazione vista per i vettori dinamici).

Linguaggio	Tecnica	t_α	Note
Java HashMap	7 Liste di trabocco basate su <code>LinkedList</code>	0.75	$O(n)$ nel caso pessimo; overhead $16n + 4m$ byte
Java HashMap	8 Liste di trabocco basate su RB Tree	0.75	$O(\log n)$ nel caso pessimo; overhead $48n + 4m$ byte
C++ <code>sparse_hash</code>	Ind. aperto, scansione quadratica	?	overhead $2n$ bit
C++ <code>dense_hash</code>	Ind. aperto, scansione quadratica	0.5	X byte per chiave-valore $\Rightarrow$ $2-3X$ overhead
C++ STL <code>unordered_map</code>	Liste di trabocco basate su liste	1.00	MurmurHash
Python	Indirizzam. aperto,	0.66	—
	...		

Nota

L'ultima riga (Python) è **troncata nella slide originale stessa** — è l'ultima pagina del file (41/41) e la tabella esce dal bordo inferiore. La colonna Tecnica riporta solo “Indirizzam. aperto, ...” e la colonna Note risulta vuota nel materiale sorgente; non ho completato io il dato mancante.

2.5 Complessità degli Algoritmi vs Complessità dei Problemi

2.5.1 Introduzione

Obiettivo: riflettere sulla complessità di **problemi** e di **algoritmi**.

- In alcuni casi si può migliorare quanto si ritiene “normale”.
- In altri casi è impossibile fare di meglio.
- Qual è il rapporto fra un problema computazionale e l'algoritmo che lo risolve?

Si parte dalle basi (*back to basics*): somme e moltiplicazioni.

2.5.2 Moltiplicare Numeri Complessi

Moltiplicazione di Numeri Complessi

$$(a + bi)(c + di) = [ac - bd] + [ad + bc]i$$

- **Input:** a, b, c, d .
- **Output:** $a \times c - b \times d$ e $a \times d + b \times c$.

Domande (modello di calcolo: la moltiplicazione costa 1, le addizioni/sottrazioni costano 0.01):

- quanto costa l'algoritmo dettato dalla definizione?
- si può fare meglio di così?
- qual è il ruolo del modello di calcolo?

Si può fare meglio: bastano **tre** prodotti.

$$p_{ac} = a \times c$$

$$p_{bd} = b \times d$$

$$s = (a + b) \times (c + d) = ac + ad + bc + bd$$

$$\text{Output: } [p_{ac} - p_{bd}] + [s - p_{ac} - p_{bd}]i$$

Tre moltiplicazioni invece di quattro: **25% in meno** nel modello scelto.

Nota mia (calcolo non riportato nelle slide)

Le slide lasciano aperte le domande sul costo. Nel modello indicato (1 per prodotto, 0.01 per somma/sottrazione):

- **definizione:** 4 prodotti + 2 somme/sottrazioni = $4 + 2 \cdot 0.01 = 4.02$;
- **tre prodotti:** 3 prodotti + 5 somme/sottrazioni ($a + b, c + d, p_{ac} - p_{bd}$ e le due sottrazioni in $s - p_{ac} - p_{bd}$) = $3 + 5 \cdot 0.01 = 3.05$. Il risparmio effettivo è $\approx 24\%$: il “25%” della slide conta solo i prodotti;
- **ruolo del modello:** se anche somme e sottrazioni costassero 1, si passerebbe da 6 a 8 operazioni e la versione “migliorata” sarebbe *peggiore*. Cosa significhi “fare meglio” dipende dal modello di calcolo.

2.5.3 Sommare Numeri Binari

Algoritmo Elementare della Somma

Procede da destra verso sinistra (dal LSB al MSB), sommando due bit e l'eventuale riporto, con costo totale $\Theta(n)$.

$$\begin{array}{rcccccccc}
& 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\
+ & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 \\
\text{riporti} & +1 & +1 & & & +1 & +1 & & \\
\hline
= & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1
\end{array}$$

Domanda: si può fare meglio di così? Per rispondere serve definire cosa significa “complessità di un **problema**”.

Definizione: Limite Superiore alla Complessità di un Problema

Notazione $O(f(n))$ – limite superiore. Fissato un modello di calcolo, un problema ha una limitazione superiore $O(f(n))$ se **esiste almeno un algoritmo corretto** che lo risolve in tempo $O(f(n))$.

Limite superiore della somma di numeri binari: il problema ha una limitazione superiore $O(n)$ (l'algoritmo elementare ha costo $\Theta(n)$).

Definizione: Limite Inferiore alla Complessità di un Problema

Notazione $\Omega(f(n))$ – limite inferiore. Fissato un modello di calcolo, un problema ha una limitazione inferiore $\Omega(f(n))$ se **ogni algoritmo corretto** che lo risolve richiede tempo $\Omega(f(n))$.

Limite inferiore della somma di numeri binari: il problema ha una limitazione inferiore $\Omega(n)$.

Domanda posta dalla slide: riuscite a dimostrarlo? La slide non riporta la dimostrazione. *Spunto mio (non nelle slide):* ogni algoritmo corretto deve leggere tutti i bit in ingresso, perché il risultato dipende da ciascuno di essi.

2.5.4 Moltiplicare Numeri Binari

Algoritmo Elementare del Prodotto

L'algoritmo imparato a scuola genera un prodotto parziale per ogni bit di un operando e somma i prodotti parziali opportunamente traslati, con costo totale $\Theta(n^2)$.

Nota grafica: esempio a 8 bit, $01100110 \times 01110011 = 010110111010010$. Lo schema ha n prodotti parziali di n bit, ognuno traslato di una posizione in più verso sinistra rispetto al precedente; i prodotti parziali relativi ai bit 0 del secondo operando sono righe di zeri.

Confronto tra gli algoritmi elementari:

$$T_{\text{sum}}(n) = \Theta(n) \qquad T_{\text{prod}}(n) = \Theta(n^2)$$

Si potrebbe concludere che il problema della moltiplicazione è *inerentemente* più costoso del problema dell'addizione, il che conferma la nostra esperienza.

Attenzione!

- Abbiamo confrontato gli **algoritmi**, non i **problemi**!
- Sappiamo solo che l'algoritmo elementare di somma è più efficiente dell'algoritmo elementare di prodotto.
- Per provare che il *problema* del prodotto è più costoso del problema della somma, dobbiamo provare che **non esiste** una soluzione in tempo lineare per il prodotto.

Un po' di storia: nel 1960 Kolmogorov enunciò in una conferenza che la moltiplicazione ha limite inferiore $\Omega(n^2)$; una settimana dopo, un suo studente provò il contrario!

Anticipazione: Divide-et-Impera. Tecnica in tre fasi:

- **Divide:** dividi il problema in sottoproblemi di dimensioni inferiori;
- **Impera:** risolvi i sottoproblemi in maniera ricorsiva;
- **Combina:** unisci le soluzioni dei sottoproblemi in modo da ottenere la risposta del problema principale.

Moltiplicazione Divide-et-Impera

Si scompone ciascun operando in parte alta e parte bassa (X in a e b , Y in c e d):

$$\begin{aligned}X &= a \cdot 2^{n/2} + b \\Y &= c \cdot 2^{n/2} + d \\XY &= ac \cdot 2^n + (ad + bc) \cdot 2^{n/2} + bd\end{aligned}$$

Assunzione semplificativa: $n = 2^k$, ovvero $k = \log n$; tutti i sottovettori hanno dimensione pari a una potenza di 2. Più avanti si vedrà che queste assunzioni possono essere rimosse.

Quattro Sottoprodotti: Codice e Ricorrenza. Il prodotto richiede 4 chiamate ricorsive su operandi di $n/2$ bit:

```
int Prodotto4(int X, int Y, int n)
if n == 1 then
    return X * Y                                % Caso base: un solo bit
scomponi X = a*2^(n/2) + b e Y = c*2^(n/2) + d    % Divide
int pac = Prodotto4(a, c, n/2)                    % Impera
int pad = Prodotto4(a, d, n/2)                    % Impera
int pbc = Prodotto4(b, c, n/2)                    % Impera
int pbd = Prodotto4(b, d, n/2)                    % Impera
int s = pad + pbc
return pac*2^n + s*2^(n/2) + pbd                  % Combina
```

$$T(n) = \begin{cases} d & n = 1 \\ 4T(n/2) + cn & n > 1 \end{cases}$$

Moltiplicare per 2^t equivale a uno *shift* di t posizioni; somme, shift e ricomposizione hanno costo lineare (da cui il termine cn).

Quattro Sottoprodotti: Analisi per Livelli. *Nota grafica:* albero di ricorsione con radice di dimensione n e 4 figli di dimensione $n/2$ (livello 1), ciascuno con 4 figli di dimensione $n/4$ (livello 2), fino ai $4^{\log n}$ nodi foglia di dimensione 1.

Livello	# nodi	Dim	Costo del livello
0	4^0	$n/2^0$	$c \cdot \frac{n}{2^0} \cdot 4^0$
1	4^1	$n/2^1$	$c \cdot \frac{n}{2^1} \cdot 4^1$
2	4^2	$n/2^2$	$c \cdot \frac{n}{2^2} \cdot 4^2$
i	4^i	$n/2^i$	$c \cdot \frac{n}{2^i} \cdot 4^i$
$\log n$	$4^{\log n}$	1	$d \cdot 4^{\log n}$

$$T(n) = \sum_{i=0}^{k-1} c \cdot \frac{n}{2^i} \cdot 4^i + d \cdot 4^k = cn \sum_{i=0}^{k-1} 2^i + d \cdot 4^k$$

Somma Geometrica Finita

$$\sum_{i=0}^t x^i = \frac{x^{t+1} - 1}{x - 1} \quad (x \neq 1)$$

Costo dei livelli interni:

$$\begin{aligned} cn \sum_{i=0}^{k-1} 2^i &= cn \cdot \frac{2^k - 1}{2 - 1} && (\text{somma geometrica finita}) \\ &= cn (2^{\log n} - 1) && (k = \log n) \\ &= cn (n - 1) && (2^{\log n} = n) \end{aligned}$$

Costo delle foglie:

$$\begin{aligned} d \cdot 4^k &= d \cdot 4^{\log_2 n} && (k = \log_2 n) \\ &= d \cdot n^{\log_2 4} && (a^{\log_b n} = n^{\log_b a}) \\ &= d \cdot n^2 && (\log_2 4 = 2) \end{aligned}$$

Complessità finale:

$$T(n) = cn(n - 1) + dn^2 = \Theta(n^2).$$

Tanto Lavoro per Nulla!

$$\begin{aligned} \text{Scolastico: } T(n) &= \Theta(n^2) \\ \text{Quattro prodotti: } T(n) &= \Theta(n^2) \end{aligned}$$

Non solo la complessità è la stessa, ma le costanti moltiplicative sono più alte. *Domanda:* è possibile fare meglio di così? Si noti che la versione ricorsiva chiama se stessa 4 volte...

Karatsuba: da Quattro a Tre Sottoprodotti. Si sfrutta lo stesso trucco dei numeri complessi: $ad + bc$ si ottiene da un solo prodotto aggiuntivo.

```

int Karatsuba(int X, int Y, int n)
if n <= 1 then
    return X * Y                                % Lo stesso caso base
scomponi X = a*2^(n/2) + b e Y = c*2^(n/2) + d
int pac = Karatsuba(a, c, n/2)                  % a x c
int pbd = Karatsuba(b, d, n/2)                  % b x d
int r = Karatsuba(a+b, c+d, n/2)                % (a+b) x (c+d) = ac+ad+bc+bd
int s = r - pac - pbd                          % ad + bc
return pac*2^n + s*2^(n/2) + pbd                % La stessa ricomposizione

```

$$T(n) = \begin{cases} d & n = 1 \\ 3T(n/2) + cn & n > 1 \end{cases}$$

Analisi per livelli (albero di ricorsione con 3 figli per nodo):

Livello	# nodi	Dim	Costo del livello
0	3^0	$n/2^0$	$c \cdot \frac{n}{2^0} \cdot 3^0$
1	3^1	$n/2^1$	$c \cdot \frac{n}{2^1} \cdot 3^1$
2	3^2	$n/2^2$	$c \cdot \frac{n}{2^2} \cdot 3^2$
i	3^i	$n/2^i$	$c \cdot \frac{n}{2^i} \cdot 3^i$
$\log n$	$3^{\log n}$	1	$d \cdot 3^{\log n}$

$$T(n) = \sum_{i=0}^{k-1} c \cdot \frac{n}{2^i} \cdot 3^i + d \cdot 3^k = cn \sum_{i=0}^{k-1} \left(\frac{3}{2}\right)^i + d \cdot 3^k$$

Costo dei livelli interni:

$$\begin{aligned}
 cn \sum_{i=0}^{k-1} \left(\frac{3}{2}\right)^i &= cn \cdot \frac{(3/2)^k - 1}{3/2 - 1} && \text{(somma geometrica finita)} \\
 &= 2cn \left(n^{\log_2(3/2)} - 1\right) && (k = \log_2 n) \\
 &= 2c \left(n^{\log_2 3} - n\right) && (\log_2(3/2) = \log_2 3 - 1)
 \end{aligned}$$

Costo delle foglie:

$$\begin{aligned}
 d \cdot 3^k &= d \cdot 3^{\log_2 n} && (k = \log_2 n) \\
 &= d \cdot n^{\log_2 3} && (a^{\log_b n} = n^{\log_b a})
 \end{aligned}$$

Complessità finale:

$$T(n) = 2c \left(n^{\log_2 3} - n\right) + d n^{\log_2 3} = \Theta\left(n^{\log_2 3}\right), \quad \log_2 3 \approx 1.585$$

Confronto della Complessità Computazionale

$$\begin{array}{ll} \text{Scolastico: } T_{\text{prod}}(n) = \Theta(n^2) & \text{es. } T_{\text{prod}}(10^6) = 10^{12} \\ \text{Karatsuba: } T_{\text{kara}}(n) = \Theta(n^{\log_2 3}) & \text{es. } T_{\text{kara}}(10^6) \approx 3 \cdot 10^9 \end{array}$$

Conclusioni:

- l'algoritmo “naif” non è sempre il migliore...
- ...può esistere spazio di miglioramento...
- ...a meno che non sia possibile dimostrare il contrario!

2.5.5 Algoritmi vs Problemi

Complessità in Tempo di un Algoritmo

La quantità di tempo richiesta dall'algoritmo sugli input di dimensione n .

- Possiamo analizzare caso peggiore, medio e migliore.
- Otteniamo un limite sul costo di **quella particolare soluzione**.

Complessità in Tempo di un Problema Computazionale

- **Limitazione superiore** $O(f(n))$: esiste un algoritmo corretto che risolve il problema in tempo $O(f(n))$.
- **Limitazione inferiore** $\Omega(g(n))$: **ogni** algoritmo corretto richiede tempo $\Omega(g(n))$.
- Se i due ordini coincidono, l'algoritmo che raggiunge il limite inferiore è **asintoticamente ottimo**.

Nota mia: le notazioni O e Ω sono quelle della Sezione 2.2; qui cambia la quantificazione, che è sugli algoritmi ($\exists$ per il limite superiore, $\forall$ per quello inferiore).

2.6 Algoritmi di Ordinamento

2.6.1 Tipologia dell'Input e dell'Analisi

Obiettivo: valutare gli algoritmi in base all'input.

- In alcuni casi, gli algoritmi si comportano diversamente a seconda delle caratteristiche dell'input.
- Conoscere in anticipo tali caratteristiche permette di scegliere il miglior algoritmo in quella situazione.
- Il problema dell'ordinamento è una buona “palestra” dove mostrare questi concetti.

Algoritmi trattati: **Selection Sort**, **Insertion Sort**, **Merge Sort**.

Tipologie di analisi:

- **Analisi del caso peggiore:** la più importante.
 - Il tempo di esecuzione nel caso peggiore è un **limite superiore** al tempo di esecuzione per qualsiasi input.
 - Per alcuni algoritmi il caso peggiore si verifica molto spesso (es.: ricerca di dati non presenti in un database).
- **Analisi del caso medio:** difficile in alcuni casi — cosa si intende per “medio”? *Distribuzione uniforme.*
- **Analisi del caso migliore:** può avere senso se si hanno informazioni particolari sull’input.

2.6.2 Il Problema dell’Ordinamento

Definizione: Problema dell’Ordinamento

- **Input:** una sequenza $A = a_1, a_2, \dots, a_n$ di n valori.
- **Output:** una sequenza $B = b_1, b_2, \dots, b_n$ che sia una **permutazione** di A e tale per cui $b_1 \leq b_2 \leq \dots \leq b_n$.

Due approcci:

- approccio “**demente**”: si generano tutte le possibili permutazioni fino a quando non se ne trova una già ordinata;
- approccio “**naïf**”: si cerca il minimo e lo si mette in posizione corretta, riducendo il problema agli $n - 1$ valori restanti.

2.6.3 Selection Sort

A ogni iterazione si cerca il minimo della parte non ordinata e lo si scambia con l’elemento in posizione i , estendendo il prefisso ordinato.

```
SelectionSort(int[] A, int n)
for i = 0 to n-2 do
    int pos = argmin(A, i, n-1)    % Minimo della parte non ordinata
    A[i], A[pos] = A[pos], A[i]    % Estende il prefisso ordinato

int argmin(int[] A, int start, int end)
    int pos = start                % Posizione del minimo parziale
    for j = start+1 to end do
        if A[j] < A[pos] then
            pos = j                % Nuovo minimo parziale
    return pos
```

Esecuzione (ogni riga mostra il vettore all’inizio dell’iterazione i ; l’ultima riga è il risultato finale):

i	A						
0	7	4	2	1	8	3	5
1	1	4	2	7	8	3	5
2	1	2	4	7	8	3	5
3	1	2	3	7	8	4	5
4	1	2	3	4	8	7	5
5	1	2	3	4	5	7	8
6	1	2	3	4	5	7	8

Costo. All'iterazione i , `argmin()` confronta l'elemento candidato con i successivi $n - i - 1$ elementi:

$$\sum_{i=0}^{n-2} (n - i - 1) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2} = \Theta(n^2).$$

Il Costo Non Dipende dall'Ordine Iniziale

Selection Sort esamina comunque tutta la parte non ordinata:

$$\text{caso migliore} = \text{caso medio} = \text{caso peggiore} = \Theta(n^2).$$

2.6.4 Insertion Sort

Algoritmo efficiente per ordinare **piccoli insiemi** di elementi. Si basa sul principio di ordinamento di una “mano” di carte da gioco (ad es. *scala quaranta*).

```

InsertionSort(int[] A, int n)
for i = 1 to n-1 do
    int temp = A[i]           % Elemento da inserire
    int j = i                 % Scorre il prefisso ordinato
    while j > 0 and A[j-1] > temp do
        A[j] = A[j-1]        % Sposta a destra un elemento maggiore
        j = j - 1
    A[j] = temp               % Inserisce nella posizione corretta

```

Esecuzione su $A = [7, 4, 2, 1, 8, 3, 5]$. Ogni riga è lo stato del vettore dopo la scrittura in posizione j durante l'iterazione i ; in rosso la cella scritta, sottolineata quando è l'inserimento finale di `temp` (convenzione ripresa dalle slide).

	0	1	2	3	4	5	6	$temp$
<i>iniziale</i>	7	4	2	1	8	3	5	–
$i=1, j=1$	7	7	2	1	8	3	5	4
$i=1, j=0$	<u>4</u>	7	2	1	8	3	5	4
$i=2, j=2$	4	7	7	1	8	3	5	2
$i=2, j=1$	4	4	7	1	8	3	5	2
$i=2, j=0$	<u>2</u>	4	7	1	8	3	5	2
$i=3, j=3$	2	4	7	7	8	3	5	1
$i=3, j=2$	2	4	4	7	8	3	5	1
$i=3, j=1$	2	2	4	7	8	3	5	1
$i=3, j=0$	<u>1</u>	2	4	7	8	3	5	1
$i=4, j=4$	1	2	4	7	<u>8</u>	3	5	8
$i=5, j=5$	1	2	4	7	8	8	5	3 [†]
$i=5, j=4$	1	2	4	7	7	8	5	3
$i=5, j=3$	1	2	4	4	7	8	5	3
$i=5, j=2$	1	2	<u>3</u>	4	7	8	5	3
$i=6, j=6$	1	2	3	4	7	8	8	5
$i=6, j=5$	1	2	3	4	7	7	8	5
$i=6, j=4$	1	2	3	4	<u>5</u>	7	8	5

Correzione di un Refuso nelle Slide

[†] Nella slide 29/51 la colonna $temp$ di questa riga riporta 5. È un refuso: $temp = A[5] = 3$ (vettore iniziale 7, 4, 2, 1, 8, 3, 5) e le tre righe successive mostrano infatti $temp = 3$. Ho riportato il valore corretto.

Invariante e Correttezza

All'inizio dell'iterazione i , il prefisso $A[0 \dots i - 1]$ è **ordinato**. L'iterazione inserisce $A[i]$ nella posizione corretta ed estende il prefisso ordinato fino ad $A[0 \dots i]$.

Il costo dipende dalla disposizione dei dati:

Caso	Struttura dell'input	Costo
Migliore	già ordinato	$\Theta(n)$
Medio	permutazioni equiprobabili	$\Theta(n^2)$
Peggior	ordinato al contrario	$\Theta(n^2)$

2.6.5 Merge Sort

Divide et Impera. Merge Sort si basa sulla tecnica divide-et-impera vista in precedenza:

- **Divide:** spezza *virtualmente* il vettore di n elementi in due sottovettori di $n/2$ elementi;
- **Impera:** chiama Merge Sort ricorsivamente sui due sottovettori;
- **Combina:** unisci (*merge*) le due sequenze ordinate.

Idea: si sfrutta il fatto che i due sottovettori sono già ordinati per ordinare più velocemente.

La Procedura Merge.

- **Input:** due porzioni adiacenti già ordinate, $A[\text{start} \dots \text{mid}]$ e $A[\text{mid} + 1 \dots \text{end}]$, e un vettore di appoggio B .
- **Idea:** si confronta il primo elemento non ancora utilizzato delle due porzioni. Il minore è anche il minore fra tutti gli elementi rimanenti e può essere copiato nella prossima posizione libera di B .
- **Output:** la porzione $A[\text{start} \dots \text{end}]$ contiene tutti gli elementi delle due porzioni, in ordine non decrescente.

Nota grafica: la slide mostra, ad ogni passo, i puntatori i e j (sulle due porzioni di A) e k (su B). Stati per $A = [1, 3, 7, 8 \mid 2, 4, 6, 9]$:

Stato	$A[i]$	$A[j]$	Contenuto di B
1	1	2	(vuoto)
2	3	2	1
3	3	4	1 2
4	7	4	1 2 3
5	7	6	1 2 3 4
6	7	9	1 2 3 4 6
7	8	9	1 2 3 4 6 7
8	$i > \text{mid}$	9	1 2 3 4 6 7 8

Ricopiando B in A si ottiene $A = 1, 2, 3, 4, 6, 7, 8, 9$.

```

Merge(int[] A, int[] B, int start, int mid, int end)
int i = start, j = mid + 1, k = start
while i <= mid and j <= end do
    if A[i] <= A[j] then
        B[k] = A[i]                % Prende dalla porzione sinistra
        i = i + 1
    else
        B[k] = A[j]                % Prende dalla porzione destra
        j = j + 1
    k = k + 1
while i <= mid do
    B[k] = A[i]                    % Copia quel che resta a sinistra
    i = i + 1; k = k + 1
for t = start to k-1 do
    A[t] = B[t]                    % Ricopia il risultato in A

```

Nota mia (non nelle slide): perché un solo ciclo di copia?

Il codice copia il resto della porzione *sinistra* ma non quello della *destra*, e non è una dimenticanza. Se la porzione sinistra si esaurisce per prima, al termine del primo ciclo $k = j$: si ricopiano in A gli elementi $B[\text{start} \dots k - 1]$, mentre $A[j \dots \text{end}]$ (già ordinato e maggiore o uguale a tutto ciò che è in B) è già al suo posto. Se invece si esaurisce prima la destra, è il secondo ciclo **while** a portare in B ciò che resta a sinistra.

Domanda: qual è il costo computazionale di Merge()? $\Rightarrow \Theta(n)$, con n numero di elementi da fondere.

Merge Sort: Programma Completo.

```

MergeSort(int[] A, int n)
int[] B = new int[0 ... n-1]           % Vettore di appoggio
MergeSortRec(A, B, 0, n-1)

MergeSortRec(int[] A, int[] B, int start, int end)
if start < end then
    int mid = floor((start + end)/2)    % Divide
    MergeSortRec(A, B, start, mid)      % Parte sinistra
    MergeSortRec(A, B, mid+1, end)      % Parte destra
    Merge(A, B, start, mid, end)        % Combina

```

Merge Sort: Esecuzione. Su $A = [33, 21, 7, 48, 28, 13, 65, 17]$. **Fase Divide** (suddivisione del problema):

Livello	Sottovettori
0	[33, 21, 7, 48, 28, 13, 65, 17]
1	[33, 21, 7, 48] [28, 13, 65, 17]
2	[33, 21] [7, 48] [28, 13] [65, 17]
3	[33] [21] [7] [48] [28] [13] [65] [17]

Fase Combina (si risale fondendo a coppie, fino alla sequenza ordinata):

Livello	Sottovettori ordinati
2	[21, 33] [7, 48] [13, 28] [17, 65]
1	[7, 21, 33, 48] [13, 17, 28, 65]
0	[7, 13, 17, 21, 28, 33, 48, 65]

Merge Sort: Analisi. **Assunzione semplificativa:** $n = 2^k$, quindi la profondità è esattamente $k = \log n$; tutti i sottovettori hanno dimensione pari a una potenza di 2.

$$T(n) = \begin{cases} d & n = 1 \\ 2T(n/2) + cn & n > 1 \end{cases}$$

Analisi per livelli: albero di ricorsione binario, con 2^i nodi di dimensione $n/2^i$ al livello i e $2^{\log n}$ foglie di dimensione 1.

Livello	# nodi	Dim	Costo del livello
0	2^0	$n/2^0$	$c \cdot n$
1	2^1	$n/2^1$	$c \cdot n$
2	2^2	$n/2^2$	$c \cdot n$
i	2^i	$n/2^i$	$c \cdot n$
$\log n$	$2^{\log n}$	1	$d \cdot 2^{\log n}$

Ogni livello interno costa cn in totale, e i livelli interni sono $\log n$:

$$T(n) = \sum_{i=0}^{\log n - 1} cn + dn = cn \log n + dn = \Theta(n \log n).$$

Un po' di Storia.

- Il censimento americano del 1880 aveva richiesto otto anni per essere completato; quello del 1890 richiese sei mesi, grazie alla **Hollerith Machine**.
- Fra il 1896 e il 1924, la Hollerith & Co. ha cambiato diversi nomi. L'ultimo? *International Business Machines*.
- Le **Collating Machines** (1936) fondevano due mazzi di schede perforate già ordinati in un unico mazzo ordinato.
- Nel 1945–48, John von Neumann descrisse per la prima volta il MergeSort partendo dall'idea delle Collating Machines.

Nota grafica: la slide riporta una foto della Hollerith Machine.

2.6.6 Limite Inferiore per l'Ordinamento

Ordinamento per confronti:

- si considerano algoritmi che ordinano confrontando coppie di elementi;
- si suppone, per semplicità, che gli elementi siano distinti;
- esistono $n!$ possibili ordinamenti dell'input;
- ogni confronto ha due esiti e corrisponde a un nodo di un **albero di decisione**;
- ogni foglia (terminazione) è associata a uno dei possibili ordinamenti.

Esempio di albero di decisione (ordinamento di tre valori a, b, c):

```
if a < b then
  if b < c then
    print a, b, c
  else
    if a < c then
      print a, c, b
    else
      print c, a, b
else
  if a < c then
    print b, a, c
  else
    if b < c then
      print b, c, a
    else
      print c, b, a
```

Nota grafica: le slide affiancano al codice l'albero di decisione corrispondente: la radice confronta $a < b$; i due nodi al livello successivo confrontano $b < c$ (ramo *True*) e $a < c$ (ramo *False*); si arriva così a $6 = 3!$ foglie, ciascuna etichettata con una delle permutazioni di a, b, c .

Teorema: Il Limite Inferiore è $\Omega(n \log n)$

Dalle foglie all'altezza. Un albero binario di altezza h contiene al massimo 2^h foglie. Per distinguere $n!$ ordinamenti deve quindi valere:

$$2^h \geq n! \quad \implies \quad h \geq \log_2(n!).$$

Stima di $\log n!$. Assumiamo n pari. Almeno $n/2$ fattori di $n!$ sono maggiori o uguali a $n/2$:

$$\log_2(n!) \geq \log_2 \left(\left(\frac{n}{2} \right)^{n/2} \right) = \frac{n}{2} \log_2 \left(\frac{n}{2} \right) = \Omega(n \log n).$$

Ogni algoritmo di ordinamento basato su confronti ha dunque, nel caso peggiore, un costo $\Omega(n \log n)$: Merge Sort (Sezione 2.6) è pertanto **asintoticamente ottimo**, nel senso della Definizione 2.5.3.

2.6.7 Counting Sort

Counting Sort Cambia le Ipotesi

Assunzione più forte: gli elementi di A sono interi compresi nell'intervallo $[0 \dots k-1]$. **Idea:** si usa $B[0 \dots k-1]$ per contare quante volte compare ciascun valore, poi si scorre B in ordine e si riscrivono in A le occorrenze contate. Counting Sort **non confronta** gli elementi fra loro: sfrutta direttamente il valore delle chiavi come indice.

```
countingSort(int[] A, int n, int k)
int[] B = new int[0 ... k-1] = {0}      % Contatori delle occorrenze
for j = 0 to n-1 do
    B[A[j]] += 1                          % Conta A[j]
j = 0
for i = 0 to k-1 do
    while B[i] > 0 do
        A[j] = i
        j += 1
        B[i] -= 1                          % Consuma un'occorrenza di i
```

Costo: $T(n, k) = \Theta(n + k)$.

Perché Non Contraddice il Limite Inferiore?

Il limite $\Omega(n \log n)$ (Teorema 2.6.6) vale per l'ordinamento **per confronti**. Counting Sort non è basato su confronti e richiede chiavi intere in un intervallo noto: il modello e le ipotesi sono cambiati.

- se $k = O(n)$, Counting Sort è **lineare**;
- se $k = \Omega(n \log n)$, non è asintoticamente più vantaggioso di Merge Sort;
- se $k = \Theta(2^n)$, il costo $\Theta(n + k)$ è addirittura **superpolinomiale** in n .

Un limite inferiore ha senso soltanto insieme alle operazioni consentite e alle informazioni disponibili sull'input.

Sommario: algoritmi di ordinamento.

Algoritmo	Migliore	Medio	Peggior	Stabile	Spazio
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	No	$\Theta(1)$
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	Sì	$\Theta(1)$
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	Sì	$\Theta(n)$
Counting Sort	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	No	$\Theta(k)$
Shell Sort	$\Theta(n)$	$O(n\sqrt{n})$	$O(n\sqrt{n})$	No	$\Theta(1)$
Quick Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	No	$\Theta(\log n)$ medio
Heap Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	No	$\Theta(1)$

Definizione: Stabilità

Un algoritmo è **stabile** se gli elementi con la stessa chiave mantengono l'ordine relativo che avevano nell'input.

Nota

Shell Sort, Quick Sort e Heap Sort compaiono qui solo in questa tabella riassuntiva, come anticipazione: non sono stati presentati in dettaglio in questo file (verosimilmente oggetto di una lezione successiva non ancora caricata).

2.6.8 Ordine Asintotico e Costanti

- per input **grandi**, l'ordine di crescita domina i fattori costanti;
- per input **piccoli** contano anche chiamate, allocazioni, memoria e architettura;
- la soglia di incrocio dipende da implementazione e macchina.

Nota grafica: un grafico sperimentale (tempo in ms vs. n da 0 a 128) confronta Insertion Sort e Merge Sort: per n piccolo Insertion Sort è più veloce, ma le due curve si incrociano attorno a $n \approx 60-64$, oltre il quale Merge Sort diventa preferibile.

Conseguenza Pratica

Nell'esperimento mostrato, un'implementazione di Merge Sort che utilizza Insertion Sort per $n \leq 64$ è più efficiente di quella che utilizza $n = 1$ come caso base – una tecnica ibrida comune nelle librerie reali.

Lo stesso principio vale per la moltiplicazione (Sezione 2.5): l'**evoluzione storica** degli algoritmi di moltiplicazione in bit-complexity mostra il progressivo abbassamento dell'esponente asintotico:

Algoritmo	Anno	Complessità in bit
Moltiplicazione scolastica	—	$\Theta(n^2)$
Karatsuba–Ofman (Toom-2)	1962	$\Theta(n^{\log_2 3})$, $\log_2 3 \simeq 1.585$
Toom–Cook (Toom-3)	1963	$\Theta(n^{\log_3 5})$, $\log_3 5 \simeq 1.465$
Toom–Cook (Toom-4)	2006	$\Theta(n^{\log_4 7})$, $\log_4 7 \simeq 1.404$
Toom–Cook (Toom-6.5)	2009	$\Theta(n^{\log_6 12})$, $\log_6 12 \simeq 1.387$
Toom–Cook (Toom-8.5)	2009	$\Theta(n^{\log_8 16}) = \Theta(n^{4/3})$
Schönhage–Strassen (FFT)	1971	$O(n \log n \log \log n)$
Fürer	2007	$O(n \log n 2^{O(\log^* n)})$
Harvey–van der Hoeven–Lecerf	2014	$O(n \log n 8^{\log^* n})$
Harvey–van der Hoeven	2018	$O(n \log n 4^{\log^* n})$
Harvey–van der Hoeven	2021	$O(n \log n)$

($\log^* n$ è il *logaritmo iterato*: il numero di volte che si può applicare $\log$ prima che il risultato scenda a ≤ 1 – cresce “praticamente mai” per n concretamente rappresentabili.)

GMP Combina Algoritmi Diversi

La *GNU Multiple Precision Arithmetic Library* (GMP) descrive, nel proprio manuale, una successione di algoritmi scelta in funzione della dimensione degli operandi:

$$\text{Scolastico} \xrightarrow{\tau_1} \text{Karatsuba} \xrightarrow{\tau_2} \text{Toom–Cook} \xrightarrow{\tau_3} \text{FFT}$$

Le soglie τ_1, τ_2, τ_3 dipendono da architettura, compilatore e implementazione: vengono determinate **sperimentalmente**, non dalla sola notazione asintotica.

Filo Conduttore della Sezione

Notazione asintotica e costanti pratiche non sono in conflitto, ma rispondono a domande diverse: l'una dice come si comporta un algoritmo per $n \rightarrow \infty$, le altre decidono quale algoritmo conviene usare per l' n che si ha davvero – da qui le implementazioni ibride (Merge/Insertion Sort) e le librerie che “combinano algoritmi diversi” (GMP).

2.7 Ricorrenze e Analisi per Livelli

2.7.1 Introduzione

Definizione: Relazione di Ricorrenza

Il costo di un algoritmo ricorsivo dipende dal costo di una o più chiamate su input più piccoli. Questa dipendenza è descritta da una **relazione di ricorrenza**.

Esempio (Merge Sort), già incontrata nella Sezione 2.6:

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) & n > 1, \\ \Theta(1) & n \leq 1. \end{cases}$$

Forma Chiusa

L'obiettivo non è necessariamente trovare una formula esatta, ma una **forma chiusa** che esprima l'ordine di crescita. Per Merge Sort: $T(n) = \Theta(n \log n)$.

Oltre l'Analisi di Algoritmi. Le ricorrenze descrivono anche quantità definite tramite istanze più piccole dello stesso problema, non solo il costo di un algoritmo.

Esempio: Il Bambino sulle Scale

Un bambino scende una scala; ad ogni passo può scendere 1, 2, 3 oppure 4 scalini. Sia $M(n)$ il numero di modi per scendere n scalini:

$$M(n) = \begin{cases} 0 & n < 0, \\ 1 & n = 0, \\ \sum_{j=1}^4 M(n-j) & n > 0. \end{cases}$$

Numeri di **tetranacci**: 1, 1, 2, 4, 8, 15, 29, 56, 108, 208, 401, 773, ...

2.7.2 Analisi per Livelli

Tre Metodi Complementari

- **Analisi per livelli**: rende visibili numero e costo delle chiamate.
- **Sostituzione**: formula un'ipotesi e la dimostra per induzione.
- **Teoremi degli esperti**: risolvono direttamente famiglie di ricorrenze.

Le quantità da calcolare a ogni livello: numero di chiamate e dimensione dei sottoproblemi; costo non ricorsivo di una singola chiamata; costo complessivo del livello.

Semplificazione Adottata nel Seguito

Non si utilizzano valori costanti come c e d per descrivere le costanti moltiplicative del lavoro di ricombinazione e del caso base: si utilizza il valore 1 per semplificare i calcoli. Le costanti moltiplicative vengono assorbite dalle notazioni asintotiche.

Una Sola Chiamata: Espansione Lineare.

$$T(n) = \begin{cases} T(n/2) + n & n > 1, \\ 1 & n \leq 1. \end{cases}$$

Assunzioni: $n = 2^k$ con k intero, $k = \log n$.

$$\begin{aligned} T(n) &= T(n/2) + n \\ &= T(n/4) + \frac{n}{2} + n \\ &= T(n/8) + \frac{n}{4} + \frac{n}{2} + n \\ &\vdots \\ &= T(1) + \sum_{i=0}^{\log n - 1} \frac{n}{2^i}. \end{aligned}$$

Più Chiamate: Organizziamo il Calcolo. Si confrontano tre ricorrenze con **quattro** sottoproblemi di dimensione $n/2$:

$$4T(n/2) + n, \quad 4T(n/2) + n^2, \quad 4T(n/2) + n^3.$$

La Domanda Decisiva

Come cambia il costo complessivo di un livello quando scendiamo nell'albero delle chiamate?

- il numero di chiamate cresce di un fattore 4;
- la dimensione di ciascun sottoproblema si dimezza;
- il lavoro non ricorsivo determina quale effetto prevale.

Primo Caso: $T(n) = 4T(n/2) + n$

Livello	N. chiamate	Dimensione	Costo/chiamata	Costo del livello
0	1	n	n	n
1	4	$n/2$	$n/2$	$n \cdot 2$
2	4^2	$n/2^2$	$n/2^2$	$n \cdot 4$
i	4^i	$n/2^i$	$n/2^i$	$n \cdot 2^i$
k	4^k	1	1	n^2

$$T(n) = n \sum_{i=0}^{k-1} 2^i + n^2 = n(n-1) + n^2 \quad (k = \log n, 4^{\log n} = n^2)$$

$$T(n) = \Theta(n^2)$$

Il costo cresce: **dominano gli ultimi livelli** e le sole foglie costano già n^2 .

Secondo Caso: $T(n) = 4T(n/2) + n^2$

Livello	N. chiamate	Dimensione	Costo/chiamata	Costo del livello
0	1	n	n^2	n^2
1	4	$n/2$	$(n/2)^2$	n^2
2	4^2	$n/2^2$	$(n/2^2)^2$	n^2
i	4^i	$n/2^i$	$(n/2^i)^2$	n^2
k	4^k	1	1	n^2

$$T(n) = \sum_{i=0}^{k-1} n^2 + n^2 = n^2 \log n + n^2 = \Theta(n^2 \log n)$$

L'aumento del numero di chiamate compensa esattamente la diminuzione del costo per chiamata: **tutti i livelli hanno lo stesso costo**.

Terzo Caso: $T(n) = 4T(n/2) + n^3$

Livello	N. chiamate	Dimensione	Costo/chiamata	Costo del livello
0	1	n	n^3	n^3
1	4	$n/2$	$(n/2)^3$	$n^3/2$
2	4^2	$n/2^2$	$(n/2^2)^3$	$n^3/2^2$
i	4^i	$n/2^i$	$(n/2^i)^3$	$n^3/2^i$
k	4^k	1	1	n^2

$$T(n) = n^3 \sum_{i=0}^{k-1} \left(\frac{1}{2}\right)^i + n^2 \leq 2n^3 + n^2 = O(n^3)$$

Poiché la sola radice costa n^3 , vale anche $T(n) = \Omega(n^3)$; dunque $T(n) = \Theta(n^3)$. **Dominano i primi livelli**.

Tre Comportamenti, una Sola Idea

Ricorrenza	Complessità	Interpretazione
$4T(n/2) + n$	$\Theta(n^2)$	dominano gli ultimi livelli
$4T(n/2) + n^2$	$\Theta(n^2 \log n)$	tutti i livelli costano uguale
$4T(n/2) + n^3$	$\Theta(n^3)$	dominano i primi livelli

Verso il Master Theorem

La complessità dipende dal confronto fra la crescita del numero di sottoproblemi e il costo del lavoro non ricorsivo.

2.8 Ricorrenze: Metodo di Sostituzione

2.8.1 Introduzione

Metodi per risolvere ricorrenze: analisi per livelli (Sezione 2.7); metodo di sostituzione, o *per tentativi*; metodo dell'esperto, o *delle ricorrenze comuni*.

Metodo di Sostituzione

È un metodo in cui si cerca di “indovinare” (*guess*) una soluzione, in base alla propria esperienza, e si dimostra che questa soluzione è corretta tramite **induzione**.

2.8.2 Primo Esempio: $T(n) = T(\lfloor n/2 \rfloor) + n$

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Nota grafica: le slide costruiscono progressivamente un diagramma a barre impilate: alla riga t ($t = 1, \dots, 6$) si affianca un blocco di larghezza n , poi $n/2$, poi $n/4$, poi $n/8 \dots$ (uno in più a ogni riga, fino a stabilizzarsi), con una freccia orizzontale finale che indica che il totale tende a $2n$ – la rappresentazione visiva della somma geometrica sotto.

$$T(n) = n \sum_{i=0}^{\log n} \left(\frac{1}{2}\right)^i \leq n \sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i = n \cdot \frac{1}{1 - \frac{1}{2}} = 2n$$

Serie Geometrica Decrescente Infinita

$$\forall x, |x| < 1 : \sum_{i=0}^{\infty} x^i = \frac{1}{1 - x}$$

Limite Superiore. Tentativo: $T(n) = O(n)$, cioè $\exists c > 0, \exists m \geq 0 : T(n) \leq cn, \forall n \geq m$.

Caso base – dimostriamo la disequazione per $T(1)$:

$$T(1) = 1 \stackrel{?}{\leq} 1 \cdot c \iff \forall c \geq 1$$

Ipotesi induttiva: $\forall k < n : T(k) \leq ck$. *Passo di induzione* – dimostriamo la disequazione per $T(n)$:

$T(n) = T(\lfloor n/2 \rfloor) + n$	
$\leq c\lfloor n/2 \rfloor + n$	Sostituzione
$\leq cn/2 + n$	Intero inferiore
$= (c/2 + 1)n$	Passo algebrico
$\stackrel{?}{\leq} cn$	Obiettivo
$\iff c/2 + 1 \leq c \iff c \geq 2$	Risultato finale

Conclusione

Abbiamo provato che $T(n) \leq cn$: nel caso base $c \geq 1$, nel passo induttivo $c \geq 2$; un valore c che rispetta entrambe le disequazioni è $c = 2$. Questo vale per $n = 1$ e per tutti i valori seguenti, quindi $m = 1$. **Abbiamo quindi provato che $T(n) = O(n)$.**

Limite Inferiore. Tentativo: $T(n) = \Omega(n)$, cioè $\exists d > 0, \exists m \geq 0 : T(n) \geq dn, \forall n \geq m$.
Caso base – dimostriamo la disequazione per $T(1)$:

$$T(1) = 1 \stackrel{?}{\geq} 1 \cdot d \iff \forall d \leq 1$$

Ipotesi induttiva: $\forall k < n : T(k) \geq dk$. *Passo di induzione:*

$$\begin{aligned} T(n) &= T(\lfloor n/2 \rfloor) + n \\ &\geq d\lfloor n/2 \rfloor + n && \text{Sostituzione} \\ &\geq dn/2 - 1 + n && \text{Intero inferiore} \\ &= \left(\frac{d}{2} - \frac{1}{n} + 1 \right) n \stackrel{?}{\geq} dn && \text{Passo algebrico} \\ &\iff \frac{d}{2} - \frac{1}{n} + 1 \geq d \iff d \leq 2 - \frac{2}{n} && \text{Risultato finale} \end{aligned}$$

Conclusione

Nel caso base $d \leq 1$; nel passo induttivo $d \leq 2 - 2/n$. Un valore d che rispetta entrambe le disequazioni, per ogni $n \geq 2$, è $d = 1$. Questo vale per $n = 2$ e per tutti i valori seguenti, quindi $m = 2$. Abbiamo quindi provato che $T(n) = \Omega(n)$, e dunque, insieme al limite superiore:

$$T(n) = O(n) \wedge T(n) = \Omega(n) \iff T(n) = \Theta(n).$$

Una Scorciatoia per il Limite Inferiore

È possibile dimostrare che $T(n) = \Omega(n)$ in maniera molto più semplice, **senza fare nemmeno ricorso all'ipotesi induttiva**:

$$T(n) = T(\lfloor n/2 \rfloor) + n \geq n \stackrel{?}{\geq} dn$$

L'ultima disequazione è vera per $d \leq 1$, condizione identica a quella del caso base.

Nota

Questa scorciatoia funziona perché il termine non ricorsivo (n) è *da solo* già dell'ordine cercato: quando accade, si può maggiore/minorare l'intera espressione buttando via la parte ricorsiva (che è comunque non negativa) e verificare la disequazione sul solo termine residuo.

2.8.3 Un Tentativo Sbagliato: $T(n) = T(n-1) + n$

$$T(n) = \begin{cases} T(n-1) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Nota grafica: le slide disegnano una “piramide” a gradini: al livello 0 un blocco di larghezza n , al livello 1 un blocco $n - 1$, al livello 2 un blocco $n - 2$, e così via fino a 2 al penultimo livello e 1 all’ultimo – la somma dei blocchi è l’area della piramide.

$$T(n) = \sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$$

Tentativo Sbagliato: $T(n) = O(n)$

Ipotesi induttiva: $\forall k < n : T(k) \leq ck$. **Passo di induzione:**

$$\begin{aligned} T(n) &= T(n-1) + n \\ &\leq c(n-1) + n && \text{Sostituzione} \\ &\leq (c+1)n - c && \text{Passo algebrico} \\ &\stackrel{?}{\leq} cn && \text{Obiettivo} \\ \implies n - c &\leq 0 && \text{Semplificando } cn \\ \implies c &\geq n && c \text{ non è costante} \end{aligned}$$

Il tentativo **fallisce**: non esiste una costante c indipendente da n che soddisfi la disequazione – coerente con il fatto che la vera complessità è $\Theta(n^2)$, non $\Theta(n)$.

2.8.4 Difficoltà Matematica: un Termine di Ordine Inferiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Nota grafica: albero binario disegnato per livelli: il livello i contiene 2^i nodi, ciascuno di costo 1, per un costo di livello pari a 2^i ; l’ultimo livello ($\log n$) contiene $2^{\log n} = n$ nodi.

$$T(n) = \sum_{i=0}^{\log n} 2^i = O(n)$$

Il Tentativo Diretto Fallisce

Tentativo: $T(n) = O(n)$. **Ipotesi induttiva:** $\forall k < n : T(k) \leq ck$. **Passo di induzione:**

$$\begin{aligned} T(n) &= T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 \\ &\leq c\lfloor n/2 \rfloor + c\lceil n/2 \rceil + 1 && \text{Sostituzione} \\ &= cn + 1 && \text{Passo algebrico} \\ &\stackrel{?}{\leq} cn && \text{Obiettivo} \\ \implies 1 &\leq 0 && \text{Impossibile} \end{aligned}$$

Cosa Succede?

Il tentativo è **corretto**... ma non riusciamo a dimostrarlo, perché resta un termine di ordine inferiore che non si elide mai: $cn + 1 \stackrel{?}{\leq} cn$.

La Soluzione: un'Ipotesi Induttiva più Stretta

Ipotesi induttiva più stretta: $\exists b > 0, \forall k < n : T(k) \leq ck - b$. **Passo di induzione:**

$$\begin{aligned}
 T(n) &= T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 \\
 &\leq c\lfloor n/2 \rfloor - b + c\lceil n/2 \rceil - b + 1 && \text{Sostituzione} \\
 &= cn - 2b + 1 && \text{Passo algebrico} \\
 &\stackrel{?}{\leq} cn - b && \text{Obiettivo} \\
 \implies -2b + 1 &\leq -b \implies b \geq 1 && \text{Passo algebrico}
 \end{aligned}$$

Caso base – dimostriamo la disequazione per $T(1)$:

$$T(1) = 1 \stackrel{?}{\leq} 1 \cdot c - b \iff \forall c \geq b + 1$$

Conclusione: nel passo induttivo $\forall b \geq 1, \forall c$; nel caso base $\forall c \geq b + 1$. Una coppia (b, c) che rispetta entrambe è $b = 1, c = 2$. Vale per $n = 1$ e tutti i successivi, quindi $m = 1$. **Abbiamo quindi provato che** $T(n) = O(n)$.

Limite Inferiore – Nessuna Difficoltà

Passo di induzione: $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 \geq d\lfloor n/2 \rfloor + d\lceil n/2 \rceil + 1 = dn + 1 \stackrel{?}{\geq} dn$, vero per ogni d . **Caso base:** $T(1) = 1 \geq d \cdot 1 \iff d \leq 1$. Abbiamo quindi provato che $T(n) = \Omega(n)$ (qui il termine di ordine inferiore aiuta anziché ostacolare, perché va nella direzione giusta della disuguaglianza).

2.8.5 Problemi con i Casi Base

$$T(n) = \begin{cases} 2T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Nota grafica: stesso tipo di diagramma a livelli: il livello 0 ha 1 nodo di costo n ; il livello 1 ha 2 nodi di costo $n/2$ ciascuno; il livello i ha 2^i nodi di costo $n/2^i$; il penultimo livello ha nodi di costo 2; l'ultimo (foglie) ha nodi di costo 1.

Tentativo: $T(n) = O(n \log n)$

Ipotesi induttiva: $\exists c > 0, \forall k < n : T(k) \leq ck \log k$. **Passo di induzione:**

$$\begin{aligned}
 T(n) &= 2T(\lfloor n/2 \rfloor) + n \\
 &\leq 2c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n && \text{Sostituzione} \\
 &\leq 2c \frac{n}{2} \log \frac{n}{2} + n && \text{Intero inferiore} \\
 &= cn(\log n - 1) + n = cn \log n - cn + n && \text{Passo algebrico} \\
 &\stackrel{?}{\leq} cn \log n && \text{Obiettivo} \\
 \implies -cn + n &\leq 0 \implies c \geq 1 && \text{Eliminazione } cn \log n
 \end{aligned}$$

Il Caso Base Fallisce – e Non è un Problema

$$T(1) = 1 \stackrel{?}{\leq} 1 \cdot c \log 1 = 0 \implies 1 \not\leq 0$$

Cosa succede? È falso, ma non è un problema: non a caso si chiama *notazione asintotica*. Il valore iniziale di m lo possiamo scegliere noi.

Si scelgono nuovi casi base, $T(2)$ e $T(3)$:

$$\begin{aligned} T(2) &= 2T(\lfloor 2/2 \rfloor) + 2 = 4 \stackrel{?}{\leq} c \cdot 2 \log 2 \iff c \geq 2 \\ T(3) &= 2T(\lfloor 3/2 \rfloor) + 3 = 5 \stackrel{?}{\leq} c \cdot 3 \log 3 \iff c \geq \frac{5}{3 \log 3} \\ T(4) &= 2T(\lfloor 4/2 \rfloor) + 4 = 2T(2) + 4 \end{aligned}$$

Non è necessario dimostrare una terza disequazione: $T(4)$ dipende da $T(2)$, un caso base **già dimostrato**, quindi può far parte del fondamento dell'induzione.

Conclusione

Nel passo induttivo $\forall c \geq 1$; nei casi base $c \geq 2$, $c \geq 5/(3 \log 3)$. Essendo tutte disequazioni con lo stesso verso, basta $c \geq \max\{1, 2, 5/(3 \log 3)\}$. Vale per $n = 2, 3$ e tutti i successivi, quindi $m = 2$.

2.8.6 Together Now: le Due Difficoltà Insieme

$$T(n) = \begin{cases} 9T(\lfloor n/3 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Primo Tentativo: $T(n) = O(n^2)$, Ipotesi Semplice

Ipotesi induttiva: $\exists c > 0 : T(k) \leq ck^2, \forall k < n$. **Passo induttivo:**

$$\begin{aligned} T(n) &= 9T(\lfloor n/3 \rfloor) + n \\ &\leq 9c(\lfloor n/3 \rfloor)^2 + n && \text{Sostituzione} \\ &\leq 9c(n^2/9) + n && \text{Limite inferiore} \\ &= cn^2 + n && \text{Passo algebrico} \\ &\not\leq cn^2 && \text{Falso} \end{aligned}$$

Stesso problema dell'esempio precedente: il termine di ordine inferiore n non si elide mai.

Secondo Tentativo: Ipotesi più Stretta

Ipotesi induttiva: $\exists c > 0 : T(k) \leq c(k^2 - k), \forall k < n$. **Passo induttivo:**

$$\begin{aligned} T(n) &= 9T(\lfloor n/3 \rfloor) + n \\ &\leq 9c(\lfloor n/3 \rfloor^2 - \lfloor n/3 \rfloor) + n && \text{Sostituzione} \\ &\leq cn^2 - 3cn + n && \text{Limite inferiore} \\ &\stackrel{?}{\leq} cn^2 - cn && \text{Obiettivo} \\ &\iff c \geq \frac{1}{2} \end{aligned}$$

Casi Base

$$T(1) = 1 \stackrel{?}{\leq} c(1^2 - 1) = 0, \quad \text{falso: } T(1) \text{ non può essere caso base.}$$

Si verificano allora $T(2), \dots, T(5)$ (tutti dipendono solo da $T(1)$ o $T(0)$):

$$\begin{aligned} T(2) &= 9T(0) + 2 = 11 \stackrel{?}{\leq} c(4 - 2) \iff c \geq 11/2 \\ T(3) &= 9T(1) + 3 = 12 \stackrel{?}{\leq} c(9 - 3) \iff c \geq 12/6 \\ T(4) &= 9T(1) + 4 = 13 \stackrel{?}{\leq} c(16 - 4) \iff c \geq 13/12 \\ T(5) &= 9T(1) + 5 = 14 \stackrel{?}{\leq} c(25 - 5) \iff c \geq 14/20 \\ T(6) &= 9T(2) + 6 \end{aligned}$$

Non è necessario andare oltre $T(5)$, perché $T(6)$ dipende da $T(2)$, già dimostrato.

Parametri Finali

$$c \geq \max \left\{ \frac{1}{2}, \frac{11}{2}, \frac{12}{6}, \frac{13}{12}, \frac{14}{20} \right\} = \frac{11}{2}, \quad m = 2$$

Nota: l'Esempio è Artificiale

Questo esempio combina volutamente le due difficoltà viste separatamente, ma in modo artificioso: se si fosse scelta fin da subito l'ipotesi più stretta $T(n) \leq cn^2 - bn$ (invece di $T(n) \leq cn^2$), il problema sui casi base non si sarebbe posto.

2.8.7 Riassumendo

Metodo di Sostituzione

- si “indovina” una possibile soluzione e si formula un'ipotesi induttiva;
- si sostituisce nella ricorrenza le espressioni $T(\cdot)$, utilizzando l'ipotesi induttiva;
- si dimostra che la soluzione è valida anche per il caso base.

Attenzione

- ad ipotizzare soluzioni troppo “strette”;
- ad alcuni casi particolari che richiedono alcune astuzie matematiche;
- ai casi base: il logaritmo può complicare le cose.

2.9 Ricorrenze: Teoremi degli Esperti (Master Theorem)

2.9.1 Introduzione

Ricorrenze divide-et-impera. Molti algoritmi generano ricorrenze della forma

$$T(n) = aT(n/b) + f(n),$$

dove a è il numero di sottoproblemi, n/b la loro dimensione e $f(n)$ il lavoro di divisione e ricombinazione.

2.9.2 Master Theorem, Versione Ridotta

Teorema (Master Theorem, Versione Ridotta)

Siano $a \geq 1$ e $b > 1$ costanti, $c, d > 0$ e $\beta \geq 0$. Per

$$T(n) = \begin{cases} d & n \leq 1, \\ aT(n/b) + cn^\beta & n > 1, \end{cases}$$

considerata dapprima per n potenza di b , posto $\alpha = \log_b a$ vale

$$T(n) = \begin{cases} \Theta(n^\alpha) & \alpha > \beta, \\ \Theta(n^\alpha \log n) & \alpha = \beta, \\ \Theta(n^\beta) & \alpha < \beta. \end{cases}$$

Dimostrazione: Analisi per Livelli. Poniamo $n = b^k$, quindi $k = \log_b n$.

Livello	N. chiamate	Dimensione	Costo/chiamata	Costo del livello
0	1	b^k	$cb^{k\beta}$	$cb^{k\beta}$
1	a	b^{k-1}	$cb^{(k-1)\beta}$	$acb^{(k-1)\beta}$
2	a^2	b^{k-2}	$cb^{(k-2)\beta}$	$a^2cb^{(k-2)\beta}$
i	a^i	b^{k-i}	$cb^{(k-i)\beta}$	$a^i cb^{(k-i)\beta}$
$k-1$	a^{k-1}	b	cb^β	$a^{k-1} cb^\beta$
k	a^k	1	d	da^k

Poiché $a = b^\alpha$, abbiamo $a^k = n^\alpha$ e $b^{k\beta} = n^\beta$.

Dimostrazione: una Somma Geometrica. Poniamo $q = \frac{a}{b^\beta} = b^{\alpha-\beta}$. Il costo del livello interno i è $cn^\beta q^i$; sommando livelli interni e foglie otteniamo

$$T(n) = dn^\alpha + cn^\beta \sum_{i=0}^{k-1} q^i.$$

$\alpha > \beta$ $q > 1$	$\alpha = \beta$ $q = 1$	$\alpha < \beta$ $0 < q < 1$
il costo cresce	il costo è costante	il costo diminuisce

Dimostrazione: $\alpha > \beta$ e $\alpha = \beta$. **Caso 1** ($\alpha > \beta$, quindi $q > 1$):

$$q^{k-1} \leq \sum_{i=0}^{k-1} q^i = \frac{q^k - 1}{q - 1} \leq \frac{q^k}{q - 1}.$$

Poiché $q^k = n^{\alpha-\beta}$, i livelli interni costano $\Theta(n^\beta n^{\alpha-\beta}) = \Theta(n^\alpha)$; le foglie costano anch'esse dn^α .

Caso 2 ($\alpha = \beta$, quindi $q = 1$): tutti i k livelli interni costano cn^α :

$$T(n) = dn^\alpha + ckn^\alpha = n^\alpha(d + c \log_b n) = \Theta(n^\alpha \log n).$$

Dimostrazione: $\alpha < \beta$. **Caso 3** ($\alpha < \beta$, quindi $0 < q < 1$):

$$1 \leq \sum_{i=0}^{k-1} q^i \leq \sum_{i=0}^{\infty} q^i = \frac{1}{1-q}.$$

I livelli interni costano dunque $\Theta(n^\beta)$: le foglie costano $dn^\alpha = O(n^\beta)$, la sola radice costa cn^β ; pertanto $T(n) = \Theta(n^\beta)$.

Nota

La dimostrazione formalizza la stessa tricotomia osservata con l'analisi per livelli dei tre esempi $4T(n/2) + n^j$ della Sezione 2.7.

Versione Ridotta: Applicazioni.

Ricorrenza	a, b	Confronto	Soluzione
$3T(n/2) + n$	3, 2	$\log_2 3 > 1$	$\Theta(n^{\log_2 3})$
$3T(n/2) + 1$	3, 2	$\log_2 3 > 0$	$\Theta(n^{\log_2 3})$
$3T(n/2) + n^2$	3, 2	$\log_2 3 < 2$	$\Theta(n^2)$
$4T(n/2) + n^2$	4, 2	$\log_2 4 = 2$	$\Theta(n^2 \log n)$

Tre algoritmi già incontrati: Karatsuba (Sezione 2.5): $a = 3, b = 2, \beta = 1$; Merge Sort (Sezione 2.6): $a = b = 2, \beta = 1$; ricerca binaria: $a = 1, b = 2, \beta = 0$.

Potenze di b e Arrotondamenti. Perché assumiamo $n = b^k$? L'assunzione rende esatta la profondità $k = \log_b n$ e semplifica i calcoli. Nella realtà, le dimensioni dei sottoproblemi richiedono interi inferiori o superiori. Se T è non decrescente, possiamo racchiudere n fra due potenze consecutive di b :

$$b^{\lfloor \log_b n \rfloor} \leq n \leq b^{\lceil \log_b n \rceil} < bn.$$

Il *padding* cambia la dimensione per meno del fattore costante b e la profondità per al più un livello: l'ordine asintotico resta invariato.

2.9.3 Master Theorem, Versione Estesa

Teorema (Master Theorem, Versione Estesa)

Sia $T(n) = aT(n/b) + f(n)$, con $a \geq 1$, $b > 1$ costanti, $f(n)$ asintoticamente positiva e $\alpha = \log_b a$.

1. Se $f(n) = O(n^{\alpha-\epsilon})$ per qualche $\epsilon > 0$, allora $T(n) = \Theta(n^\alpha)$.
2. Se $f(n) = \Theta(n^\alpha)$, allora $T(n) = \Theta(n^\alpha \log n)$.
3. Se $f(n) = \Omega(n^{\alpha+\epsilon})$ per qualche $\epsilon > 0$ e $af(n/b) \leq cf(n)$ definitivamente, per qualche $c < 1$, allora $T(n) = \Theta(f(n))$.

Nei casi 1 e 3 serve una **distanza polinomiale**; nel caso 3 serve anche la **condizione di regolarità**.

Versione Estesa: Applicazioni.

Ricorrenza	α	Caso	Soluzione
$4T(n/2) + n \log n$	2	1	$\Theta(n^2)$
$T(2n/3) + 1$	0	2	$\Theta(\log n)$
$3T(n/4) + n \log n$	$\log_4 3$	3	$\Theta(n \log n)$

Per il terzo esempio la regolarità vale, perché

$$3f(n/4) = \frac{3}{4}n(\log n - \log 4) \leq \frac{3}{4}n \log n = \frac{3}{4}f(n).$$

Che Cosa Significa un Tentativo Fallito?

Ricorrenza: $T(n) = 2T(n/2) + n \log n$, quindi $\alpha = 1$ e $f(n) = n^\alpha \log n$.

- **Non** è il caso 1: $\frac{n^\alpha \log n}{n^{\alpha-\epsilon}} = n^\epsilon \log n \rightarrow \infty$.
- **Non** è il caso 2: $\frac{n^\alpha \log n}{n^\alpha} = \log n \rightarrow \infty$.
- **Non** è il caso 3: $\frac{n^\alpha \log n}{n^{\alpha+\epsilon}} = \frac{\log n}{n^\epsilon} \rightarrow 0$.

Il fattore $\log n$ crea una distanza più che costante, ma meno che polinomiale: resta nello spazio fra i tre casi.

2.9.4 Master Theorem con Fattori Logaritmici

Estensione: Fattori Logaritmici

Sia $T(n) = aT(n/b) + \Theta(n^\beta \log^k n)$, con $k \in \mathbb{R}$ e termine non ricorsivo positivo definitivamente. Posto $\alpha = \log_b a$:

$$T(n) = \begin{cases} \Theta(n^\alpha) & \beta < \alpha, \\ \Theta(n^\beta \log^k n) & \beta > \alpha, \\ \Theta(n^\alpha \log^{k+1} n) & \beta = \alpha, k > -1, \\ \Theta(n^\alpha \log \log n) & \beta = \alpha, k = -1, \\ \Theta(n^\alpha) & \beta = \alpha, k < -1. \end{cases}$$

Due esempi:

- $2T(n/2) + n \log n$: $\alpha = \beta = 1$ e $k = 1 > -1$, quindi $T(n) = \Theta(n \log^2 n)$.
- $2T(n/2) + n/\log n$: $\alpha = \beta = 1$ e $k = -1$, quindi $T(n) = \Theta(n \log \log n)$.

2.9.5 Oltre il Master Theorem

Sottoproblemi di Dimensioni Diverse

Il Master Theorem non si applica, per esempio, a

$$T(n) = T(n/3) + T(2n/3) + n.$$

Teorema di Akra–Bazzi

Tratta molte ricorrenze divide-et-impera con sottoproblemi sbilanciati. Il suo enunciato è però troppo tecnico per gli obiettivi del corso.

2.10 Ricorrenze: Altre Ricorrenze Comuni

Teorema (Ricorrenze Lineari di Ordine Costante)

Siano $a_1, \dots, a_h$ interi non negativi, non tutti nulli, $h \geq 1$ costante e $\beta \geq 0$. Se

$$T(n) = \begin{cases} \Theta(1) & n \leq m, \\ \sum_{i=1}^h a_i T(n - i) + \Theta(n^\beta) & n > m, \end{cases}$$

con $m \geq h$ costante e $a = \sum_{i=1}^h a_i$, allora:

- se $a = 1$, $T(n) = \Theta(n^{\beta+1})$;
- se $a \geq 2$, $T(n) = O(a^n)$.

Il secondo limite è generale e può non essere stretto: per Fibonacci dà $O(2^n)$, mentre il limite preciso è $\Theta(\varphi^n)$.

Esempi:

Ricorrenza	a	β	Risultato
$T(n) = T(n-10) + n^2$	1	2	$\Theta(n^3)$
$T(n) = T(n-1) + n$	1	1	$\Theta(n^2)$
$T(n) = 2T(n-1) + 1$	2	0	$O(2^n)$

Nota mia: la seconda riga è la stessa ricorrenza “trabocchetto” della Sezione 2.8 ($T(n) = T(n-1) + n$), qui inquadrata nel teorema generale invece che dimostrata per sostituzione – coerente con il risultato $\Theta(n^2)$ già trovato lì.

Per $2T(n-1)+1$, l'albero delle chiamate fornisce anche il limite inferiore corrispondente: $T(n) = \Theta(2^n)$.

Esercizio

Siano $T(n) = 7T(n/2) + n^2$ la funzione di costo di un algoritmo A , e $T'(n) = aT'(n/4) + n^2$ la funzione di costo di un algoritmo A' . Qual è il massimo valore intero di a che rende A' asintoticamente più veloce di A ?

Soluzione: $T(n) = \Theta(n^{\log_2 7}) = \Theta(n^{\log_4 49})$ (cambio di base: $\log_2 7 = \log_4 7^2 = \log_4 49$).
Costo di A' :

- $a < 16$: $T'(n) = \Theta(n^2)$;
- $a = 16$: $T'(n) = \Theta(n^2 \log n)$;
- $a > 16$: $T'(n) = \Theta(n^{\log_4 a})$.

Per essere strettamente più veloce serve $\log_4 a < \log_4 49$, cioè $a < 49$. Il massimo valore intero è quindi $\boxed{a = 48}$.

2.11 Back to Algorithms!: la Massima Sottosequenza Contigua

Il Problema (dedotto dal codice, non da una definizione esplicita in slide)

Dato un vettore A di n interi (positivi e negativi), trovare la massima somma ottenibile da una sua sottosequenza **contigua** $A[i \dots j]$ (la sottosequenza vuota, di somma 0, è ammessa come caso limite).

Si analizzano quattro versioni via via più efficienti, applicando le tecniche di questa sezione.

Versione 1: la Forza Bruta.

```
int maxsum1(int[] A, int n) {
    int maxSoFar = 0;
    for (int i=0; i < n; i++) {
        for (int j=i; j < n; j++) {
            int sum = 0;
            for (int k=i; k <= j; k++) {
                sum = sum + A[k];
            }
        }
    }
}
```

```

    }
    maxSoFar = max(maxSoFar, sum);
  }
}
return maxSoFar;
}

```

Si prova ogni coppia (i, j) e si ricalcola da zero la somma di $A[i \dots j]$. La complessità può essere approssimata contando il numero di esecuzioni della riga più interna:

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} (j - i + 1)$$

Limite superiore – vogliamo provare che $T(n) = O(n^3)$, cioè $\exists c_2 > 0, \exists m \geq 0 :$
 $T(n) \leq c_2 n^3, \forall n \geq m$:

$$\begin{aligned}
 T(n) &= \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} (j - i + 1) \\
 &\leq \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} n \leq \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n \\
 &= \sum_{i=0}^{n-1} n^2 = n^3 \stackrel{?}{\leq} c_2 n^3
 \end{aligned}$$

Questa disequazione è vera per $n \geq m = 0$ e $c_2 \geq 1$.

Limite inferiore – vogliamo provare che $T(n) = \Omega(n^3)$. Assumiamo per semplicità che n sia multiplo di 4:

$$\begin{aligned}
 T(n) &= \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} (j - i + 1) \\
 &\geq \sum_{i=0}^{n/4-1} \sum_{j=i+n/2}^{i+3n/4-1} (j - i + 1) \\
 &\geq \frac{n}{4} \cdot \frac{n}{4} \cdot \frac{n}{2} = \frac{n^3}{32} \stackrel{?}{\geq} c_1 n^3
 \end{aligned}$$

L'ultima disequazione è vera per $c_1 \leq 1/32$. Dunque $T(n) = \Theta(n^3)$.

Nota (lettura della dimostrazione)

Il limite inferiore restringe la doppia somma a $i \in [0, n/4)$ e, per ciascuno di questi i , a $j \in [i + n/2, i + 3n/4)$: sono $n/4$ valori di i , $n/4$ valori di j per ciascun i , e in questa regione $j - i + 1 > n/2$ sempre – da cui il prodotto $\frac{n}{4} \cdot \frac{n}{4} \cdot \frac{n}{2}$.

Versione 2: un Ciclo in Meno.

```

int maxsum2(int[] A, int n) {
    int maxSoFar = 0;
    for (int i=0; i < n; i++) {

```

```

    int sum = 0;
    for (int j=i; j < n; j++) {
        sum = sum + A[j];
        maxSoFar = max(maxSoFar, sum);
    }
}
return maxSoFar;
}

```

Si evita di ricalcolare la somma da zero ad ogni j : si aggiorna incrementalmente. La complessità (contando i passi nel ciclo più interno):

$$T(n) = \sum_{i=0}^{n-1} (n - i)$$

Prova che $T(n) = \Theta(n^2)$:

$$T(n) = \sum_{i=0}^{n-1} (n - i) = \sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$$

Questo non richiede ulteriori dimostrazioni.

Versione 3: Divide et Impera.

```

int maxsum_rec(int[] A, int i, int j) {
    if (i==j)
        return max(0, A[i]);
    int m = (i+j) / 2;
    int maxs = maxsum_rec(A, i, m);
    int maxd = maxsum_rec(A, m+1, j);
    int maxss = 0;
    int sum = 0;
    for (int k=m; k>=i; k--) {
        sum = sum+A[k];
        maxss = max(maxss, sum);
    }
    // Continua...
    int maxdd = 0;
    sum = 0;
    for (int k=m+1; k<=j; k++) {
        sum = sum+A[k];
        maxdd = max(maxdd, sum);
    }
    return max(max(maxs,maxd),
               maxss+maxdd);
}

int maxsum3(int[] A, int n) {
    return maxsum_rec(A,0,n-1);
}

```

Divide: si spezza $A[i \dots j]$ a metà. **Impera:** le due chiamate ricorsive `maxs` e `maxd` risolvono i sottoproblemi (la miglior somma tutta a sinistra, tutta a destra). **Combina:** il primo ciclo calcola la migliore somma che *termina* nella metà sinistra (`maxss`); il secondo la migliore somma che *inizia* nella metà destra (`maxdd`); il risultato finale confronta le due soluzioni ricorsive con quella che attraversa il punto medio (`maxss+maxdd`).

Analisi. Relazione di ricorrenza:

$$T(n) = 2T(n/2) + n$$

Per il Master Theorem, versione ridotta (Teorema 2.9.2): $a = 2$, $b = 2$, $\beta = 1$. Poiché $\alpha = \log_2 2 = 1 = \beta$, siamo nel secondo caso e quindi

$$T(n) = \Theta(n \log n).$$

Versione 4: Lineare.

```
int maxsum4(int A[], int n) {
    int maxSoFar = 0;
    int maxHere = 0;
    for (int i=0; i < n; i++) {
        maxHere = max(maxHere+A[i], 0);
        maxSoFar = max(maxSoFar, maxHere);
    }
    return maxSoFar;
}
```

È facile vedere che la complessità di questa versione è $\Theta(n)$.

Nota mia (non nelle slide): il nome dell'algoritmo

Questa versione lineare è nota in letteratura come **algoritmo di Kadane**. L'idea: `maxHere` è la miglior somma di una sottosequenza che termina esattamente in i ; se aggiungere $A[i]$ la rende negativa conviene ripartire da 0 (da qui il `max(..., 0)`), altrimenti conviene estenderla.

Riepilogo delle Quattro Versioni

Versione	Complessità
1 (forza bruta)	$\Theta(n^3)$
2 (somma incrementale)	$\Theta(n^2)$
3 (divide et impera)	$\Theta(n \log n)$
4 (Kadane)	$\Theta(n)$

Lo stesso problema, quattro complessità diverse: un caso di scuola per tutte le tecniche viste in questa sezione (limiti superiori/inferiori per doppia somma, Master Theorem, e infine una soluzione che non ha nemmeno bisogno di ricorrenze).

2.12 Strutture Dati Astratte

2.12.1 Definizioni

Definizioni di Base

- **Dato:** in un linguaggio di programmazione, un valore che una variabile può assumere.
- **Tipo di dato astratto:** un modello matematico, dato da una collezione di valori e un insieme di operazioni ammesse su questi valori.
- **Tipi di dato primitivi:** forniti direttamente dal linguaggio. Esempi: `int` (+, −, *, /, %), `boolean` (!, &&, ||).

“Specifica” e “implementazione” di un tipo di dato astratto:

- **Specifica:** “manuale d’uso”, nasconde i dettagli implementativi all’utente;
- **Implementazione:** realizzazione vera e propria.

Specifica	Implementazione
Numeri reali	IEEE-754
Pile	Pile basate su vettori / Pile basate su puntatori
Code	Code basate su vettori circolari / Code basate su puntatori

Definizione: Strutture di Dati

Le **strutture di dati** sono collezioni di dati, caratterizzate più dall’**organizzazione** della collezione piuttosto che dal tipo dei dati contenuti.

Come caratterizzare le strutture dati:

- un insieme di **operatori** che permettono di manipolare la struttura;
- un modo **sistematico** di organizzare l’insieme dei dati.

Alcune tipologie di strutture di dati:

- **Lineari / Non lineari** (presenza di una sequenza);
- **Statiche / Dinamiche** (variazione di dimensione, contenuto);
- **Omogenee / Disomogenee** (dati contenuti).

Panoramica per linguaggio:

Tipo	Java	C++	Python
Sequenze	List, Queue, Deque, LinkedList, ArrayList, Stack, ArrayDeque	list, forward_list, vector, stack, queue, deque	list, tuple
Insiemi	Set, TreeSet, HashSet, LinkedHashSet	set, unordered_set	set, frozenset
Dizionari	Map, HashTree, HashMap, LinkedHashMap	map, unordered_map	dict
Alberi	—	—	—
Grafi	—	—	—

(Alberi e grafi non hanno tipicamente un supporto diretto nella libreria standard: si costruiscono con classi proprie o su vettori/liste.)

2.12.2 Sequenza

Definizione: Sequenza

Una struttura dati **dinamica, lineare** che rappresenta una sequenza **ordinata** di valori, dove un valore può comparire più di una volta. L'ordine all'interno della sequenza è importante.

Operazioni ammesse:

- data la posizione, è possibile aggiungere/togliere elementi: data $s = s_1, s_2, \dots, s_n$, l'elemento s_i è in posizione pos_i ; esistono le posizioni (fittizie) $\text{pos}_0, \text{pos}_{n+1}$;
- è possibile accedere direttamente alla testa/coda;
- è possibile accedere sequenzialmente a tutti gli altri elementi.

Specifica – SEQUENCE:

```
% Restituisce true se la sequenza e' vuota
boolean isEmpty()
% Restituisce true se p e' uguale a pos_0 oppure a pos_(n+1)
boolean finished(POS p)
% Restituisce la posizione del primo elemento
POS head()
% Restituisce la posizione dell'ultimo elemento
POS tail()
% Restituisce la posizione dell'elemento che segue p
POS next(POS p)
% Restituisce la posizione dell'elemento che precede p
POS prev(POS p)
% Inserisce l'elemento v di tipo item nella posizione p.
% Restituisce la posizione del nuovo elemento, che diviene
% il predecessore di p
```

```

POS insert(POS p, item v)
% Rimuove l'elemento contenuto nella posizione p.
% Restituisce la posizione del successore di p,
% che diviene successore del predecessore di p
POS remove(POS p)
% Legge l'elemento di tipo item contenuto nella posizione p
item read(POS p)
% Scrive l'elemento v di tipo item nella posizione p
write(POS p, item v)

```

Esempi d'uso:

```

// Java
List<String> lista = new LinkedList<String>();
lista.add("two");
lista.addFirst("one");
lista.addLast("three");
// Result: ['one', 'two', 'three']

```

```

// C++
std::list<int> lista;
lista.push_front(2);
lista.push_front(1);
lista.push_back(3);
// Result: [1,2,3]

```

```

# Python
lista = ["one", "three"]
lista.insert(1, "two")
# Result: ['one', 'two', 'three']

```

2.12.3 Insiemi

Definizione: Insieme

Una struttura dati **dinamica**, **non lineare** che memorizza una collezione **non ordinata** di elementi **senza valori ripetuti**. L'ordinamento fra elementi è dato dall'eventuale relazione d'ordine definita sul tipo degli elementi stessi.

Operazioni ammesse:

Operazioni base:

inserimento
cancellazione
verifica contenimento

Operazioni di ordinamento:

massimo, minimo

Operazioni insiemistiche:

unione
intersezione
differenza

Iteratori:

foreach $x \in S$ do

Specifica – SET:

```

% Restituisce la cardinalita' dell'insieme

```

```

int size()
% Restituisce true se x e' contenuto nell'insieme
boolean contains(item x)
% Inserisce x nell'insieme, se non gia' presente
insert(item x)
% Rimuove x dall'insieme, se presente
remove(item x)
% Restituisce un nuovo insieme che e' l'unione di A e B
SET union(SET A, SET B)
% Restituisce un nuovo insieme che e' l'intersezione di A e B
SET intersection(SET A, SET B)
% Restituisce un nuovo insieme che e' la differenza di A e B
SET difference(SET A, SET B)

```

Esempi d'uso:

```

// Java
Set<String> docenti = new TreeSet<>();
docenti.add("Alberto");
docenti.add("Cristian");
docenti.add("Alessio");
// Result: {"Alberto", "Alessio", "Cristian"}

// C++
std::set<std::string> frutta;
frutta.insert("mele"); frutta.insert("pere");
frutta.insert("banane"); frutta.insert("mele");
frutta.remove("mele")
// Result: {"banane", "pere"}

# Python
items = {"rock", "paper", "scissors", "rock"}
print(items)           # {"rock", "paper", "scissors"}
print("Spock" in items) # False
print("lizard" not in items) # True

```

2.12.4 Dizionari

Definizione: Dizionario

Struttura dati che rappresenta il concetto matematico di **relazione univoca** $R : D \rightarrow C$, o associazione chiave-valore. L'insieme D è il **dominio** (elementi detti **chiavi**); l'insieme C è il **codominio** (elementi detti **valori**).

Operazioni ammesse:

- ottenere il valore associato ad una particolare chiave (se presente), o **nil** se assente;
- inserire una nuova associazione chiave-valore, cancellando eventuali associazioni precedenti per la stessa chiave;

- rimuovere un'associazione chiave-valore esistente.

Specifica – DICTIONARY:

```
% Restituisce il valore associato alla chiave k se presente,
% nil altrimenti
item lookup(item k)
% Associa il valore v alla chiave k
insert(item k, item v)
% Rimuove l'associazione della chiave k
remove(item k)
```

Esempi d'uso:

```
// Java
Map<String, String> capoluoghi = new HashMap<>();
capoluoghi.put("Toscana", "Firenze");
capoluoghi.put("Lombardia", "Milano");
capoluoghi.put("Sardegna", "Cagliari");
```

```
// C++
std::map<std::string, int> wordcounts;
std::string s;
while (std::cin >> s && s != "end")
    ++wordcounts[s];
```

```
# Python
v = {}
v[10] = 5
v["alberto"] = 42
v[10] + v["alberto"]
# Result: 47
```

2.12.5 Alberi e Grafi

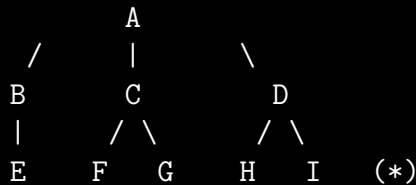
Definizione: Albero Ordinato

- Un albero ordinato è dato da un insieme finito di elementi detti **nodi**.
- Uno di questi nodi è designato come **radice**.
- I rimanenti nodi, se esistono, sono partizionati in insiemi **ordinati** e **disgiunti**, anch'essi alberi ordinati.

Nota grafica: le slide illustrano un albero radicato in A , con figli B, C, D ; B ha figlio E ; C ha figli F, G ; D ha figli H e un ultimo nodo.

Refuso nella Slide Originale

Nella slide 15/35, l'ultimo figlio di D è etichettato di nuovo " G " (già usato come figlio di C): impossibile in un albero, dove ogni nodo ha un solo genitore. *Ipotesi mia* (non nella slide): il nodo era quasi certamente destinato a chiamarsi I , per completare la sequenza alfabetica $A-I$ su 9 nodi. Uso I nello schema sotto, segnalando che è una correzione.



(*) corretto da " G " a " I " – vedi nota sopra.

Definizione: Grafo

La struttura dati **grafo** è composta da:

- un insieme di elementi detti **nodi** o **vertici**;
- un insieme di coppie (ordinate oppure no) di nodi detti **archi**.

Nota grafica: le slide mostrano un piccolo grafo **orientato** di esempio a quattro nodi A, B, C, D (disposti a quadrato) con cinque archi diretti fra loro, incluso un arco diagonale – puramente illustrativo della definizione, senza un significato specifico da memorizzare.

Operazioni: tutte le operazioni su alberi e grafi ruotano attorno alla possibilità di effettuare **visite** su di essi (specifica completa più avanti nel corso).

Commenti Conclusivi

I concetti di sequenza, insieme, dizionario sono collegati:

- insieme delle chiavi / insieme dei valori;
- scorrere la sequenza di tutte le chiavi.

Alcune realizzazioni sono “naturali”: sequenza $\leftrightarrow$ lista; albero astratto $\leftrightarrow$ albero basato su puntatori.

Esistono tuttavia realizzazioni alternative: insieme come vettore booleano; albero come vettore dei padri.

La scelta della struttura di dati ha riflessi sull’efficienza e sulle operazioni ammesse: dizionario come hash table $\Rightarrow$ lookup $O(1)$, ricerca minimo $O(n)$; dizionario come albero $\Rightarrow$ lookup $O(\log n)$, ricerca minimo $O(1)$.

Lista bidirezionale senza sentinella – inserimento (Java):

```
public Pos insert(Pos pos, Object v) {
    Pos t = new Pos(v);
    if (head == null) {
        head = tail = t;                // Insert in an empty list
    }
}
```

```

} else if (pos == null) {
    t.pred = tail;                // Insert at the end
    tail.succ = t;
    tail = t;
} else {
    t.pred = pos.pred;           // Insert in front of an existing position
    if (t.pred != null)
        t.pred.succ = t;
    else
        head = t;
    t.succ = pos;
    pos.pred = t;
}
return t;
}

```

2.12.6 Pila

Definizione: Pila (Stack)

Una struttura dati **dinamica**, **lineare** in cui l'elemento rimosso dall'operazione di cancellazione è predeterminato: "quello che per **meno** tempo è rimasto nell'insieme" (**LIFO** – *Last-in, First-out*).

Specifica – STACK:

```

% Restituisce true se la pila e' vuota
boolean isEmpty()
% Inserisce v in cima alla pila
push(item v)
% Estrae l'elemento in cima alla pila e lo restituisce al chiamante
item pop()
% Legge l'elemento in cima alla pila
item top()

```

Possibili utilizzi:

- nei linguaggi con procedure: gestione dei record di attivazione;
- nei linguaggi *stack-oriented*: le operazioni prendono gli operandi dallo stack e inseriscono il risultato nello stack (es. Postscript, Java bytecode).

Possibili implementazioni:

- tramite liste **bidirezionali**: puntatore all'elemento *top*;
- tramite **vettore**: dimensione limitata, overhead più basso.

Pila basata su vettore – pseudocodice:

```

STACK
item[] A          % Elementi
int size          % Cursore
int capacity      % Dim. massima

```

```

STACK Stack(int dim)
    STACK t = new STACK
    t.A = new int[0 ... dim-1]
    t.capacity = dim
    t.size = 0
    return t

```

```

item top()
    precondition: size > 0
    return A[size-1]

```

```

boolean isEmpty()
    return size == 0

```

```

item pop()
    precondition: size > 0
    size = size - 1
    return A[size]

```

```

push(item v)
    precondition: size < capacity
    A[size] = v
    size = size + 1

```

Pila basata su vettore – Java:

```

public class VectorStack implements Stack {

    /** Vector containing the elements */
    private Object[] A;

    /** Number of elements in the stack */
    private int n;

    public VectorStack(int dim) {
        n = 0;
        A = new Object[dim];
    }

    public boolean isEmpty() {
        return n==0;
    }

    public Object top() {

```

```

        if (n == 0)
            throw new IllegalStateException("Stack is empty");
        return A[n-1];
    }

    public Object pop() {
        if (n == 0)
            throw new IllegalStateException("Stack is empty");
        return A[--n];
    }

    public void push(Object o) {
        if (n == A.length)
            throw new IllegalStateException("Stack is full");
        A[n++] = o;
    }
}

```

2.12.7 Coda

Definizione: Coda (Queue)

Una struttura dati **dinamica**, **lineare** in cui l'elemento rimosso dall'operazione di cancellazione è predeterminato: "quello che per **più** tempo è rimasto nell'insieme" (**FIFO** – *First-in, First-out*).

Specifica – QUEUE:

```

% Restituisce true se la coda e' vuota
boolean isEmpty()
% Inserisce v in fondo alla coda
enqueue(item v)
% Estrae l'elemento in testa alla coda e lo restituisce al chiamante
item dequeue()
% Legge l'elemento in testa alla coda
item top()

```

Possibili utilizzi: nei sistemi operativi, i processi in attesa di utilizzare una risorsa vengono gestiti tramite una coda – la politica FIFO è *fair*.

Possibili implementazioni:

- tramite **liste monodirezionali**: puntatore *head* per l'estrazione, puntatore *tail* per l'inserimento;
- tramite **array circolari**: dimensione limitata, overhead più basso.

Coda Basata su Vettore Circolare. La circolarità può essere implementata con l'operazione modulo. Bisogna prestare attenzione ai problemi di **overflow** (buffer pieno).

Nota grafica – verificata sulla slide: quattro stati successivi dello stesso buffer, con i puntatori *head* (primo elemento) e *head+n* (prima cella libera, subito dopo l'ultimo elemento):

```

[ ][ ][3][6][54][...][...][43][ ][ ][ ]      head=2, head+n=8  (stato iniziale)
      ^head                ^head+n

[ ][ ][3][6][54][...][...][43][12][ ][ ]      dopo enqueue(12)
      ^head                ^head+n

[ ][ ][ ][6][54][...][...][43][12][ ][ ]      dopo dequeue() -> 3
      ^head                ^head+n

[33][ ][ ][6][54][...][...][43][12][15][17]    dopo enqueue(15,17,33)
  ^head+n      ^head

```

Nell'ultimo passo, esaurito lo spazio libero a destra, `enqueue(33)` “avvolge” il vettore e occupa la cella 0: è proprio questo *wrap-around*, implementato con l'aritmetica modulo, a rendere il vettore **circolare**.

Pseudocodice:

```

QUEUE
item[] A          % Elementi
int size          % Dim. attuale
int head          % Testa
int capacity      % Dim. massima

QUEUE Queue(int dim)
    QUEUE t = new QUEUE
    t.A = new int[0 ... dim-1]
    t.capacity = dim
    t.head = 0
    t.size = 0
    return t

item top()
    precondition: size > 0
    return A[head]

boolean isEmpty()
    return size == 0

item dequeue()
    precondition: size > 0
    item t = A[head]
    head = (head + 1) mod capacity
    size = size - 1
    return t

enqueue(item v)
    precondition: size < capacity
    A[(head + size) mod capacity] = v
    size = size + 1

```

Java:

```
public class VectorQueue implements Queue {

    /** Element vector */
    private Object[] A;

    /** Current number of elements in the queue */
    private int n;

    /** Top element of the queue */
    private int head;

    public VectorQueue(int dim) {
        n = 0;
        head = 0;
        A = new Object[dim];
    }

    public boolean isEmpty() {
        return n==0;
    }

    public Object top() {
        if (n == 0)
            throw new IllegalStateException("Queue is empty");
        return A[head];
    }

    public Object dequeue() {
        if (n == 0)
            throw new IllegalStateException("Queue is empty");
        Object t = A[head];
        head = (head+1) % A.length;
        n = n-1;
        return t;
    }

    public void enqueue(Object v) {
        if (n == A.length)
            throw new IllegalStateException("Queue is full");
        A[(head+n) % A.length] = v;
        n = n+1;
    }
}
```

2.13 Alberi: Introduzione

2.13.1 Esempi

Prima di ogni definizione formale, alcuni esempi di strutture ad albero che si incontrano quotidianamente:

- **Tassonomia biologica:** una gerarchia $Kingdom \rightarrow Phylum \rightarrow Class \rightarrow Order \rightarrow Family \rightarrow Genus \rightarrow Species$, ad es. $Animalia \rightarrow Chordata \rightarrow Mammalia \rightarrow Primati \rightarrow Hominidae \rightarrow Homo \rightarrow Sapiens$ (Uomo), con altri rami paralleli per $Felidae \rightarrow Felis \rightarrow Leo/Domestica$ (Leone, Gatto) e $Muscidae \rightarrow Musca \rightarrow Domestica$ (Mosca).
- **Filesystem UNIX:** la radice `/` con sottodirectory `dev/`, `etc/`, `sbin/`, `tmp/`, `Users/`, `usr/`, `var/`, ciascuna a sua volta contenente altre sottodirectory (es. `etc/` contiene `cups/`, `httpd/`, `init.d/`, `postfix/`).
- **Documento HTML / DOM:** un documento

```
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html"/>
    <title>simple</title>
  </head>
  <body>
    <h1>A simple web page</h1>
    <ul>
      <li>List item one</li>
      <li>List item two</li>
    </ul>
    <h2>
      <a href="http://www.google.com">Google</a>
    </h2>
  </body>
</html>
```

corrisponde all'albero $html \rightarrow \{head \rightarrow \{meta, title\}, body \rightarrow \{h1, ul \rightarrow \{li, li\}, h2 \rightarrow a\}\}$.

2.13.2 Definizioni

Albero Radicato – Definizione 1

Un albero consiste di un insieme di **nodi** e un insieme di **archi orientati** che connettono coppie di nodi, con le seguenti proprietà:

- un nodo dell'albero è designato come **nodo radice**;
- ogni nodo n , a parte la radice, ha esattamente un arco entrante;
- esiste un cammino unico dalla radice ad ogni nodo;
- l'albero è connesso.

Albero Radicato – Definizione 2 (Ricorsiva)

Un albero è dato da:

- un insieme vuoto, **oppure**
- un nodo radice e zero o più **sottoalberi**, ognuno dei quali è un albero; la radice è connessa alla radice di ogni sottoalbero con un arco orientato.

Nota grafica: le slide raffigurano questa definizione come un nodo *Root* con k sottoalberi pendenti $T_1, T_2, \dots, T_k$.

Terminologia (verificata sulla slide, albero di riferimento a 13 nodi $A, \dots, M$):

- A è la **radice**; B, C sono radici dei **sottoalberi** di A ;
- D, E sono **figli** di B ; B è il **padre** di D, E ; D, E sono **fratelli**;
- i nodi senza figli (nell'esempio: H, I, J, K, G, L, M) sono **foglie**; gli altri nodi sono **nodi interni**.

Profondità, Livello, Altezza

- **Profondità** (*depth*) di un nodo: la lunghezza del cammino semplice dalla radice al nodo (misurata in numero di archi).
- **Livello** (*level*): l'insieme di nodi alla stessa profondità.
- **Altezza** (*height*) dell'albero: la profondità massima delle sue foglie.

2.14 Alberi Binari

2.14.1 Introduzione

Definizione: Albero Binario

Un **albero binario** è un albero radicato in cui ogni nodo ha al massimo due figli, identificati come **figlio sinistro** e **figlio destro**.

Nota

Due alberi T e U che hanno gli stessi nodi, gli stessi figli per ogni nodo e la stessa radice, sono **distinti** qualora un nodo u sia designato come figlio sinistro di v in T e come figlio destro di v in U .

Nota grafica – verificata sulla slide: tre alberi T_1, T_2, T_3 con la stessa struttura *insiemistica* $A(B(C, D))$ con E pendente da un quarto nodo – ma con E designato rispettivamente come figlio sinistro di C (T_1), figlio destro di C (T_2), figlio sinistro di D (T_3): tre alberi *binari* distinti secondo la nota sopra, pur avendo lo stesso “disegno” se si ignora sinistra/destra.

Specifica – TREE (albero binario):

```
% Costruisce un nuovo nodo, contenente v, senza figli o genitori
Tree(item v)
% Legge il valore memorizzato nel nodo
```

```

item read()
% Modifica il valore memorizzato nel nodo
write(item v)
% Restituisce il padre, oppure nil se questo nodo e' radice
TREE parent()
% Restituisce il figlio sinistro (destro) di questo nodo;
% restituisce nil se assente
TREE left()
TREE right()
% Inserisce il sottoalbero radicato in t come figlio
% sinistro (destro) di questo nodo
insertLeft(TREE t)
insertRight(TREE t)
% Distrugge (ricorsivamente) il figlio sinistro (destro)
% di questo nodo
deleteLeft()
deleteRight()

```

2.14.2 Implementazione

Campi memorizzati nei nodi: `parent` (reference al nodo padre), `left` (reference al figlio sinistro), `right` (reference al figlio destro).

Nota grafica: le slide illustrano un albero di puntatori: ogni nodo è un box con tre campi `parent/left/right`, e i box sono collegati fra loro esattamente secondo la struttura dell'albero che rappresentano – la realizzazione più diretta della Definizione 2.14.1.

TREE

```

Tree(item v)
    TREE t = new TREE
    t.parent = nil
    t.left = t.right = nil
    t.value = v
    return t

```

```

insertLeft(TREE T)
    if left == nil then
        T.parent = this
        left = T

```

```

insertRight(TREE T)
    if right == nil then
        T.parent = this
        right = T

```

```

deleteLeft()
    if left != nil then
        left.deleteLeft()
        left.deleteRight()

```

```

        delete left
        left = nil

deleteRight()
    if right != nil then
        right.deleteLeft()
        right.deleteRight()
        delete right
        right = nil

```

(simmetrico a *insertLeft/insertRight*: distrugge ricorsivamente l'intero sottoalbero sinistro/destro prima di cancellare la radice del sottoalbero stesso.)

2.14.3 Visite

Definizione: Visita di un Albero

Una **visita** (o **ricerca**) è una strategia per analizzare tutti i nodi di un albero.

Visita in profondità <i>Depth-First Search</i> (DFS)	Visita in ampiezza <i>Breadth-First Search</i> (BFS)
Per visitare un albero, si visita ricorsivamente ognuno dei suoi sottoalberi. Tre varianti: pre/in/post visita (<i>pre/in/post order</i>). Richiede uno stack .	Ogni livello dell'albero viene visitato, uno dopo l'altro. Si parte dalla radice. Richiede una queue .

Depth-First Search.

```

dfs(TREE t)
if t != nil then
    % pre-order visit of t
    print t
    dfs(t.left())
    % in-order visit of t
    print t
    dfs(t.right())
    % post-order visit of t
    print t

```

Le **tre varianti** sono lo **stesso codice**: cambia solo quale delle tre chiamate a **print** si considera “quella buona”. Applicate all'albero $A(B(C, D), E(F, G))$ (verificato: A ha figli B, E ; B ha figli C, D ; E ha figli F, G):

Variante	Sequenza prodotta
Pre-order (radice, sinistra, destra)	$A B C D E F G$
In-order (sinistra, radice, destra)	$C B D A F E G$
Post-order (sinistra, destra, radice)	$C D B F G E A$

Esempi di Applicazione. Contare i nodi – post-visita:

```
int count(TREE T)
if T == nil then
    return 0
else
    Cl = count(T.left())
    Cr = count(T.right())
    return Cl + Cr + 1
```

Il conteggio dei due sottoalberi (C_l , C_r) va calcolato **prima** di poterlo sommare: è intrinsecamente una post-visita (si “usa” il nodo, cioè lo si conta, solo dopo aver visitato entrambi i sottoalberi).

Stampare espressioni aritmetiche – in-visita:

```
int printExp(TREE T)
if T.left() == nil and T.right() == nil then
    print T.read()
else
    print "("
    printExp(T.left())
    print T.read()
    printExp(T.right())
    print ")"
```

Su un albero d’espressione con radice $*$, figlio sinistro $+(4, 9)$ e figlio destro $+(3, 4)$, l’in-visita produce esattamente la notazione infissa con parentesi: $((4+9) * (3+4))$ – l’operatore va stampato *fra* i due operandi, da cui l’in-visita.

Costo Computazionale

Il costo di una visita di un albero contenente n nodi è $\Theta(n)$, in quanto ogni nodo viene visitato al massimo una volta.

2.15 Alberi Generici

Specifica – TREE (albero generico):

```
% Costruisce un nuovo nodo, contenente v, senza figli o genitori
Tree(item v)
% Legge il valore memorizzato nel nodo
item read()
% Modifica il valore memorizzato nel nodo
write(item v)
% Restituisce il padre, oppure nil se questo nodo e' radice
TREE parent()
% Restituisce il primo figlio, oppure nil se questo nodo e' una foglia
TREE leftmostChild()
% Restituisce il prossimo fratello, oppure nil se assente
TREE rightSibling()
% Inserisce il sottoalbero t come primo figlio di questo nodo
```

```

insertChild(TREE t)
% Inserisce il sottoalbero t come prossimo fratello di questo nodo
insertSibling(TREE t)
% Distruggi l'albero radicato identificato dal primo figlio
deleteChild()
% Distruggi l'albero radicato identificato dal prossimo fratello
deleteSibling()

```

Esempio: Node (Java 8, DOM)

La stessa idea – padre, primo figlio, fratello successivo – è alla base dell'interfaccia `org.w3c.dom.Node` usata per rappresentare il DOM di un documento HTML/XML (Sezione 2.13, esempio del documento HTML):

```

package org.w3c.dom;
public interface Node {

    /** The parent of this node. */
    public Node getParentNode();

    /** The first child of this node. */
    public Node getFirstChild()

    /** The node immediately following this node. */
    public Node getNextSibling()

    /** Inserts the node newChild before the existing child node refChild
    public Node insertBefore(Node newChild, Node refChild)

    /** Adds the node newChild to the end of the list of children of th
    public Node appendChild(Node newChild)

    /** Removes the child node indicated by oldChild from the list of c
    public Node removeChild(Node oldChild)

    [...]
}

```

2.15.1 Visite

Depth-First Search.

```

dfs(TREE t)
if t != nil then
    % pre-order visit of node t
    print t
    TREE u = t.leftmostChild()
    while u != nil do
        dfs(u)
        u = u.rightSibling()

```

```
% post-order visit of node t
print t
```

Nota mia (non nella slide): perché manca l'in-order?

A differenza del DFS per alberi binari (Sezione 2.14), qui compaiono solo pre-order e post-order. Con un numero arbitrario di figli non esiste una posizione “di mezzo” univoca fra un figlio e l'altro dove inserire la visita del nodo stesso: l'in-order è un concetto specifico degli alberi *binari*, dove “fra sinistra e destra” è ben definito.

Breadth-First Search.

```
bfs(TREE t)

QUEUE Q = Queue()
Q.enqueue(t)
while not Q.isEmpty() do
    TREE u = Q.dequeue()
    % visita per livelli nodo u
    print u
    u = u.leftmostChild()
    while u != nil do
        Q.enqueue(u)
        u = u.rightSibling()
```

Applicata all'albero-esempio $A(B(C, D), E(F, G))$ già usato nella Sezione 2.14: **Sequence:** $A\ B\ E\ C\ D\ F\ G$ – si visita un livello alla volta, da sinistra a destra.

2.15.2 Implementazione

Esistono diversi modi per memorizzare un albero, più o meno indicati a seconda del numero massimo e medio di figli presenti: **vettore dei figli**, **primo figlio/prossimo fratello**, **vettore dei padri**.

Realizzazione con Vettore dei Figli. Campi memorizzati nei nodi: **parent** (reference al nodo padre); un **vettore dei figli**, che a seconda del numero di figli può comportare una discreta quantità di spazio sprecato.

Realizzazione Primo Figlio, Prossimo Fratello. Implementato come una **lista di fratelli**: ogni nodo ha tre riferimenti (**parent**, **child**, **sibling**) invece di un vettore di dimensione variabile.

TREE

TREE parent	% Reference al padre
TREE child	% Reference al primo figlio
TREE sibling	% Reference al prossimo fratello
item value	% Valore memorizzato nel nodo

```

Tree(item v)                                % Crea un nuovo nodo
    TREE t = new TREE
    t.value = v
    t.parent = t.child = t.sibling = nil
    return t

insertChild(TREE t)
    t.parent = self
    t.sibling = child                        % Inserisce t prima dell'attuale primo figlio
    child = t

insertSibling(TREE t)
    t.parent = parent
    t.sibling = sibling                      % Inserisce t prima dell'attuale prossimo fratello
    sibling = t

deleteChild()
    TREE newChild = child.rightSibling()
    delete(child)
    child = newChild

deleteSibling()
    TREE newSibling = sibling.rightSibling()
    delete(sibling)
    sibling = newSibling

delete(TREE t)
    TREE u = t.leftmostChild()
    while u != nil do
        TREE next = u.rightSibling()
        delete(u)
        u = next
    delete t

```

Realizzazione con Vettore dei Padri. L'albero è rappresentato da un vettore i cui elementi contengono il valore associato al nodo e l'indice della posizione del padre nel vettore.

Indice	Valore	Indice del padre
0	A	-1
1	B	0
2	E	0
3	C	1
4	D	1
5	F	2
6	G	2

Stesso albero $A(B(C, D), E(F, G))$ usato in tutta questa sezione, qui rappresentato in modo compatto: nessun puntatore esplicito, solo un indice intero per padre.

2.16 Insiemi e Dizionari: Riassunto Finale

2.16.1 Introduzione

Insiemi	Dizionari
Collezione di oggetti	Associazioni chiave-valore

Implementazione: molte delle strutture dati viste finora si prestano a realizzare sia insiemi sia dizionari, ciascuna con vantaggi e svantaggi.

Nota mia (non nella slide): perché non si parla mai di “dizionario” nel seguito?

Le tecniche presentate in questa sezione riguardano sempre e solo la realizzazione di un **insieme**. Un dizionario è, concettualmente, un insieme di coppie chiave-valore anziché di sole chiavi: ogni tecnica vista per gli insiemi (vettore booleano escluso, che richiede chiavi intere) si applica direttamente memorizzando coppie invece di chiavi nude. Per questo un riassunto delle tecniche per insiemi vale, *mutatis mutandis*, anche per i dizionari.

2.16.2 Insiemi Realizzati con Vettori Booleani

Insieme	Rappresentazione
Interi $0 \dots m - 1$; collezione di m oggetti memorizzati in un vettore	Vettore booleano di m elementi
Vantaggi	Svantaggi
Notevolmente semplice; efficiente verificare se un elemento appartiene all'insieme	Memoria occupata $O(m)$, indipendente dalle dimensioni effettive; alcune operazioni inefficienti – $O(m)$

SET (vettore booleano) – campi e operazioni base:

```
SET
boolean[] V
int size
int dim

SET Set(int m)
    SET t = new SET
    t.size = 0
    t.dim = m
    t.V = new boolean[0 ... m-1] = {false}
    return t

boolean contains(int x)
    if 0 <= x <= dim-1 then
        return V[x]
    else
```

```

        return false

int size()
    return size

insert(int x)
    if 0 <= x <= dim-1 then
        if not V[x] then
            size = size + 1
            V[x] = true

remove(int x)
    if 0 <= x <= dim-1 then
        if V[x] then
            size = size - 1
            V[x] = false

```

Operazioni insiemistiche:

```

SET union(SET A, SET B)
    int newsize = max(A.dim, B.dim)
    SET C = Set(newsize)
    for i = 0 to A.dim-1 do
        if A.contains(i) then
            C.insert(i)
    for i = 0 to B.dim-1 do
        if B.contains(i) then
            C.insert(i)
    return C

SET difference(SET A, SET B)
    SET C = Set(A.dim)
    for i = 0 to A.dim-1 do
        if A.contains(i) and not B.contains(i) then
            C.insert(i)
    return C

SET intersection(SET A, SET B)
    int newsize = min(A.dim, B.dim)
    SET C = Set(newsize)
    for i = 0 to min(A.dim, B.dim)-1 do
        if A.contains(i) and B.contains(i) then
            C.insert(i)
    return C

```

2.16.3 BitSet

Java – class `java.util.BitSet`

Metodo	Operazione
<code>void and(BitSet set)</code>	Intersection
<code>void or(BitSet set)</code>	Union
<code>int cardinality()</code>	Set size
<code>void clear(int i)</code>	Remove
<code>void set(int i)</code>	Insert
<code>boolean get(int i)</code>	Contains

C++ STL:

- `std::bitset` – struttura dati bitset con dimensione fissata nel template al momento della compilazione;
- `std::vector<bool>` – specializzazione di `std::vector` per ottimizzare la memorizzazione, dimensione dinamica.

2.16.4 Insiemi Realizzati con Liste/Vettori Non Ordinati

Costo Operazioni

- Operazioni di ricerca, inserimento e cancellazione: $O(n)$;
- operazioni di inserimento (assumendo assenza): $O(1)$;
- operazioni di unione, intersezione e differenza: $O(nm)$.

Stessa operazione della Sezione precedente, qui su rappresentazione a lista/vettore invece che a vettore booleano:

```
SET difference(SET A, SET B)
  SET C = Set()
  foreach s in A do
    if not B.contains(s) then
      C.insert(s)
  return C
```

Ogni chiamata a `B.contains(s)` costa $O(m)$ (scansione lineare di B), ripetuta n volte: da qui il costo $O(nm)$.

2.16.5 Insiemi Realizzati con Liste/Vettori Ordinati

```
LIST intersection(LIST A, LIST B)
  LIST C = Set()
  POS pos_a = A.head()
  POS pos_b = B.head()
  while not A.finished(pos_a) and not B.finished(pos_b) do
    if A.read(pos_a) == B.read(pos_b) then
```

```

        C.insert(C.tail(), A.read(pos_a))
        pos_a = A.next(pos_a)
        pos_b = B.next(pos_b)
    else if A.read(pos_a) < B.read(pos_b) then
        pos_a = A.next(pos_a)
    else
        pos_b = B.next(pos_b)
return C

```

Il classico algoritmo a due puntatori sul *merge* di sequenze ordinate: si avanza **entrambi** i puntatori (copiando l'elemento in *C*) solo quando i valori coincidono; altrimenti si avanza solo il puntatore che punta al valore **più piccolo**.

Costo Operazioni

- Ricerca: $O(n)$ su liste, $O(\log n)$ su vettori (ricerca binaria);
- inserimento e cancellazione: $O(n)$;
- unione, intersezione e differenza: $O(n)$.

2.16.6 Insiemi – Strutture Dati Complesse

Alberi bilanciati

Ricerca, inserimento, cancellazione: $O(\log n)$
 Iterazione: $O(n)$
 Con ordinamento
 Implementazioni: Java `TreeSet`,
 Python `OrderedSet`, C++ STL
`set`

Hash table

Ricerca, inserimento, cancellazione: $O(1)$
 Iterazione: $O(m)$
 Senza ordinamento
 Implementazioni: Java `HashSet`,
 Python `set`, C++ STL
`unordered_set`

(Dettagli implementativi di entrambe le famiglie: vedi le sezioni dedicate agli Alberi Binari di Ricerca e all'Hashing.)

2.16.7 Insiemi – Riassunto

Struttura	contains/lookup	insert	remove	min	foreach	Ordine
Vettore booleano	$O(1)$	$O(1)$	$O(1)$	$O(m)$	$O(m)$	Sì
Lista non ordinata	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	No
Lista ordinata	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	Sì
Vettore ordinato	$O(\log n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	Sì
Alberi bilanciati	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Sì
Hash (mem. interna)	$O(1)$	$O(1)$	$O(1)$	$O(m)$	$O(m)$	No
Hash (mem. esterna)	$O(1)$	$O(1)$	$O(1)$	$O(m + n)$	$O(m + n)$	No

$m \equiv$ dimensione del vettore o della tabella hash.

Il Compromesso Centrale

Nessuna struttura vince su tutti i fronti: le **tabelle hash** danno operazioni base $O(1)$ ma **rinunciano all'ordinamento**; gli **alberi bilanciati** garantiscono $O(\log n)$ su *tutto*, **min** incluso, **mantenendo** l'ordinamento. La scelta dipende da quali operazioni servono davvero.

2.16.8 Complessità in Python

List.

Sintassi	Operazione	Caso medio	Caso pessimo ammortizzato
<code>L.copy()</code>	Copy	$O(n)$	$O(n)$
<code>L.append(x)</code>	Append	$O(1)$	$O(1)$
<code>L.insert(i,x)</code>	Insert	$O(n)$	$O(n)$
<code>L.remove(x)</code>	Remove	$O(n)$	$O(n)$
<code>L[i]</code>	Index	$O(1)$	$O(1)$
<code>for x in L</code>	Iterator	$O(n)$	$O(n)$
<code>L[i:i+k]</code>	Slicing	$O(k)$	$O(k)$
<code>L.extend(s)</code>	Extend	$O(k)$	$O(k)$
<code>x in L</code>	Contains	$O(n)$	$O(n)$
<code>min(L), max(L)</code>	Min, Max	$O(n)$	$O(n)$
<code>len(L)</code>	Get length	$O(1)$	$O(1)$

$n = \text{len}(L)$.

Set.

Sintassi	Operazione	Caso medio	Caso pessimo
<code>x in S</code>	Contains	$O(1)$	$O(n)$
<code>for x in S</code>	Iterator	$O(n)$	$O(n)$
<code>S.add(x)</code>	Insert	$O(1)$	$O(n)$
<code>S.remove(x)</code>	Remove	$O(1)$	$O(n)$
<code>S T</code>	Union	$O(n+m)$	$O(n \cdot m)$
<code>S&T</code>	Intersection	$O(\min(n, m))$	$O(n \cdot m)$
<code>S-T</code>	Difference	$O(n)$	$O(n \cdot m)$

$n = \text{len}(S)$, $m = \text{len}(T)$.

Nota mia (non nella slide): perché “Caso pessimo” e non “ammortizzato” per Set?

Nella tabella **List** le due colonne coincidono sempre: un vettore Python ha costi prevedibili, senza sorprese da caso a caso. Nella tabella **Set**, invece, caso medio e caso pessimo **divergono** ($O(1)$ contro $O(n)$ per **contains/add/remove**): è la firma tipica di una struttura basata su **hashing** (Sezione dedicata all'Hashing), dove il caso pessimo dipende da quante collisioni si accumulano su una stessa cella – un evento raro in pratica, ma non escluso dall'analisi asintotica.

2.17 Alberi Binari di Ricerca

2.17.1 Introduzione

Dizionario (richiamo)

Un dizionario è un insieme dinamico che implementa le seguenti funzionalità: `item lookup(item k)`, `insert(item k, item v)`, `remove(item k)` (Definizione 2.12.4).

Possibili implementazioni:

Struttura dati	lookup	insert	remove
Vettore ordinato	$O(\log n)$	$O(n)$	$O(n)$
Vettore non ordinato	$O(n)$	$O(1)^*$	$O(1)^*$
Lista non ordinata	$O(n)$	$O(1)^*$	$O(1)^*$

* Assumendo che l'elemento sia già stato trovato, altrimenti $O(n)$.

Idea Ispiratrice

Portare l'idea di **ricerca binaria** negli alberi.

Memorizzazione: le associazioni chiave-valore vengono memorizzate in un albero binario; ogni nodo u contiene una coppia $(u.key, u.value)$; le chiavi devono appartenere a un insieme **totalmente ordinato**.

```
TREE
TREE parent
TREE left
TREE right
item key
item value
```

Proprietà dell'ABR

1. Le chiavi contenute nei nodi del sottoalbero **sinistro** di u sono **minori** di $u.key$.
2. Le chiavi contenute nei nodi del sottoalbero **destro** di u sono **maggiori** di $u.key$.

Queste due proprietà permettono di realizzare un algoritmo di ricerca dicotomica.

Albero di riferimento, verificato e riusato in tutti gli esempi seguenti: radice 10, figli 6, 15; 6 ha figli 4, 8; 15 ha figli 12, 18 – cioè $10(6(4, 8), 15(12, 18))$.

Specifica:

```
% Getters
item key()
item value()
TREE left()
TREE right()
```

```

TREE parent()

% Dizionario
item lookup(item k)
insert(item k, item v)
remove(item k)

% Ordinamento
TREE successorNode(TREE t)
TREE predecessorNode(TREE t)
TREE min()
TREE max()

```

Funzioni interne – l'oggetto `DICTIONARY` espone `lookup/insert/remove` delegando a tre funzioni ricorsive che operano direttamente sui nodi:

```

DICTIONARY
TREE tree

Dictionary()
    tree = nil

TREE lookupNode(TREE T, item k)
TREE insertNode(TREE T, item k, item v)
TREE removeNode(TREE T, item k)

```

2.17.2 Ricerca

`lookupNode()`

Restituisce il nodo dell'albero T che contiene la chiave k , se presente; restituisce `nil` se non presente.

Implementazione dizionario:

```

item lookup(item k)
    TREE t = lookupNode(tree, k)
    if t != nil then
        return t.value()
    else
        return nil

```

Esempio (albero di riferimento $10(6(4, 8), 15(12, 18))$, ricerca di 3): $u = 6$; $3 < 6 \Rightarrow u = 2$ (sinistra); $3 > 2 \Rightarrow u = 4$ (destra); $3 < 4 \Rightarrow u = 3$ (sinistra); $3 = 3$: trovato. (Nota: questo esempio usa un secondo albero di riferimento più piccolo, $6(2(1, 4(3, _)), 8)$, riutilizzato nella sezione *successore-predecessore*.)

Implementazione iterativa:

```

TREE lookupNode(TREE T, item k)
TREE u = T
while u != nil and u.key != k do

```

```

    if k < u.key then
        u = u.left           % Sotto-albero di sinistra
    else
        u = u.right          % Sotto-albero di destra
return u

```

Esempio (stesso albero, ricerca di 5, non presente): $u = 6$; $5 < 6 \Rightarrow u = 2$; $5 > 2 \Rightarrow u = 4$; $5 > 4 \Rightarrow u = \text{nil}$ (destra); return nil (non trovato).

Implementazione ricorsiva:

```

TREE lookupNode(TREE T, item k)
if T == nil or T.key == k then
    return T
else
    return lookupNode(iif(k < T.key, T.left, T.right), k)

```

2.17.3 Minimo-Massimo

Esempio (albero $6(2(1, 4(3, _)), 8(_, 12(9, 15)))$): min del sottoalbero radicato in 6 è 1; max del sottoalbero radicato in 6 è 15; max del sottoalbero radicato in 2 è 4; min del sottoalbero radicato in 8 è 8 (nessun figlio sinistro).

```

TREE min(TREE T)
    TREE u = T
    while u.left != nil do
        u = u.left
    return u

```

```

TREE max(TREE T)
    TREE u = T
    while u.right != nil do
        u = u.right
    return u

```

2.17.4 Successore-Predecessore

Definizione: Successore

Il **successore** di un nodo u è il più piccolo nodo maggiore di u .

Tre esempi (stesso albero $6(2(1, 4(3, _)), 8(_, 12(9, 15)))$): successore di 12 è 15; successore di 2 è 3; successore di 9 è 12; successore di 4 è 6.

Caso 1: u ha Figlio Destro

Il successore v è il **minimo del sottoalbero destro** di u .

Caso 2: u non ha Figlio Destro

Risalendo attraverso i padri, il successore è il primo avo v tale per cui u sta nel sottoalbero **sinistro** di v .

Implementazione – verificata visivamente: `return p` è fuori dal ciclo `while`, eseguito una sola volta al termine della risalita:

```
TREE successorNode(TREE t)
if t == nil then
    return t
if t.right != nil then          % Caso 1
    return min(t.right)
else                            % Caso 2
    TREE p = t.parent
    while p != nil and t == p.right do
        t = p
        p = p.parent
    return p
```

```
TREE predecessorNode(TREE t)
if t == nil then
    return t
if t.left != nil then           % Caso 1
    return max(t.left)
else                            % Caso 2
    TREE p = t.parent
    while p != nil and t == p.left do
        t = p
        p = p.parent
    return p
```

Per passare da successore a predecessore: `right` diventa `left`, `min` diventa `max` – i due algoritmi sono l'uno lo speculare dell'altro.

2.17.5 Inserimento

`insertNode()`

Inserisce un'associazione chiave-valore (k, v) nell'albero T . Se la chiave è già presente, sostituisce il valore associato; altrimenti viene inserita una nuova associazione. Se $T = \text{nil}$, restituisce il primo nodo dell'albero; altrimenti restituisce T inalterato.

Implementazione dizionario:

```
insert(item k, item v)
    tree = insertNode(tree, k, v)
```

Esempio (inserimento di 5 nell'albero $6(2(1, 4(3, _)), 8(_, 12(9, 15))))$: $u = 6$; $5 < 6 \Rightarrow u = 2$ (sinistra); $5 > 2 \Rightarrow u = 4$ (destra); $5 > 4 \Rightarrow u = \text{nil}$ (destra); inserito come figlio destro di 4.

```
TREE insertNode(TREE T, item k, item v)
TREE p = nil                    % Padre
TREE u = T
while u != nil and u.key != k do % Cerca posizione inserimento
```

```

    p = u
    u = iif(k < u.key, u.left, u.right)
if u != nil and u.key == k then
    u.value = v                                % Chiave gia' presente
else
    TREE new = Tree(k, v)                      % Crea un nodo coppia chiave-valore
    link(p, new, k)
    if p == nil then
        T = new                                % Primo nodo ad essere inserito
return T                                       % Restituisce albero non modificato o nuovo

link(TREE p, TREE u, item k)
if u != nil then
    u.parent = p                                % Registrazione padre
if p != nil then
    if k < p.key then p.left = u                % Attaccato come figlio sinistro
    else p.right = u                           % Attaccato come figlio destro

```

2.17.6 Cancellazione

removeNode()

Rimuove il nodo contenente la chiave k dall'albero T ; restituisce la radice dell'albero (potenzialmente cambiata).

```

remove(item k)
    tree = removeNode(tree, k)

```

Sull'albero di riferimento a 9 nodi $7(2(1, 5(3(_, 4), 6)), 8(_, 12(9, 15)))$ (verificato visivamente):

Caso 1: u Senza Figli

Si elimina semplicemente. *Esempio*: eliminazione di 5 (foglia).

Caso 2: u ha un Solo Figlio f

Si elimina u e si attacca f all'ex-padre p di u , in sostituzione di u (*short-cut*).
Esempio: eliminazione di 4.

Caso 3: u ha due Figli

Esempio: eliminazione di 2 (radice del sottoalbero sinistro), in quattro passi:

1. si individua il successore s di u (qui: 3 – il successore non ha mai figlio sinistro, per costruzione);
2. si “stacca” il successore s dalla sua posizione originale;
3. si attacca l’eventuale **figlio destro** di s (qui: 4) al padre di s , in sostituzione di s (*short-cut*, esattamente come nel Caso 2);
4. si **copia** s su u (la coppia chiave-valore di s sovrascrive quella di u) e si rimuove il nodo s ormai duplicato.

Implementazione (le tre parti che seguono sono lo stesso codice, mostrato progressivamente ramo per ramo nelle slide):

```
TREE removeNode(TREE T, item k)
TREE t
TREE u = lookupNode(T, k)
if u != nil then
    if u.left == nil and u.right == nil then          % Caso 1
        link(u.parent, nil, k)
        delete u
    else if u.left != nil and u.right != nil then      % Caso 3
        TREE s = successorNode(u)
        link(s.parent, s.right, s.key)
        u.key = s.key
        u.value = s.value
        delete s
    else if u.left != nil and u.right == nil then      % Caso 2 (figlio sinistro)
        link(u.parent, u.left, k)
        if u.parent == nil then T = u.left
        delete u
    else                                              % Caso 2 (figlio destro)
        link(u.parent, u.right, k)
        if u.parent == nil then T = u.right
        delete u
return T
```

Dimostrazione (perché funziona):

- **Caso 1 – nessun figlio:** eliminare foglie non cambia l’ordine dei nodi rimanenti.
- **Caso 2 – un solo figlio:** se u è il figlio destro (sinistro) di p , tutti i valori nel sottoalbero di f sono maggiori (minori) di p ; quindi f può essere attaccato come figlio destro (sinistro) di p al posto di u .
- **Caso 3 – due figli:** il successore s è sicuramente $\geq$ dei nodi nel sottoalbero sinistro di u ed è sicuramente $\leq$ dei nodi nel sottoalbero destro di u , quindi può essere sostituito a u . A quel punto, si ricade nel Caso 2 per l’eventuale figlio destro di s .

2.17.7 Costo Computazionale

Osservazione

Tutte le operazioni sono confinate ai nodi posizionati lungo un cammino semplice dalla radice a una foglia. Posto h = altezza dell'albero, il **tempo di ricerca** è $O(h)$.

Caso pessimo: $h = O(n)$ – si verifica quando l'albero degenera in una lista (es. inserimenti in ordine già ordinato: $1(_, 2(_, 3(_, 4(_, 5(_, 6))))$), tutti figli destri).

Caso ottimo: $h = O(\log n)$ – l'albero è perfettamente bilanciato (es. $10(6(4, 8), 15(12, 18))$), lo stesso albero di riferimento della Sezione 2.17).

Domanda Aperta (verso la prossima sezione)

Dato che il costo di ogni operazione dipende da h , e che h può degenerare fino a $O(n)$: come si fa a **garantire** $h = O(\log n)$, indipendentemente dalla sequenza di inserimenti e cancellazioni? Risposta nella Sezione 2.18, con gli alberi binari di ricerca **bilanciati**.

2.18 Alberi Binari di Ricerca Bilanciati

2.18.1 Altezza degli Alberi Binari di Ricerca

Quanto vale h ? Dipende dalla sequenza di inserimenti e cancellazioni.

- **Caso pessimo:** $h = \Theta(n)$ (lista degenerare).
- **Caso medio:** per un ABR costruito per inserimenti **casuali**, è possibile dimostrare che l'altezza attesa è $\Theta(\log n)$.
- **Caso generale:** analizzare l'andamento dell'altezza per sequenze arbitrarie di inserimenti e cancellazioni è un problema aperto, difficile.

Nella Realtà

Non si analizza il caso medio sperando che vada bene: si utilizzano tecniche che **bilanciano** l'albero, **garantendo** $h = O(\log n)$ in ogni caso.

2.18.2 Alberi Binari di Ricerca Bilanciati

Definizione: Fattore di Bilanciamento

Il **fattore di bilanciamento** $\beta(v)$ di un nodo v è la differenza fra le altezze dei due sottoalberi di v .

Tre famiglie di tecniche di bilanciamento:

- **Alberi AVL** (Adelson-Velskii e Landis): $\beta(v) \in \{-1, 0, 1\}$ per ogni nodo v ; il bilanciamento è ripristinato tramite **rotazioni**.
- **B-alberi:** $\beta(v) = 0$ per ogni nodo v (bilanciamento perfetto); pensati per strutture in **memoria secondaria**.

- **Alberi 2-3 (e varianti):** $\beta(v) = 0$; il bilanciamento è ripristinato tramite operazioni di *merge* e *split* dei nodi.

In Questo Corso

Si studiano gli **alberi Red-Black**, una rappresentazione alternativa (ma equivalente) degli alberi 2-3, che si presta meglio all'implementazione su un albero binario.

2.18.3 Definizioni

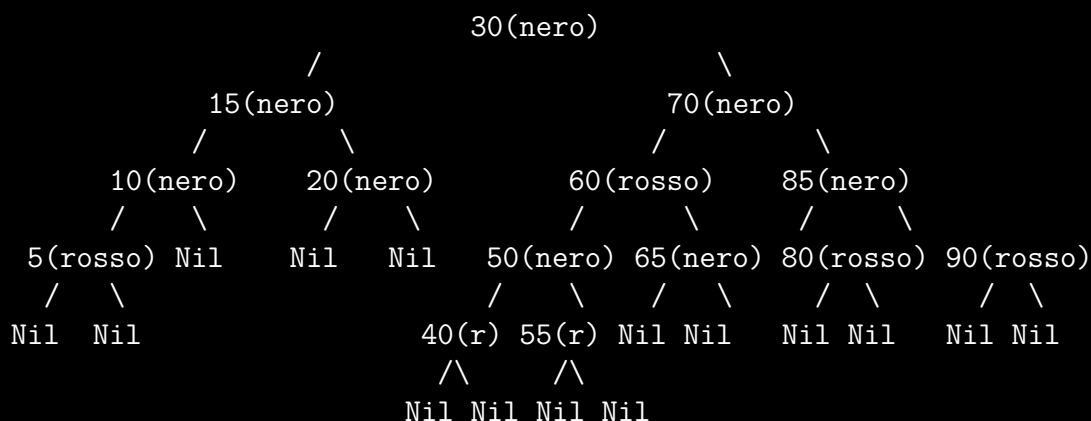
Definizione: Albero Red-Black

Un **albero Red-Black** (Guibas e Sedgwick, 1978) è un albero binario di ricerca in cui ogni nodo è colorato di **rosso** o **nero**, soggetto ai seguenti vincoli:

1. la **radice** è nera;
2. tutte le foglie (Nil) sono considerate **nere**;
3. entrambi i figli di un nodo **rosso** sono **neri** (un nodo **nero** può invece avere figli di qualunque colore);
4. ogni cammino semplice da un nodo u a una delle foglie contenute nel suo sottoalbero ha lo **stesso numero di nodi neri**.

2.18.4 Esempi

Albero di riferimento per questa sezione, colori verificati a più livelli di ingrandimento: radice 30 (nero); 15 (nero) e 70 (nero); 10 (nero) e 20 (nero) sotto 15; 60 (rosso) e 85 (nero) sotto 70; 5 (rosso) unico figlio di 10; 50 (nero) e 65 (nero) sotto 60; 80 (rosso) e 90 (rosso) sotto 85; 40 (rosso) e 55 (rosso) sotto 50; tutte le foglie Nil (nere).



Verifica dei quattro vincoli su questo esempio:

1. la radice (30) è nera. ✓
2. tutte le foglie Nil sono nere (per convenzione). ✓
3. ogni nodo rosso (5, 60, 80, 90, 40, 55) ha entrambi i figli neri – si noti che 60 (rosso) ha figlio 50 che è **nero** pur avendo a sua volta figli **rossi** (40, 55): un nodo nero può avere figli di qualunque colore. ✓

4. ogni cammino radice–foglia attraversa esattamente 4 nodi neri (contando anche la foglia `Nil` d’arrivo): ad es. 30, 15, 10, `Nil` oppure 30, 70, 60, 50, `Nil` (60 non conta, essendo rosso) oppure 30, 70, 85, `Nil`. ✓

Reality Check

Implementazioni reali basate su alberi Red-Black: `java.util.TreeMap` e `java.util.TreeSet` (Java); `std::map` e `std::set` (C++ STL); l’albero `rbtree` del kernel Linux (usato per scheduling dei processi, gestione della memoria virtuale, e altro).

2.18.5 Memorizzazione

```
TREE
TREE parent
TREE left
TREE right
int color
item key
item value
```

Nodi Nil

Nodo sentinella il cui scopo è avere accesso al colore di entrambi i figli, evitando di dover gestire casi particolari quando uno dei due è `nil`. Al posto di un puntatore `nil`, si usa un puntatore a un nodo `Nil` di colore **nero**. Ne esiste **uno solo**, condiviso da tutto l’albero, per risparmiare memoria.

2.18.6 Altezza Nera

Definizione: Altezza Nera

L’**altezza nera** $bh(v)$ di un nodo v è il numero di nodi neri lungo ogni cammino da v (escluso) a ogni foglia (**inclusa**) del suo sottoalbero. L’**altezza nera di un albero Red-Black** è pari all’altezza nera della sua radice.

Entrambe le definizioni sono ben poste perché tutti i cammini hanno lo stesso numero di nodi neri (Vincolo 4, Definizione 2.18.3).

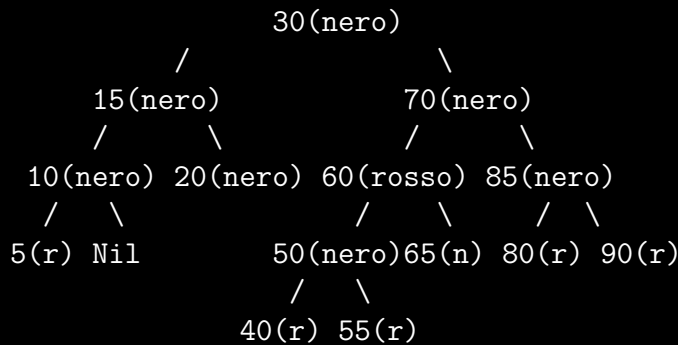
2.18.7 Esempi: Colorazioni Diverse

Più Colorazioni Sono Possibili

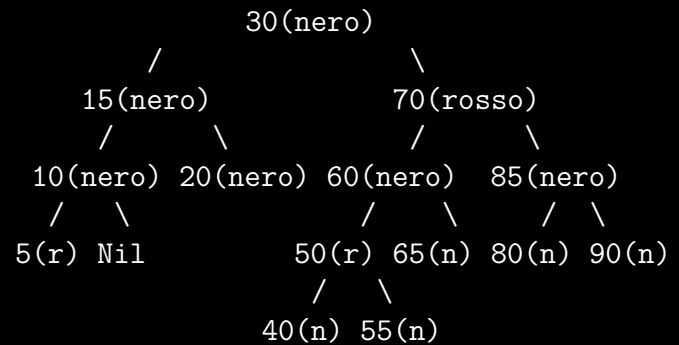
La **stessa forma** d’albero ammette colorazioni diverse, tutte valide. Due versioni verificate a piena risoluzione, entrambe con $bh(r) = 3$:

- **Versione 1** (= l’esempio della Sezione 2.18, Definizioni): 30, 15, 70, 10, 20, 65, 85 neri; 60, 5, 80, 90, 40, 55 rossi; 50 **nero**.
- **Versione 2**: stessa forma, ma 70, 50 diventano **rossi** mentre 60, 80, 90, 40, 55 diventano **neri** (15, 10, 20, 85, 5 restano invariati).

Versione 1



Versione 2



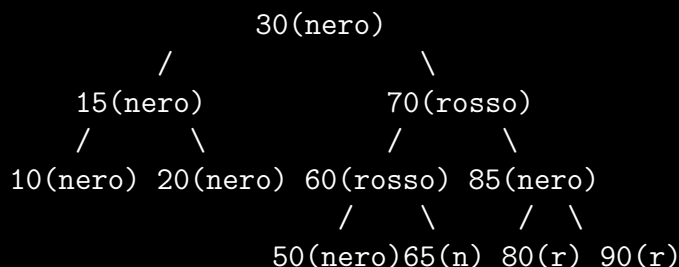
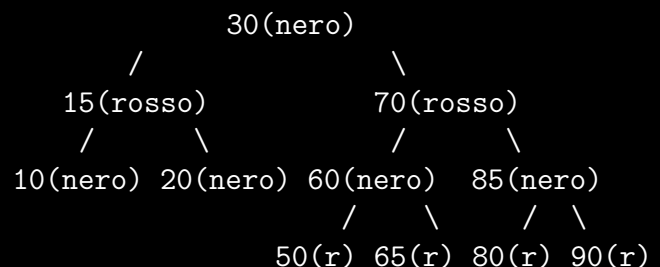
(abbreviazioni: nero=*n*, rosso=*r*; figli *Nil* di 5, 20, 40, 55, 65, 80, 90 omessi per compattezza.)

In entrambe le versioni, ogni cammino radice–foglia attraversa esattamente 3 nodi neri *esclusa la radice*: ad es. in Versione 2, $30 \rightarrow 70 \rightarrow 60 \rightarrow 50 \rightarrow 40 \rightarrow \text{Nil}$ ha, esclusa la radice, i nodi neri 60, 40, Nil.

Cambiare Colorazione Può Cambiare l'Altezza Nera

Su una forma d'albero più semplice (senza i nipoti di 10), due colorazioni ugualmente **valide** (tutti e quattro i vincoli rispettati in entrambe) producono **altezze nere diverse**: verificato a piena risoluzione.

- $bh(r) = 3$: 30, 10, 20, 50, 65, 85 neri; 15 (qui) nero; 70, 60, 80, 90 rossi².
- $bh(r) = 2$: 30, 10, 20, 60, 85 neri; 15, 70, 50, 65, 80, 90 rossi.

 $bh(r) = 3$  $bh(r) = 2$ 

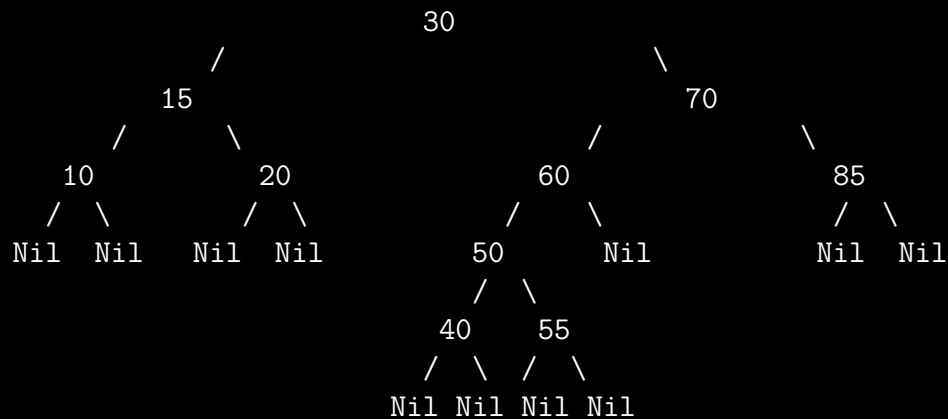
Perché Succede

Nella colorazione $bh = 2$, lungo il cammino $30 \rightarrow 15 \rightarrow 10 \rightarrow \text{Nil}$ (esclusa la radice) i nodi neri sono 10, Nil: il nodo 15, diventato rosso, non contribuisce più al conteggio. “Scaricare” di rosso un nodo nero riduce di 1 l'altezza nera di ogni cammino che lo attraversa – a patto di farlo in modo uniforme su *tutti* i cammini, per non violare il Vincolo 4.

2.18.8 Esempio: Questo Albero Può Essere un Albero Red-Black?

Si consideri la forma d'albero $30(15(10, 20), 70(60(50(40, 55), _), 85))$, inizialmente **non colorata**:

²In questa versione semplificata (senza i nipoti 40, 55), la colorazione di partenza ha 60 rosso anziché nero: una terza distribuzione di colori diversa, ancora valida, ancora con $bh = 3$ – ulteriore conferma che la stessa forma ammette molte colorazioni corrette.



Passo 1: un Vincolo sulla Possibile Altezza Nera

Guardando il lato sinistro, $2 \leq bh(r) \leq 3$; guardando il lato destro, $bh(r) \geq 3$. Quindi, se questo albero ammette una colorazione valida, deve necessariamente avere $bh(r) = 3$.

Tentativo 1: 60 e 70 Entrambi Neri

Impossibile! Perché dovremmo violare il Vincolo 3: con 70, 60 entrambi neri, per raggiungere $bh = 3$ sul cammino $70 \rightarrow 60 \rightarrow 50 \rightarrow 40 \rightarrow \text{Nil}$ resterebbe un solo “slot nero” per 50, 40, Nil insieme – che forzerebbe 50 e 40 entrambi rossi, ma un nodo rosso (50) non può avere un figlio rosso (40).

Tentativo 2: 60 Rosso

Impossibile per il Vincolo 4! Esistono cammini con 2 nodi neri (es. $70 \rightarrow 60 \rightarrow \text{Nil}$, dove 60 essendo rosso non conta: 70, Nil) e cammini con 3 nodi neri (es. $70 \rightarrow 60 \rightarrow 50 \rightarrow 40 \rightarrow \text{Nil}$): valori diversi sullo stesso sottoalbero.

Tentativo 3: 70 Rosso

Impossibile per il Vincolo 4! Stesso problema: esistono cammini con 2 nodi neri (es. $70 \rightarrow 85 \rightarrow \text{Nil}$, dove 70 rosso non conta: 85, Nil) contro cammini più lunghi che ne richiederebbero 3.

Tentativo 4 (Ultima Possibilità): Radice Rossa

Si prova a colorare di rosso *quasi* tutto l'albero (30, 15, 70, 10, 20, 60, 85, 40, 55 rossi, solo 50 nero), nel disperato tentativo di minimizzare i nodi neri in modo uniforme. **Impossibile:** non rispetta il Vincolo 1 (la radice 30 è rossa, non nera) e non rispetta comunque il Vincolo 4.

Conclusione

Questo albero NON può essere un albero Red-Black! Non tutte le forme di albero binario ammettono una colorazione valida: la struttura stessa dell'albero può essere incompatibile con i quattro vincoli, indipendentemente da come si distribuiscono i colori.

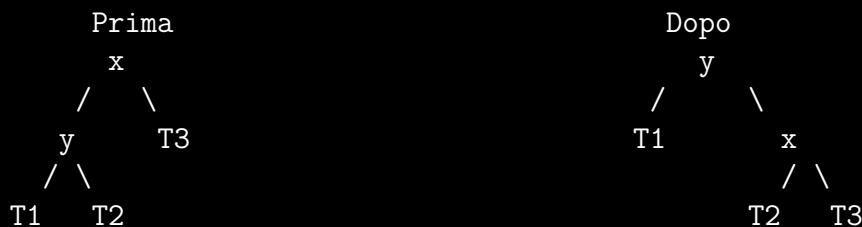
2.18.9 Rotazioni

Quando un'Operazione Viola i Vincoli

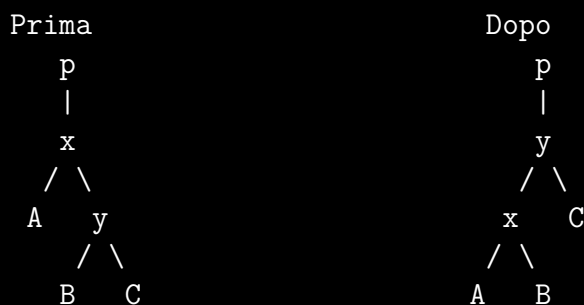
È possibile che le condizioni di bilanciamento (i quattro vincoli della Definizione 2.18.3) risultino violate dopo un inserimento o una cancellazione. Si può agire in due modi:

- modificando i **colori** nella zona della violazione;
- operando dei **ribilanciamenti** dell'albero tramite **rotazioni** (destra o sinistra).

Rotazione a Destra. *Nota grafica – verificata:* **Prima:** x è radice del sottoalbero (altezza $h + 2$), con figlio sinistro y (altezza $h + 1$, a sua volta con sottoalberi T_1 e T_2) e figlio destro il sottoalbero T_3 (altezza h). **Dopo:** y è la nuova radice (altezza $h + 1$), con figlio sinistro T_1 e figlio destro x ; x ha ora come figli T_2 (sinistro) e T_3 (destro).



Rotazione a Sinistra. *Nota grafica – verificata, stato “Prima”/“Dopo”:* p è il padre di x ; x ha figlio sinistro A e figlio destro y ; y ha figlio sinistro B e figlio destro C $\rightarrow$ dopo la rotazione, p ha come figlio y ; y ha figlio sinistro x e figlio destro C (invariato); x ha figlio sinistro A (invariato) e figlio destro B (il sottoalbero “di mezzo”, che passa da y a x).



```

TREE rotateLeft(TREE x)
TREE y = x.right
TREE p = x.parent
(1) x.right = y.left           % Il sottoalbero B diventa figlio destro di x
(1) if y.left != nil then y.left.parent = x
(2) y.left = x                 % x diventa figlio sinistro di y
(2) x.parent = y
(3) y.parent = p               % y diventa figlio di p
(3) if p != nil then
    if p.left == x then p.left = y else p.right = y
return y

```

Operazioni: (1) far diventare B figlio destro di x ; (2) far diventare x il figlio sinistro di y ; (3) far diventare y figlio di p , il vecchio padre di x .

Perché Funziona

Le chiavi nel sottoalbero B sono maggiori della chiave in x e vengono correttamente spostate nel sottoalbero destro di x : la proprietà di albero binario di ricerca (Proprietà 2.17.1) resta rispettata dopo la rotazione.

Costo e Simmetria

Il costo di una rotazione è ovviamente $O(1)$: coinvolge un numero costante di riassegnazioni di puntatori. La rotazione destra `rotateRight()` si ottiene semplicemente scambiando fra loro ogni occorrenza delle variabili `left` e `right` nel codice di `rotateLeft()` – le due operazioni sono l’una lo speculare dell’altra, esattamente come si è visto per `successorNode/predecessorNode` (Sezione 2.17).